Phân khúc khách hàng dựa trên hành vi mua sắm

Project: Customer Segmentation with K-means Clustering

Instructor: Nguyễn Thái Hà

TA: Nguyễn Thọ Anh Khoa

STA: Nguyễn Phúc Thịnh

[Link souce code]

[Link QA]

Customer Segmentation with K-means Clustering
K-means Algorithm
Cluster Profiles
Targeted Campaigns

Outline bài giảng

Phần 1: Giới thiệu & bối cảnh business

- Bối cảnh thách thức
- Mục tiêu dự án
- Thách thức & Cơ hội từ dữ liệu
- Phạm vi & Kỳ vọng dự án

Phần 2: Nền tảng lý thuyết & Hướng tiếp cận

- Supervised vs Unsupervised Learning
- Bài toán phân cụm khách hàng
- Giới thiệu các phương pháp phân cụm
- Giới thiệu các phương pháp Transformation

Phần 3: Triển khai dự án

- Cấu trúc dự án
- Feature Engineering
- Transformation
- Phạm vi & Kỳ vọng

Phần 4: Kết quả của dự án

- EDA: Insights chính
- Kết quả phân cụm
- Đánh giá kết quả phân cụm
- Hướng mở rộng trong tương lai

Outline bài giảng

Phần 1: Giới thiệu & bối cảnh business

- Bối cảnh thách thức
- Mục tiêu dự án
- Thách thức & Cơ hội từ dữ liệu
- Phạm vi & Kỳ vọng dự án

Phần 2: Nền tảng lý thuyết & Hướng tiếp cận

- Supervised vs Unsupervised Learning
- Bài toán phân cụm khách hàng
- Giới thiệu các phương pháp phân cụm
- Giới thiệu các phương pháp Transformation

Phần 3: Triển khai dự án

- Cấu trúc dự án
- Feature Engineering
- Transformation
- Phạm vi & Kỳ vọng

Phần 4: Kết quả của dự án

- EDA: Insights chính
- Kết quả phân cụm
- Đánh giá kết quả phân cụm
- Hướng mở rộng trong tương lai

Bối cảnh & Mục tiêu của dự án

BỐI CẢNH

- Dữ liệu khách hàng phong phú, nhưng **chưa được khai thác tối ưu dữ liệu**
- **Chi phí marketing cao** trong khi **hành vi khách hàng ngày càng đa dạng và phức tạp**.
- Marketing **"one-size-fits-all"** không còn hiệu quả.

MỤC TIÊU

- **Hiểu sâu khách hàng:** Phân tích patterns ẩn trong dữ liệu giao dịch
- **Tối ưu marketing**: Cá nhân hóa chiến lược cho từng nhóm khách hàng
- Phân khúc khách hàng dựa trên **hành vi mua sắm thực tế** để tối ưu chiến lược CRM, chiến dịch marketing, danh mục sản phẩm và phân bổ ngân sách hiệu quả.

Giới thiệu về dữ liệu

Nguồn dữ liệu

- UCI Online Retail Dataset
- Doanh nghiệp: UK-based online retail company (non-store)
- Ngành hàng: Quà tặng & đồ gia dụng độc đáo

Phạm vi thời gian

- 01/12/2010 – 09/12/2011
- Gần một năm dữ liệu giao dịch liên tục
- Dủ để phân tích xu hướng và mùa vụ.

Quy mô ban đầu

- 541,909 giao dịch
- ~4,400 khách hàng
- ~4,000 sản phẩm
- 37 quốc gia

Sau làm sạch

- ~397K giao dịch hợp lệ (~540K giao dịch tổng)
- ~4,373 khách hàng tại UK
- Tập trung vào thị trường chính
- Phân tích sâu và chính xác hơn.

Cấu trúc trường dữ liệu chính

InvoiceNo

Mô tả: Mã hóa đơn (6 chữ số)
Ví dụ: 536365

StockCode

Mô tả: Mã sản phẩm
Ví dụ: 85123A

Description

Mô tả: Tên sản phẩm
Ví dụ: WHITE HANGING HEART

Quantity

Mô tả: Số lượng mua
Ví dụ: 6

InvoiceDate

Mô tả: Thời gian giao dịch
Ví dụ: 2010-12-01 08:26:00

UnitPrice

Mô tả: Đơn giá (GBP)
Ví dụ: 2.55

CustomerID

Mô tả: ID khách hàng
Ví dụ: 17850

Country

Mô tả: Quốc gia
Ví dụ: United Kingdom

Thách thức & Cơ hội từ dữ liệu

Thách thức

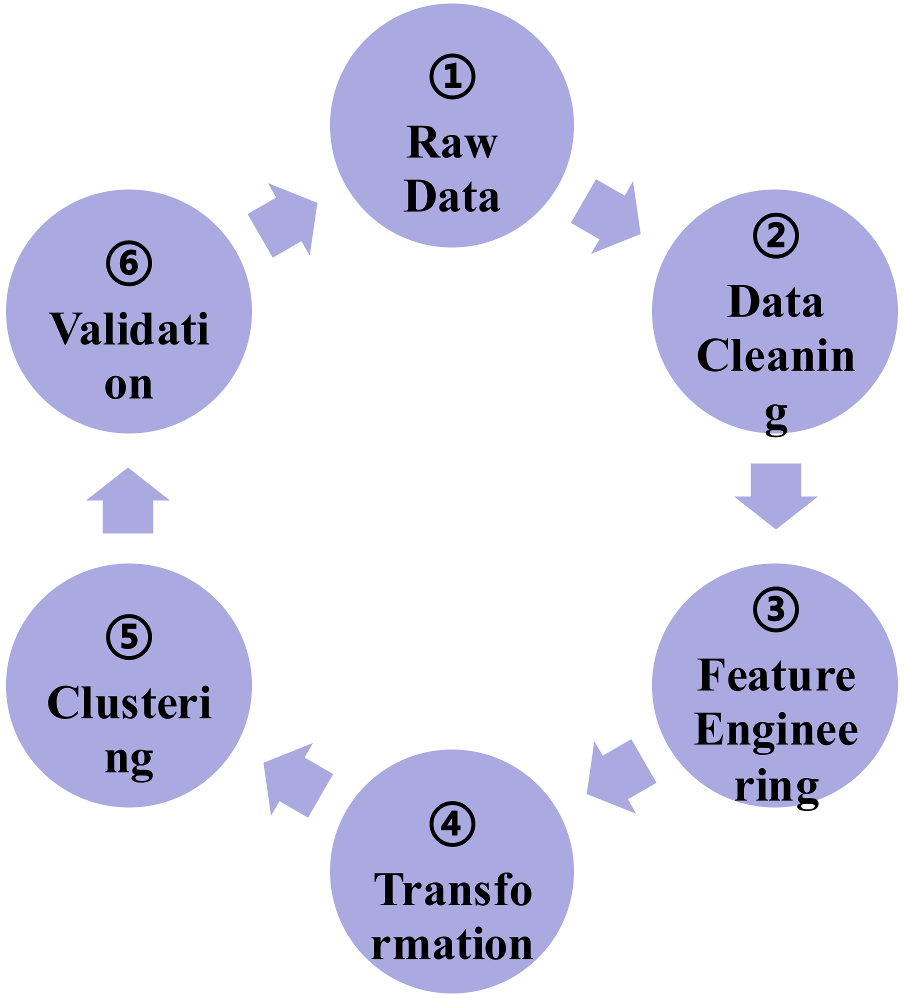
- **Thiếu CustomerID** trong nhiều giao dịch.
- **Quantity âm** (đơn hủy/hoàn trả) cần xử lý riêng.
- **Outliers cực trị** ảnh hưởng phân tích.
- **Phân phối lệch (skewed)** cần biến đổi, cân bằng.

Cơ hội từ dữ liệu

- Dữ liệu giao dịch chi tiết, **đa chiều**.
- Có **time series** để phân tích xu hướng.
- **Đa dạng sản phẩm** mở ra nhiều hướng phân tích.
- Phù hợp phân tích **hành vi** ở nhiều góc độ.
- **Dữ liệu thực tế** từ doanh nghiệp vận hành.

Phạm vi & Kỳ vọng dự án

Quy trình end-to-end solution



① Raw Data

② Data Cleaning

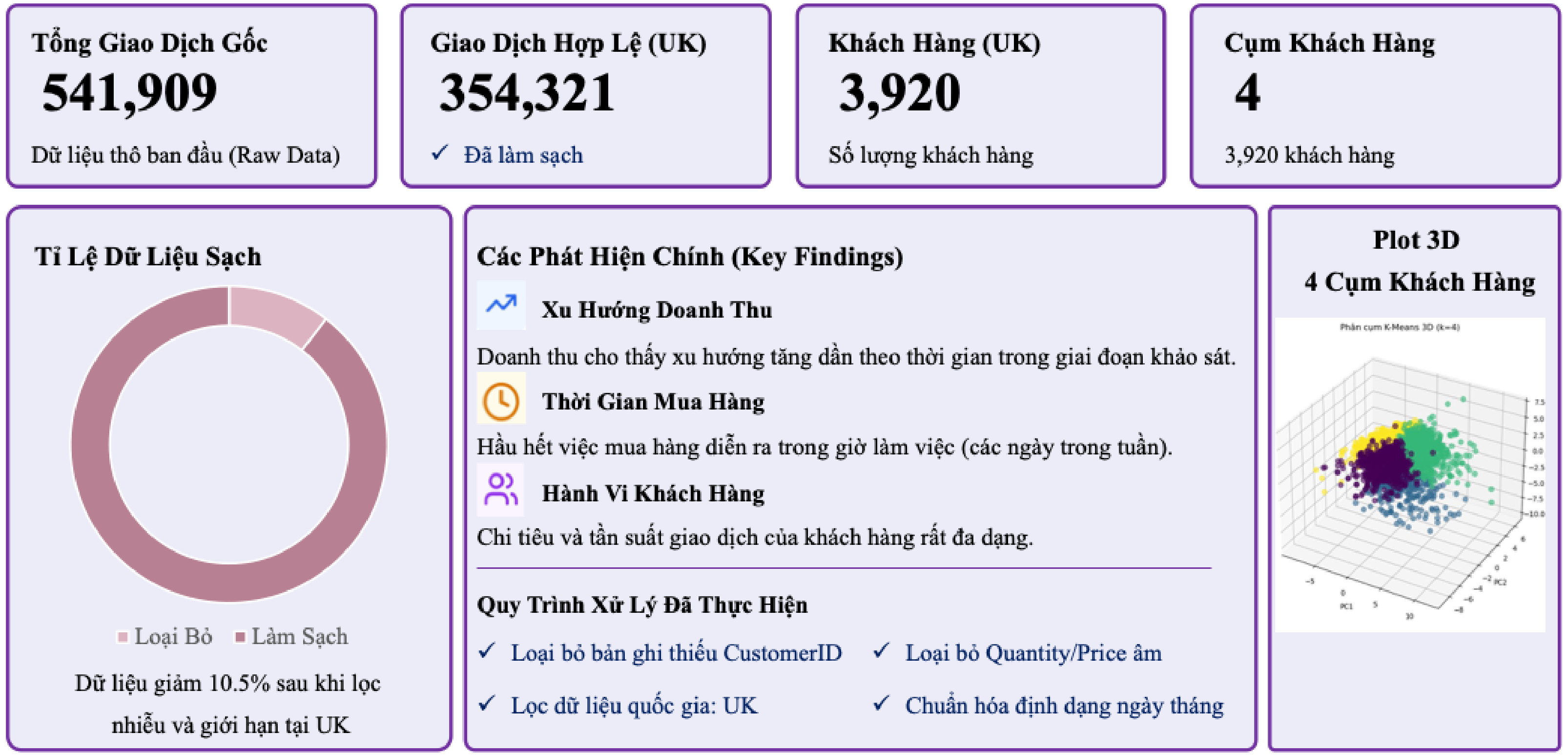
③ Feature Engineering

④ Transformation

⑤ Clustering

⑥ Validation

Kết quả kỳ vọng



Tổng Giao Dịch Gốc
541,909
Dữ liệu thô ban đầu (Raw Data)

Giao Dịch Hợp Lệ (UK)
354,321
✓ Đã làm sạch

Khách Hàng (UK)
3,920
Số lượng khách hàng

Cụm Khách Hàng
4
3,920 khách hàng

Tỉ Lệ Dữ Liệu Sạch

▪ Loại Bỏ ▪ Làm Sạch

Dữ liệu giảm 10.5% sau khi lọc nhiễu và giới hạn tại UK

Các Phát Hiện Chính (Key Findings)

Xu Hướng Doanh Thu
Doanh thu cho thấy xu hướng tăng dần theo thời gian trong giai đoạn khảo sát.

Thời Gian Mua Hàng
Hầu hết việc mua hàng diễn ra trong giờ làm việc (các ngày trong tuần).

Hành Vi Khách Hàng
Chi tiêu và tần suất giao dịch của khách hàng rất đa dạng.

Quy Trình Xử Lý Đã Thực Hiện

- ✓ Loại bỏ bản ghi thiếu CustomerID
- ✓ Lọc dữ liệu quốc gia: UK
- ✓ Loại bỏ Quantity/Price âm
- ✓ Chuẩn hóa định dạng ngày tháng

Plot 3D
4 Cụm Khách Hàng

Chi tiết từng bước (bức tranh tổng quan)

01

Raw Data

Dữ liệu giao dịch gốc 541K records

02

Data Cleaning

Loại bỏ noise, xử lý missing values, chuẩn hoá định dạng

03

Feature Engineering

Tạo 16 features toàn diện cấp khách hàng

04

Transformation

Box-Cox normalization + StandardScaler

05

Clustering

K-means với tối ưu số cụm

06

Validation

Đánh giá chất lượng và diễn giải business

Tóm tắt: Giới thiệu & Bối cảnh Business

1 Bối cảnh thách thức

Dữ liệu khách hàng phân tán, marketing "one-size-fits-all" không hiệu quả, ROI không tỷ lệ thuận với budget.

2 Vấn đề cần giải quyết

Cần phân khúc khách hàng để tối ưu chiến lược marketing và tăng ROI.

3 Dữ liệu

UCI Online Retail Dataset - bán lẻ trực tuyến tại Anh, 541K bản ghi, 8 trường chính.

4 Thách thức kỹ thuật

Missing CustomerID, quantity âm, outliers cực trị.

5 Hướng tiếp cận

Quy trình end-to-end từ dữ liệu thô → features → clustering → validation → business insights.

6 Kỳ vọng

Phân khúc khách hàng có thể hành động được với 4 cụm rõ ràng.

Outline bài giảng

Phần 1: Giới thiệu & bối cảnh business

- Bối cảnh thách thức
- Mục tiêu dự án
- Thách thức & Cơ hội từ dữ liệu
- Phạm vi & Kỳ vọng dự án

Phần 2: Nền tảng lý thuyết & Hướng tiếp cận

- Supervised vs Unsupervised Learning
- Bài toán phân cụm khách hàng
- Giới thiệu các phương pháp phân cụm
- Giới thiệu các phương pháp Transformation

Phần 3: Triển khai dự án

- Cấu trúc dự án
- Feature Engineering
- Transformation
- Phạm vi & Kỳ vọng

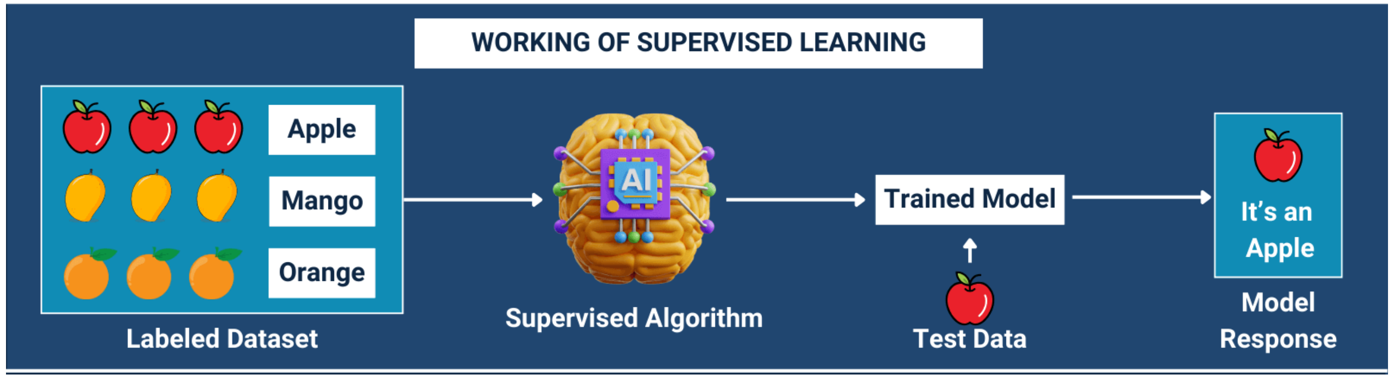
Phần 4: Kết quả của dự án

- EDA: Insights chính
- Kết quả phân cụm
- Đánh giá kết quả phân cụm
- Hướng mở rộng trong tương lai

Supervised vs Unsupervised Learning

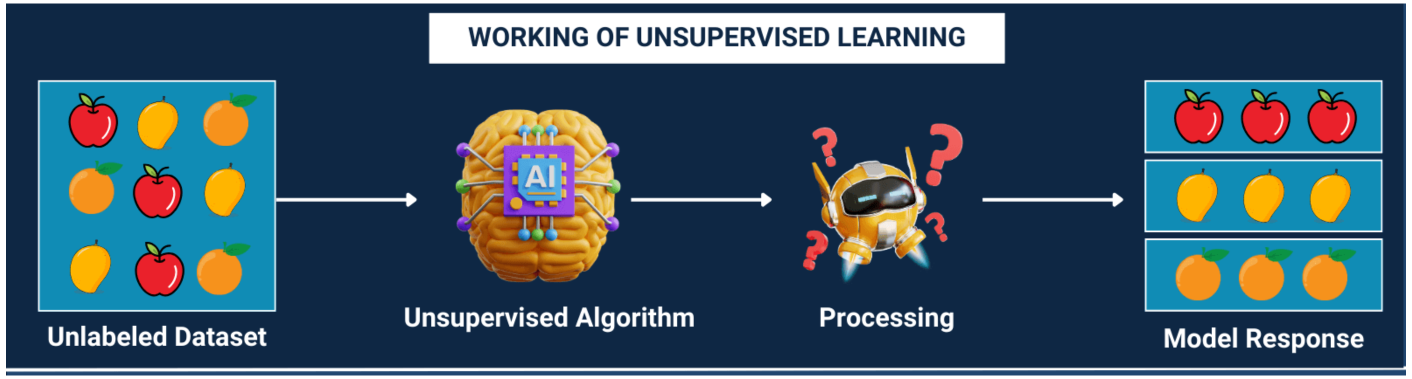
Supervised Learning

- ✓ **Đặc điểm:** Dữ liệu có nhãn (đã biết kết quả)
- ✓ **Mục tiêu:** Tối ưu khả năng dự đoán hoặc giải thích nguyên nhân
- ✓ **Ví dụ:** Dự đoán churn, phân loại theo nhãn định sẵn, regression
- ✓ **Validation:** Accuracy, Precision, Recall, F1-score


WORKING OF SUPERVISED LEARNING
Apple
Mango
Orange
Labeled Dataset
AI
Supervised Algorithm
Trained Model
Test Data
It's an Apple
Model Response

Unsupervised Learning

- ✓ **Đặc điểm:** : Không có nhãn, tìm cấu trúc tự nhiên
- ✓ **Mục tiêu:** Clustering, dimensionality reduction, pattern discovery
- ✓ **Ví dụ:** Phân khúc khách hàng, phát hiện anomaly, topic modeling
- ✓ **Validation:** Silhouette Score, Elbow Method, business interpretation


WORKING OF UNSUPERVISED LEARNING
Unlabeled Dataset
AI
Unsupervised Algorithm
Processing
Model Response

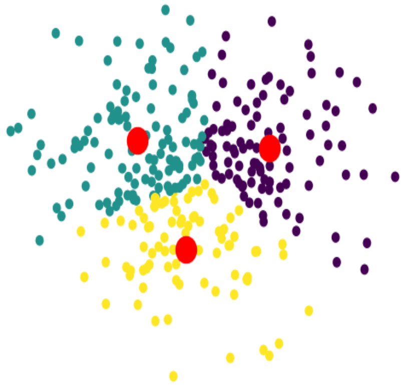
Nguồn ảnh: https://futureskillsacademy.com/blog/supervised-and-unsupervised-learning/

Bài toán phân cụm khách hàng

Unsupervised Learning cung cấp nhiều phương pháp tiếp cận phân cụm, mỗi phương pháp có ưu điểm riêng phù hợp với từng loại dữ liệu và yêu cầu business.

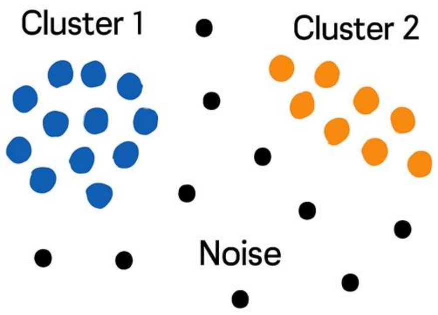
K-means

Phân cụm dựa trên khoảng cách Euclidean, tối ưu WCSS. **Nhanh, đơn giản, dễ diễn giải**.



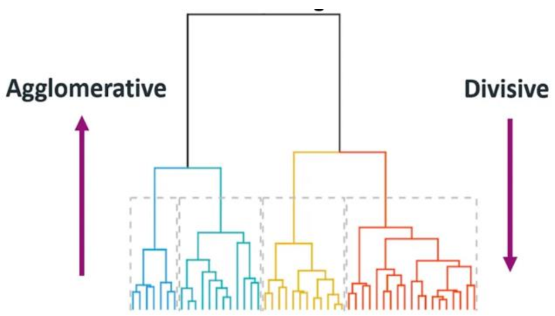
DBSCAN

Phân cụm dựa trên mật độ, tự động phát hiện outliers. Phù hợp với **cụm hình dạng bất thường**.


Cluster 1
Cluster 2
Noise

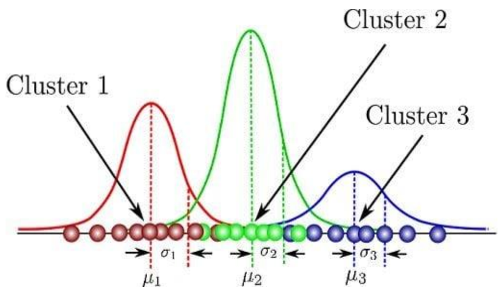
Hierarchical Clustering

Xây dựng cây phân cấp (dendrogram), không cần chọn k trước. Hữu ích cho **phân tích cấu trúc lồng nhau**.


Agglomerative
Divisive

Gaussian Mixture

Mô hình xác suất, cho phép điểm thuộc nhiều cụm với độ tin cậy khác nhau. Phù hợp **cụm chồng lấn**.


Cluster 1
Cluster 2
Cluster 3
σ1
σ2
σ3
μ1
μ2
μ3

Lựa chọn trong dự án: K-means vì tính đơn giản, hiệu quả tính toán cao, và khả năng diễn giải centroid rõ ràng cho business users.

K-means Algorithm

K-means là thuật toán phân cụm phổ biến nhất, chia dữ liệu thành k cụm sao cho các điểm trong cùng cụm có khoảng cách gần nhau nhất, và xa các cụm khác nhất.

Mục tiêu tối ưu

Tối thiểu **WCSS (Within-Cluster Sum of Squares)** - tổng bình phương khoảng cách từ mỗi điểm đến centroid của cụm chứa nó.
Minimize WCSS:

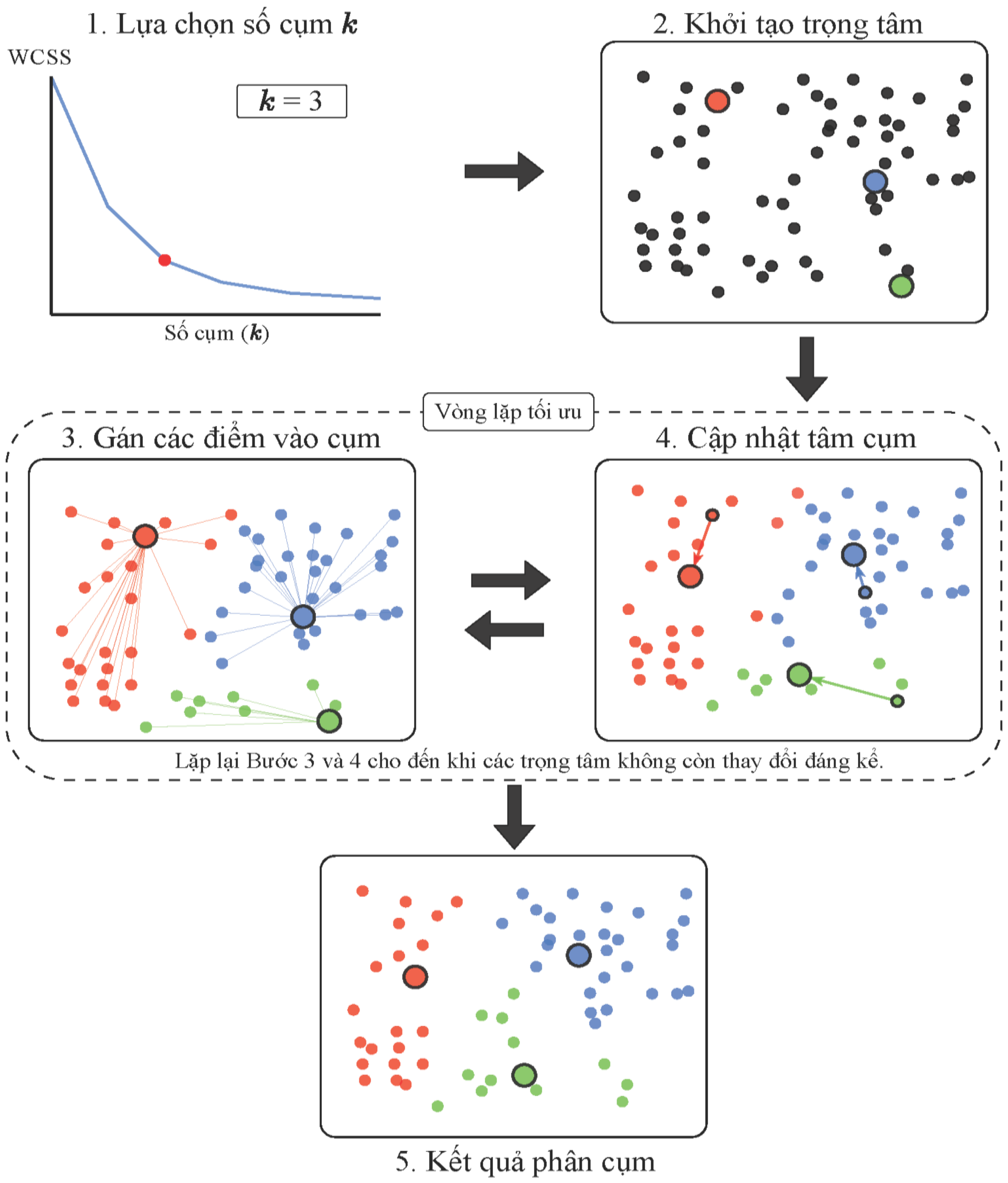
$$J = \Sigma^{k}_{i=1} \Sigma_{x} \in C_{i} \, ||x - \mu_{i}||^{2}$$

Cơ chế lặp

Khởi tạo k centroids ngẫu nhiên → Gán điểm vào cụm gần nhất → Cập nhật centroid → Lặp lại cho tới hội tụ. Sử dụng khoảng cách **Euclidean**.

Chọn số cụm k

Sử dụng **Elbow Method** (quan sát điểm "gối") và **Silhouette Score** (đo độ tách biệt).


1. Lựa chọn số cụm k
WCSS
k = 3
Số cụm (k)
2. Khởi tạo trọng tâm
Vòng lặp tối ưu
3. Gán các điểm vào cụm
4. Cập nhật tâm cụm
Lặp lại Bước 3 và 4 cho đến khi các trọng tâm không còn thay đổi đáng kể.
5. Kết quả phân cụm

K-means: Ưu điểm & Hạn chế

Ưu điểm nổi bật

- ✓ **Đơn giản & trực quan:** Dễ hiểu, dễ giải thích cho business users
- ✓ **Hiệu suất cao:** Tốc độ nhanh, mở rộng tốt với dữ liệu lớn **(O(n·k·d·i))**
- ✓ **Centroid rõ ràng:** Dễ dàng diễn giải đặc trưng trung bình của từng cụm
- ✓ **Phù hợp dữ liệu cân bằng:** Hoạt động tốt khi cụm có kích thước và hình dạng tương đối đều

Hạn chế & cách khắc phục

- × **Cần chọn k trước:** Khắc phục bằng Elbow + Silhouette
- × **Nhạy với outliers:** Khắc phục bằng Box-Cox transformation
- × **Nhạy với khởi tạo:** Khắc phục bằng n_init=10 (chạy nhiều lần)
- × **Giả định cụm "đều":** Khắc phục bằng chuẩn hoá + scale dữ liệu

Why Box-Cox Transformation?

Box-Cox là kỹ thuật biến đổi dữ liệu để làm cho phân phối gần với phân phối chuẩn (normal distribution) hơn, giúp cải thiện hiệu quả của các thuật toán Machine Learning.

Vấn đề phân phối lệch

Dữ liệu hành vi mua sắm thường có **phân phối lệch phải** (right-skewed) với nhiều giá trị nhỏ và một số giá trị cực lớn.

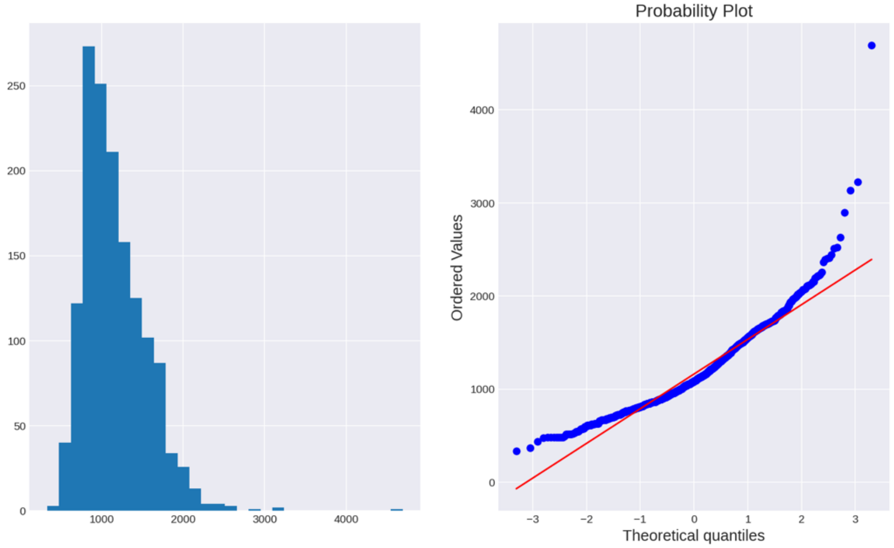
Thang đo khác nhau

Các features có **đơn vị và thang đo khác nhau** (số lượng, giá tiền, tần suất) → K-means nhạy cảm với thang đo.

Lợi ích của Box-Cox

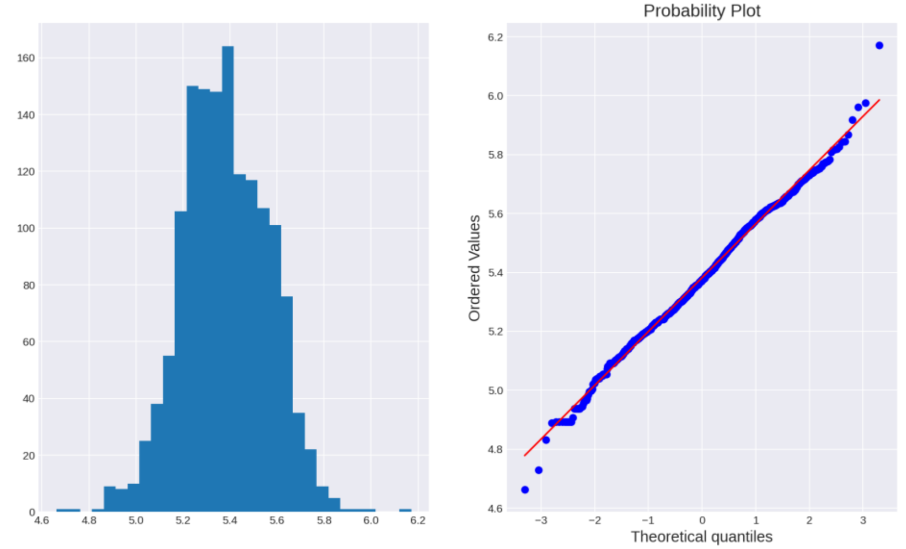
Giúp **chuẩn hoá phân phối**, **ổn định phương sai**, và hạn chế ảnh hưởng của outliers trước khi scaling.

Before Transformation



$$y_i^{(\lambda)} = \begin{cases} \dfrac{y_i^{\lambda} - 1}{\lambda} & \text{if } \lambda \neq 0, \\ \ln(y_i) & \text{if } \lambda = 0, \end{cases}$$

After Transformation



Tham khảo: Công thức & Quy trình Box-Cox

Công thức toán học

$$y_i^{(\lambda)} = \begin{cases} \dfrac{y_i^{\lambda} - 1}{\lambda} & \text{if } \lambda \neq 0, \\ \ln(y_i) & \text{if } \lambda = 0, \end{cases}$$

Trong đó λ là tham số được tối ưu bằng phương pháp Maximum Likelihood Estimation (MLE).

Xử lý giá trị ≤0

Dịch chuyển dữ liệu bằng cách cộng 1 (hoặc giá trị nhỏ) để đảm bảo tất cả giá trị > 0

Tối ưu λ

Sử dụng MLE để tìm giá trị λ tối ưu cho từng feature, tối đa hoá likelihood

Biến đổi

Áp dụng công thức Box-Cox với λ đã tối ưu cho toàn bộ features

Scaling

Chuẩn hoá dữ liệu sau biến đổi bằng StandardScaler (mean=0, std=1)

PCA: Mục đích & Sử dụng

PCA là kỹ thuật giảm chiều dữ liệu, biến đổi tuyến tính các features tương quan thành tập các thành phần chính (principal components) không tương quan, sắp xếp theo độ quan trọng giảm dần.

Mục đích trong dữ liệu

Giảm chiều dữ liệu

Giảm số lượng biến mà vẫn giữ lại thông tin quan trọng

Giảm nhiễu

Loại bỏ thành phần nhiễu và giữ lại các thành phần chính

Trực quan hóa

Biến đổi dữ liệu chiều cao thành 2D hoặc 3D để dễ quan sát

Lựa chọn thành phần

Tiêu chí

- Dựa trên explained variance ratio
- Dựa trên phương sai tích lũy

Khuyến nghị

- Giữ lại 80-90% tổng phương sai
- Số thành phần tùy thuộc vào mục đích phân tích

PCA dùng cho **visualization để đánh giá chất lượng phân cụm trực quan**, không thay thế 16 features gốc trong clustering.

Outline bài giảng

Phần 1: Giới thiệu & bối cảnh business

- Bối cảnh thách thức
- Mục tiêu dự án
- Thách thức & Cơ hội từ dữ liệu
- Phạm vi & Kỳ vọng dự án

Phần 2: Nền tảng lý thuyết & Hướng tiếp cận

- Supervised vs Unsupervised Learning
- Bài toán phân cụm khách hàng
- Giới thiệu các phương pháp phân cụm
- Giới thiệu các phương pháp Transformation

Phần 3: Triển khai dự án

- Cấu trúc dự án
- Feature Engineering
- Transformation
- Phạm vi & Kỳ vọng

Phần 4: Kết quả của dự án

- EDA: Insights chính
- Kết quả phân cụm
- Đánh giá kết quả phân cụm
- Hướng mở rộng trong tương lai

Cấu trúc Repository

Tổng quan tổ chức mã nguồn và tài liệu dự án

notebooks/

- 01_cleaning_eda.ipynb
- 02_feature_eng.ipynb
- 03_modeling.ipynb

Quy trình phân tích từng bước, dễ theo dõi và tái hiện

src/

- clustering_lib.py

CORE FUNCTIONS

Data Cleaning Feature Eng

PCA K-means

Đóng gói logic tái sử dụng, dễ bảo trì

docs/

- project_description.md
- requirements.txt

Tài liệu chi tiết và dependencies để tái tạo môi trường

Design patterns: Áp dụng **OOP** với trách nhiệm rõ ràng và **Pipeline pattern** có thể compose linh hoạt.

Cấu trúc Notebook thực thi

01_cleaning_and_eda.ipynb

- Load 541K records
- Clean (loại 'C', UK-only, xử lý CustomerID)
- EDA (xu hướng theo thời gian, top sản phẩm, phân bố địa lý, outliers)

02_feature_engineering.ipynb

- Tạo RFM baseline
- Tạo 16 features toàn diện
- Phân tích phân phối
- Box-Cox transformation
- StandardScale
- PCA variance analysis

03_modeling.ipynb

- Chọn k tối ưu (Elbow, Silhouette, optional Gap)
- Train K-means với best k
- Validate (Silhouette, WCSS, PCA viz)
- Phân tích centroid
- Business interpretation
- Actionable insights

Phương châm thiết kế class cho dự án

clustering_library.py: Thư viện được thiết kế để thực hiện phân khúc khách hàng từ dữ liệu giao dịch bán lẻ. Thư viện này bao gồm các công cụ để làm sạch dữ liệu, tạo đặc trưng, phân cụm và trực quan hóa kết quả.

Class 1: DataCleaner

Mục đích: Xử lý và làm sạch dữ liệu giao dịch bán lẻ từ file raw

Chức năng chính:

- Load dữ liệu từ file CSV
- Làm sạch dữ liệu (loại bỏ missing values, invalid records)
- Tạo các time features
- Tính toán RFM metrics (Recency, Frequency, Monetary)
- Save data đã được clean vào folder cụ thể

Class 2: FeatureEngineer

Mục đích: Tạo các features tổng hợp ở mức khách hàng từ dữ liệu giao dịch

Chức năng chính:

- Load clean data và chuẩn bị cho feature engineering
- Tổng hợp features hành vi mua sắm cho mỗi khách hàng
- Áp dụng Box-Cox và standardization để chuẩn hóa phân phối
- Vẽ box plots và histograms cho các features
- Save các feature đã được xử lý

Phương châm thiết kế class cho dự án

clustering_library.py: Thư viện được thiết kế để thực hiện phân khúc khách hàng từ dữ liệu giao dịch bán lẻ. Thư viện này bao gồm các công cụ để làm sạch dữ liệu, tạo đặc trưng, phân cụm và trực quan hóa kết quả.

Class 3: ClusterAnalyzer

Mục đích: Thực hiện phân tích clustering và visualization

Chức năng chính:

- Áp dụng PCA để giảm chiều dữ liệu
- Tìm số cluster tối ưu (Elbow method, Silhouette score)
- Thực hiện K-means clustering
- Visualization clusters trong không gian 2D/3D
- Tạo radar charts để phân tích đặc điểm clusters

Class 4: DataVisualizer

Mục đích: Tạo các visualization để phân tích dữ liệu

Chức năng chính:

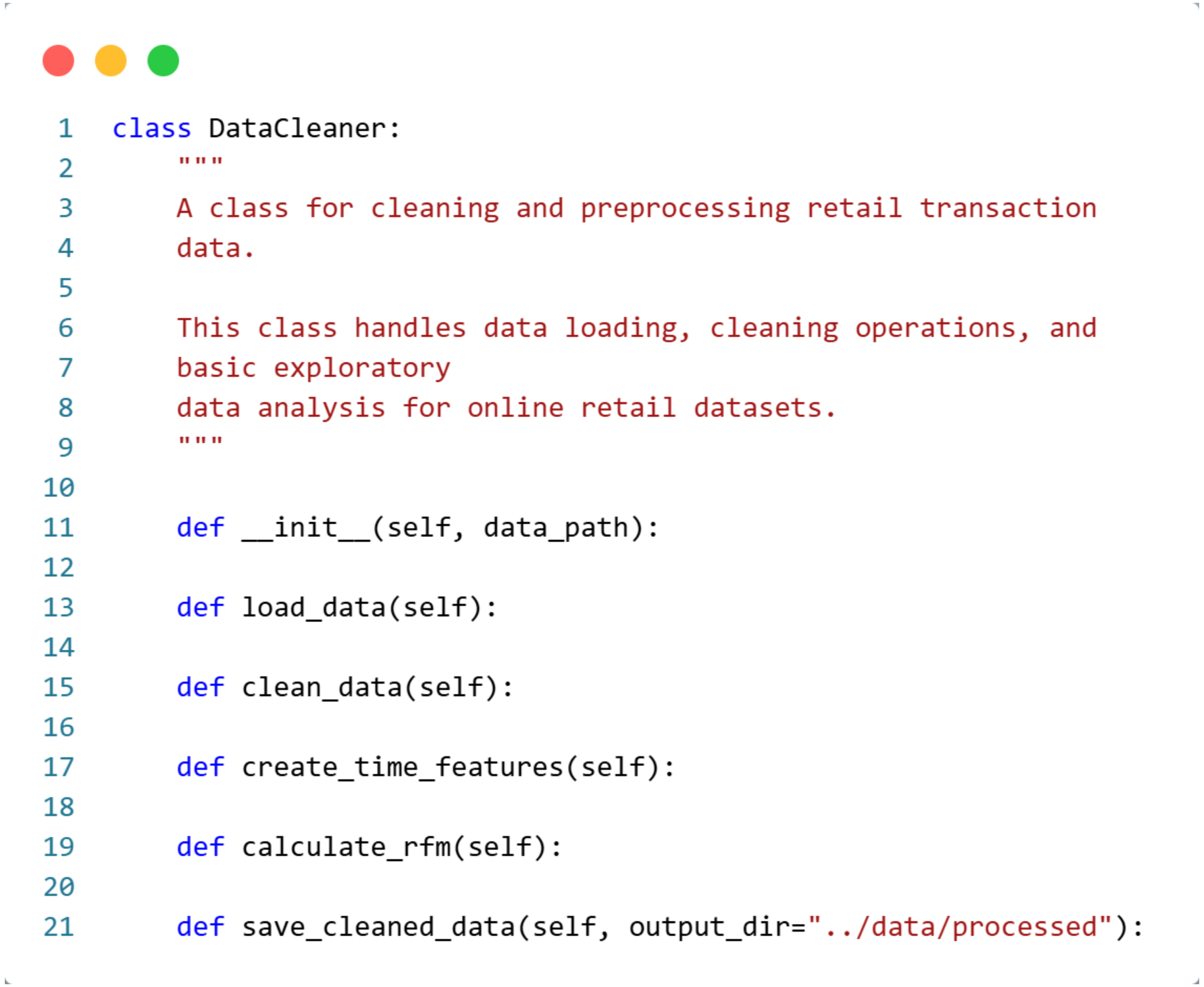
- Vẽ biểu đồ doanh thu theo thời gian
- Vẽ phân tích patterns theo ngày/giờ
- Vẽ phân tích sản phẩm theo số lượng và revenue
- Vẽ phân phối hành vi khách hàng
- RFM analysis visualization

Class 1: DataCleaner

Mục đích: Xử lý và làm sạch dữ liệu giao dịch bán lẻ từ file raw

Chức năng chính:

- Load dữ liệu từ file CSV
- Làm sạch dữ liệu (loại bỏ missing values, invalid records)
- Tạo các time features
- Tính toán RFM metrics (Recency, Frequency, Monetary)
- Save data đã được clean vào folder cụ thể



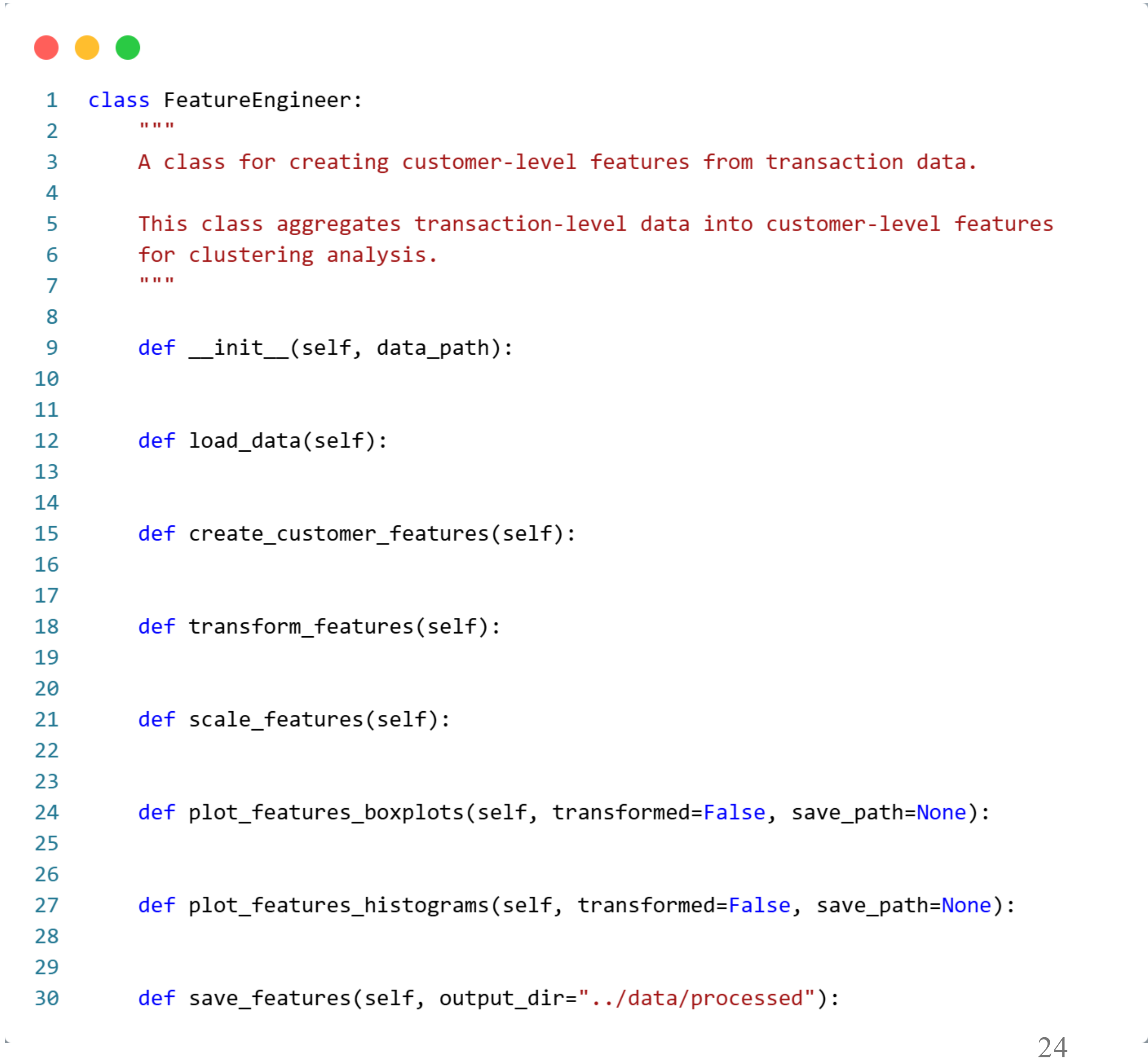
```python
class DataCleaner:
    """
    A class for cleaning and preprocessing retail transaction
    data.

    This class handles data loading, cleaning operations, and
    basic exploratory
    data analysis for online retail datasets.
    """

    def __init__(self, data_path):

    def load_data(self):

    def clean_data(self):

    def create_time_features(self):

    def calculate_rfm(self):

    def save_cleaned_data(self, output_dir="../data/processed"):
```

Class 2: FeatureEngineer

Mục đích: Tạo các features tổng hợp ở mức khách hàng từ dữ liệu giao dịch

Chức năng chính:

- Load clean data và chuẩn bị cho feature engineering
- Tổng hợp features hành vi mua sắm cho mỗi khách hàng
- Áp dụng Box-Cox và standardization để chuẩn hóa phân phối
- Vẽ box plots và histograms cho các features
- Save các feature đã được xử lý



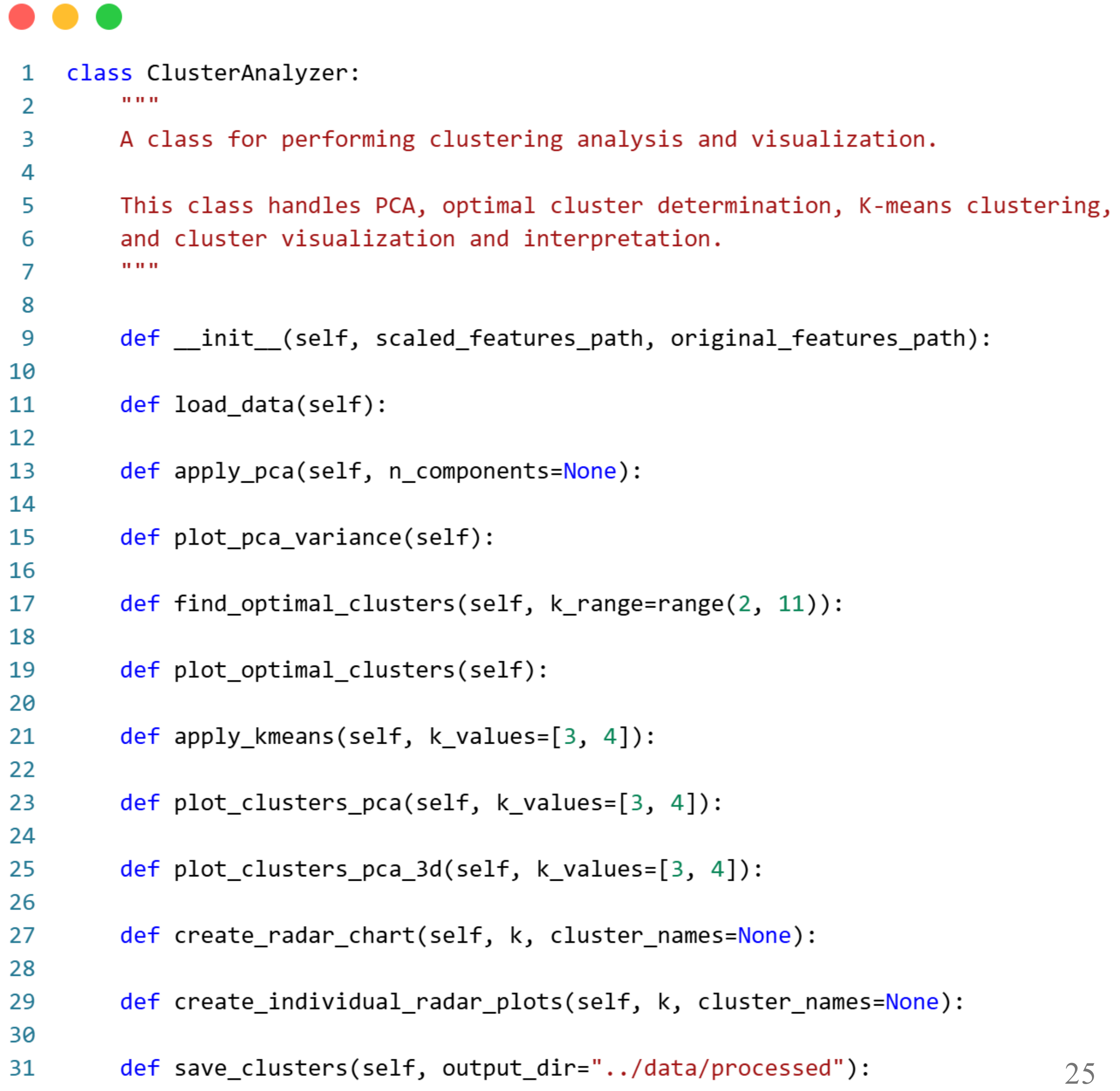
```python
class FeatureEngineer:
    """
    A class for creating customer-level features from transaction data.

    This class aggregates transaction-level data into customer-level features
    for clustering analysis.
    """

    def __init__(self, data_path):


    def load_data(self):


    def create_customer_features(self):


    def transform_features(self):


    def scale_features(self):


    def plot_features_boxplots(self, transformed=False, save_path=None):


    def plot_features_histograms(self, transformed=False, save_path=None):


    def save_features(self, output_dir="../data/processed"):
```

Class 3: ClusterAnalyzer

Mục đích: Thực hiện phân tích clustering và visualization

Chức năng chính:

- Áp dụng PCA để giảm chiều dữ liệu
- Tìm số cluster tối ưu (Elbow method, Silhouette score)
- Thực hiện K-means clustering
- Visualization clusters trong không gian 2D/3D
- Tạo radar charts để phân tích đặc điểm clusters



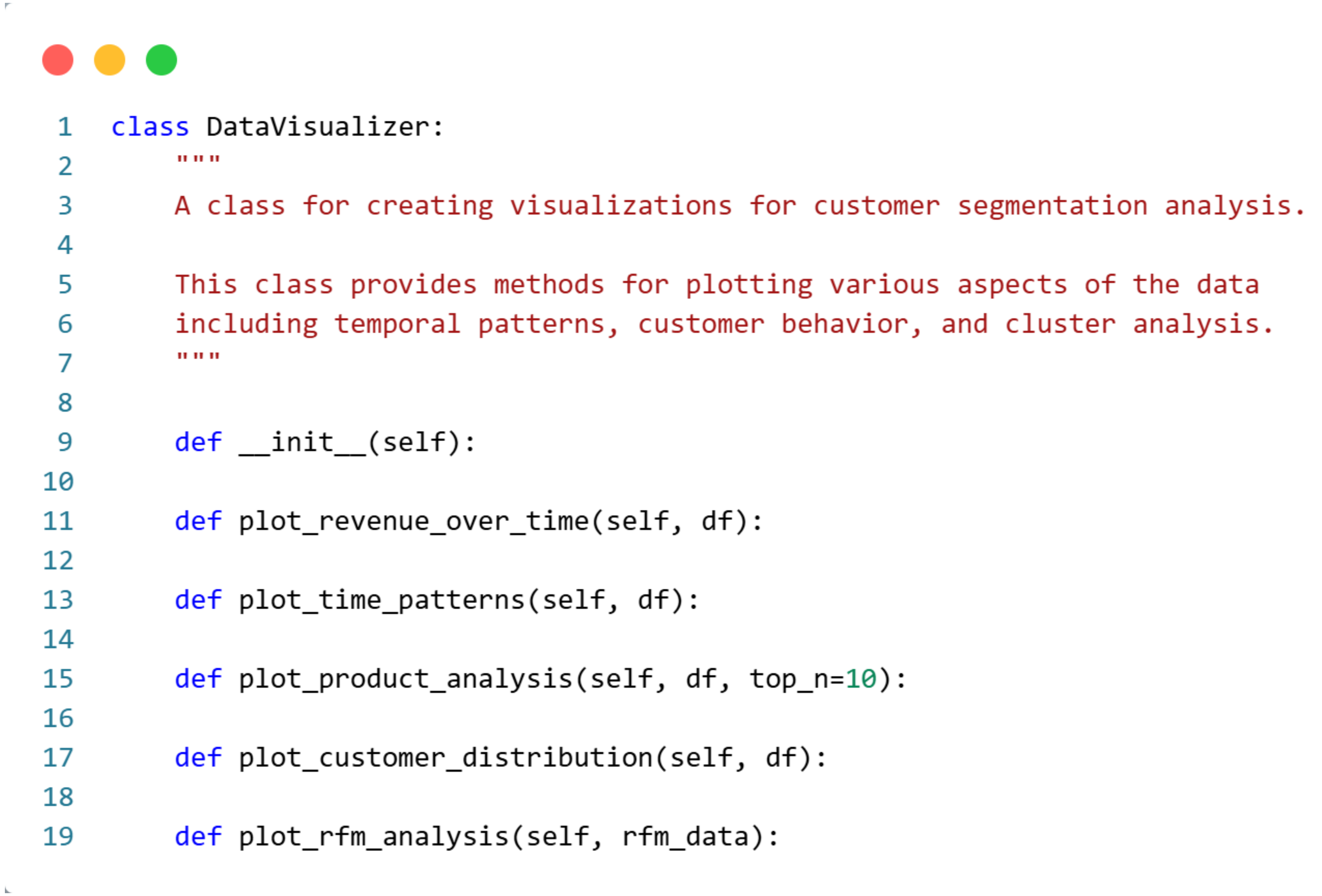
```python
class ClusterAnalyzer:
    """
    A class for performing clustering analysis and visualization.

    This class handles PCA, optimal cluster determination, K-means clustering,
    and cluster visualization and interpretation.
    """

    def __init__(self, scaled_features_path, original_features_path):

    def load_data(self):

    def apply_pca(self, n_components=None):

    def plot_pca_variance(self):

    def find_optimal_clusters(self, k_range=range(2, 11)):

    def plot_optimal_clusters(self):

    def apply_kmeans(self, k_values=[3, 4]):

    def plot_clusters_pca(self, k_values=[3, 4]):

    def plot_clusters_pca_3d(self, k_values=[3, 4]):

    def create_radar_chart(self, k, cluster_names=None):

    def create_individual_radar_plots(self, k, cluster_names=None):

    def save_clusters(self, output_dir="../data/processed"):
```

Class 4: DataVisualizer

Mục đích: Tạo các visualization để phân tích dữ liệu

Chức năng chính:

- Vẽ biểu đồ doanh thu theo thời gian
- Vẽ phân tích patterns theo ngày/giờ
- Vẽ phân tích sản phẩm theo số lượng và revenue
- Vẽ phân phối hành vi khách hàng
- RFM analysis visualization



```python
class DataVisualizer:
    """
    A class for creating visualizations for customer segmentation analysis.

    This class provides methods for plotting various aspects of the data
    including temporal patterns, customer behavior, and cluster analysis.
    """

    def __init__(self):

    def plot_revenue_over_time(self, df):

    def plot_time_patterns(self, df):

    def plot_product_analysis(self, df, top_n=10):

    def plot_customer_distribution(self, df):

    def plot_rfm_analysis(self, rfm_data):
```

Data Cleaning: Nguyên tắc & Thao tác

1. Tạo TotalPrice

Tính **TotalPrice = Quantity × UnitPrice** để đo giá trị thực tế của từng dòng giao dịch.

2. Loại đơn hủy

Loại bỏ các **InvoiceNo bắt đầu bằng 'C'** (cancelled invoices) để tập trung vào giao dịch thực tế.

3. Lọc thị trường

Giữ lại chỉ **United Kingdom** để tập trung phân tích thị trường chính, tránh bias từ thị trường nhỏ.

4. Xử lý missing & invalid

Bỏ records thiếu **CustomerID**, lọc **Quantity > 0** và **UnitPrice > 0** để đảm bảo data hợp lệ.

5. Tạo temporal features

Tạo **DayOfWeek** và **HourOfDay** từ InvoiceDate phục vụ EDA và feature engineering về thời gian.

Feature Engineering: RFM - Framework tham chiếu

RFM là phương pháp phân khúc khách hàng truyền thống dựa trên 3 chiều hành vi cốt lõi. Trong dự án này, RFM đóng vai trò tham chiếu để diễn giải và validate kết quả K-means.

Recency (R)

Ngày từ lần mua gần nhất

Đo độ "gần đây" của khách hàng. Giá trị thấp = khách hàng active gần đây. Quan trọng cho chiến dịch retention.

Frequency (F)

Số lần mua (hóa đơn)

Đo mức độ trung thành và tương tác. Giá trị cao = khách hàng thường xuyên. Quan trọng cho loyalty programs.

Monetary (M)

Tổng chi tiêu

Đo giá trị vòng đời khách hàng (CLV). Giá trị cao = khách hàng VIP. Quan trọng cho phân bổ tài nguyên.

Feature Engineering: Bộ 16 features được sử dụng

Bộ 16 features ở cấp độ khách hàng nắm bắt các khía cạnh khác nhau của hành vi mua hàng, vượt xa RFM truyền thống.

Chỉ số cơ bản

- Sum_Quantity
- Mean_UnitPrice
- Mean_TotalPrice
- Sum_TotalPrice
- Count_Invoice
- Count_Stock

Tổng hợp theo sản phẩm

- Mean_InvoiceCountPerStock
- Mean_StockCountPerInvoice

Tổng hợp theo hóa đơn

- Mean_UnitPriceMeanPerInvoice
- Mean_QuantitySumPerInvoice
- Mean_TotalPriceMeanPerInvoice
- Mean_TotalPriceSumPerInvoice

Tổng hợp theo loại sản phẩm

- Mean_UnitPriceMeanPerStock
- Mean_QuantitySumPerStock
- Mean_TotalPriceMeanPerStock
- Mean_TotalPriceSumPerStock

Feature Engineering: Ý nghĩa Business của 16 Features

Khối lượng & Tần suất

- **Sum_Quantity, Count_Invoice**
- Đo lường mức độ tương tác và quy mô mua sắm. Phân biệt khách hàng "mua nhiều lần" vs "mua số lượng lớn".

Giá trị & Biên lợi nhuận

- **Sum_TotalPrice, Mean_TotalPrice, Mean_UnitPrice**
- Đo CLV và sở thích giá. Phân biệt "VIP tiêu nhiều" vs "săn giá rẻ" vs "premium lover".

Đa dạng & Giỏ hàng

- **Count_Stock, Mean_StockCountPerInvoice**
- Đo hành vi khám phá và cross-selling. Phân biệt "explorer" vs "focused buyer".

Hành vi theo đơn & sản phẩm

- **Mean_*PerInvoice, Mean_*PerStock**
- Đo mức độ tập trung/phân tán chi tiêu. Hỗ trợ chiến lược up-sell và product bundling.

Tham Khảo: Cách tính 16 Features

Quy trình feature engineering sử dụng pandas groupby và tổng hợp đa cấp để tạo các đặc trưng phức tạp từ dữ liệu giao dịch.

Aggregate cấp 1

Gom theo CustomerID để tính Sum_Quantity, Count_Invoice, Sum_TotalPrice, Count_Stock, Mean_UnitPrice, Mean_TotalPrice.

Aggregate cấp 2

Gom theo CustomerID + InvoiceNo để tính metrics trung bình trên mỗi đơn hàng (Mean_*PerInvoice).

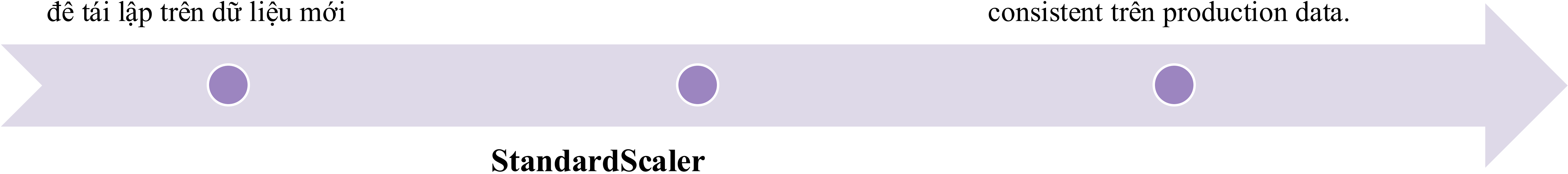
Aggregate cấp 3

Gom theo CustomerID + StockCode để tính metrics trung bình trên mỗi sản phẩm (Mean_*PerStock).

Đóng gói & tái sử dụng

Mã hoá toàn bộ logic trong class FeatureEngineer để dễ bảo trì và áp dụng cho dữ liệu mới.

Transformation & Scaling Pipeline



Box-Cox Transform

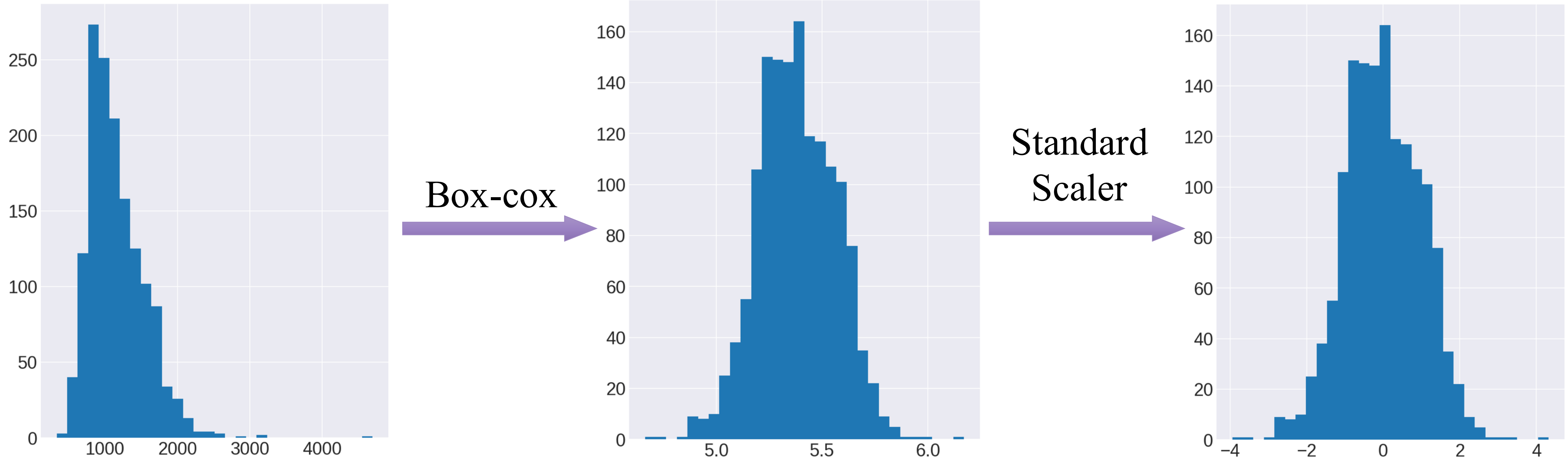
- Áp dụng cho **từng feature**, dịch +1 nếu có giá trị ≤0. Lưu λ tối ưu để tái lập trên dữ liệu mới

StandardScaler

- Chuẩn hoá về **mean=0, std=1** để đưa các features về cùng thang đo trước khi clustering

Lưu transformers

- Lưu Box-Cox lambdas và StandardScaler fitted để transform consistent trên production data.



PCA & Chọn số thành phần

Fit PCA Trên dữ liệu đã scale (16 features → n components)

Theo dõi explained_variance_ratio của từng PC

Tính cumulative variance để xác định ngưỡng

Vẽ biểu đồ Scree plot và Cumulative variance

Sử dụng trong dự án

Chỉ dùng **PC1-PC2** để trực quan hoá các cụm trong không gian 2D.

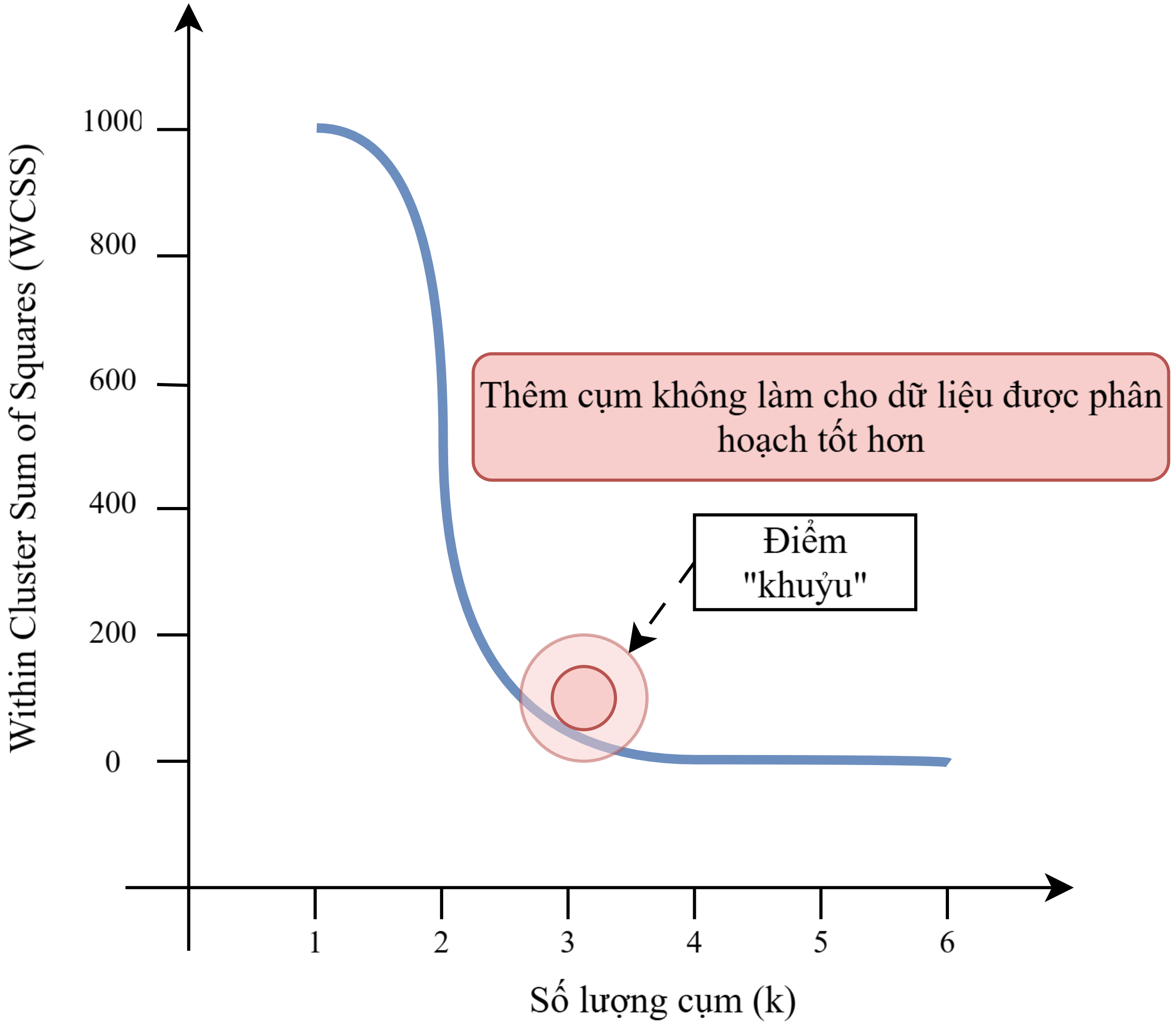
Clustering vẫn thực hiện trên **16 features** gốc để giữ đầy đủ thông tin.

Tối ưu số cụm k

Phương pháp kết hợp: Sử dụng 2 metrics bổ trợ để chọn k tối ưu, tránh overfitting hoặc underfitting.

Elbow Method (WCSS)

- ✓ Quét k ∈ [2, 10], tính **Inertia/WCSS** cho mỗi k
- ✓ Tìm điểm "gối" (elbow) - nơi tốc độ giảm WCSS chậm lại đáng kể
- ✓ Điểm elbow cho thấy **diminishing returns** khi tăng k



Thêm cụm không làm cho dữ liệu được phân hoạch tốt hơn

Điểm "khuỷu"

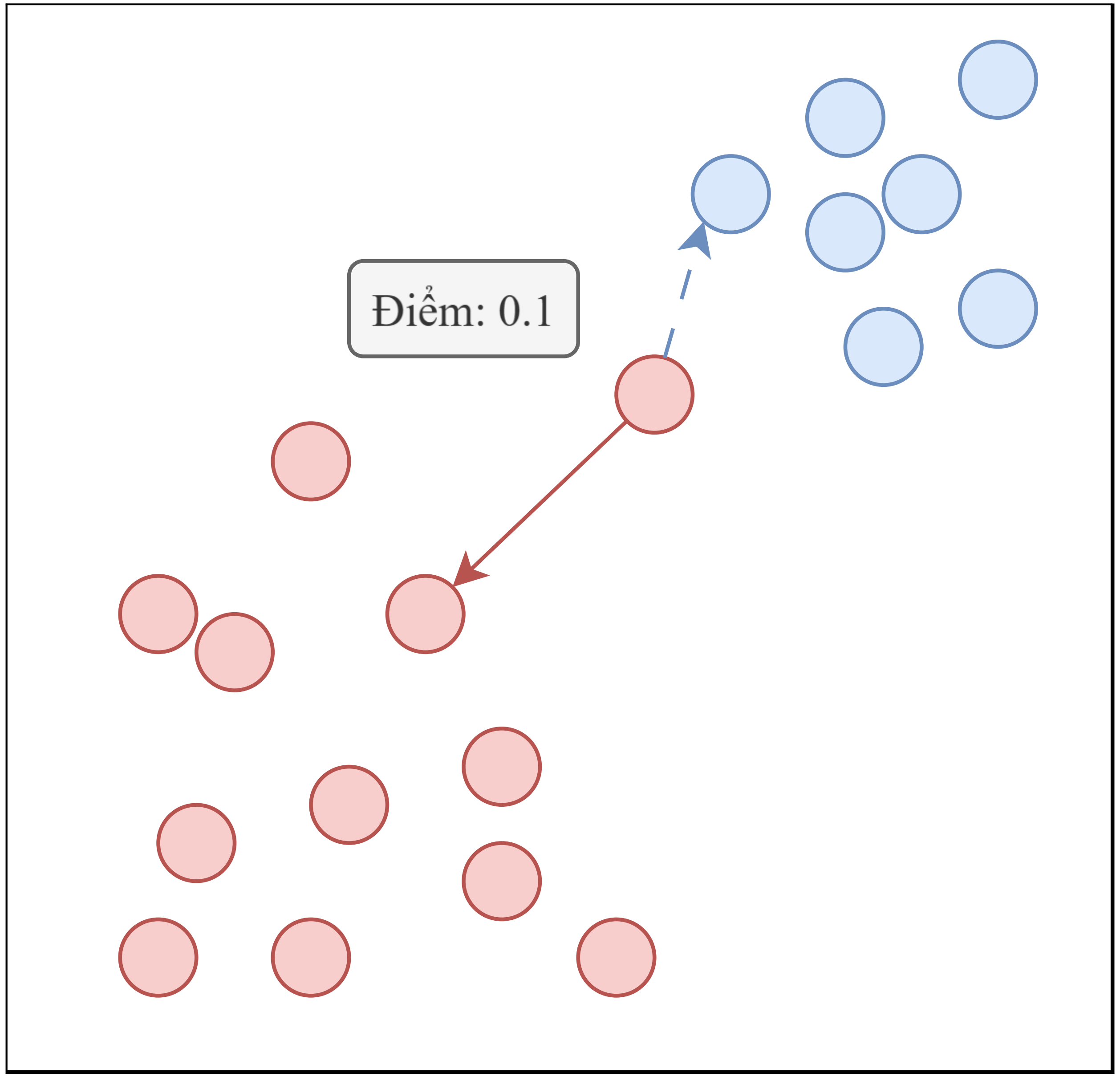
Tối ưu số cụm k

Phương pháp kết hợp: Sử dụng 2 metrics bổ trợ để chọn k tối ưu, tránh overfitting hoặc underfitting.

Silhouette Score

- ✓ Đo độ **tách biệt giữa các cụm**, giá trị trong [-1, 1]
- ✓ Score cao = cụm cohesive và well-separated.
- ✓ Chọn k có Silhouette Score cao nhất

Output: Hàm trong thư viện tự động tính toán, vẽ biểu đồ và đánh dấu k tối ưu theo Silhouette.



Điểm Silhouette Trung Bình: 0.7

Đánh giá Nội bộ & Ổn định

Silhouette Score

Đo độ tách biệt: Giá trị cao (gần 1) = cụm rõ ràng

Phân tích sâu: Xem Silhouette plot cho từng điểm để phát hiện "điểm biên giới" gần nhiều cụm

Ngưỡng tốt: > 0.5 là acceptable, > 0.7 là excellent

WCSS/Inertia

Elbow curve: Quan sát điểm "gối" để chọn k hợp lý

Trade-off: k càng lớn, WCSS càng giảm nhưng overfitting tăng

Insight: Diminishing returns sau điểm elbow

PCA Visualization

Scatter plot: Vẽ PC1 vs PC2, màu theo cluster

Kiểm tra: Các cụm có tách biệt rõ ràng không?

Centroid: Đánh dấu tâm cụm để thấy vị trí tương đối

Outline bài giảng

Phần 1: Giới thiệu & bối cảnh business

- Bối cảnh thách thức
- Mục tiêu dự án
- Thách thức & Cơ hội từ dữ liệu
- Phạm vi & Kỳ vọng dự án

Phần 2: Nền tảng lý thuyết & Hướng tiếp cận

- Supervised vs Unsupervised Learning
- Bài toán phân cụm khách hàng
- Giới thiệu các phương pháp phân cụm
- Giới thiệu các phương pháp Transformation

Phần 3: Triển khai dự án

- Cấu trúc dự án
- Feature Engineering
- Transformation
- Phạm vi & Kỳ vọng

Phần 4: Kết quả của dự án

- EDA: Insights chính
- Kết quả phân cụm
- Đánh giá kết quả phân cụm
- Hướng mở rộng trong tương lai

Kết quả EDA: Insights chính

Tổng Giao Dịch Gốc

541,909

Dữ liệu thô ban đầu (Raw Data)

Giao Dịch Hợp Lệ (UK)

354,321

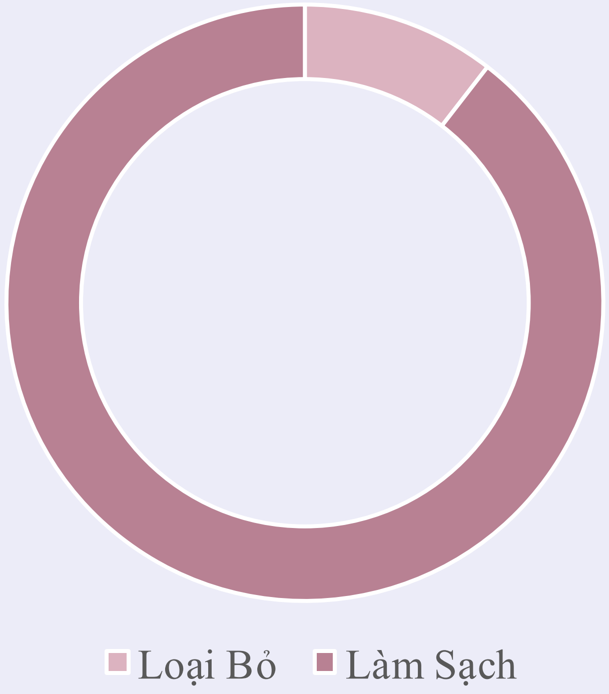
✓ Đã làm sạch

Khách Hàng (UK)

3,920

Số lượng khách hàng

Tỉ Lệ Dữ Liệu Sạch



Dữ liệu giảm 10.5% sau khi lọc nhiễu và giới hạn tại UK

Các Phát Hiện Chính (Key Findings)

Xu Hướng Doanh Thu

Doanh thu cho thấy xu hướng tăng dần theo thời gian trong giai đoạn khảo sát.

Thời Gian Mua Hàng

Hầu hết việc mua hàng diễn ra trong giờ làm việc (các ngày trong tuần).

Hành Vi Khách Hàng

Chi tiêu và tần suất giao dịch của khách hàng rất đa dạng.

Quy Trình Xử Lý Đã Thực Hiện

- ✓ Loại bỏ bản ghi thiếu CustomerID
- ✓ Lọc dữ liệu quốc gia: UK
- ✓ Loại bỏ Quantity/Price âm
- ✓ Chuẩn hóa định dạng ngày tháng

Kết quả EDA: Insights chính

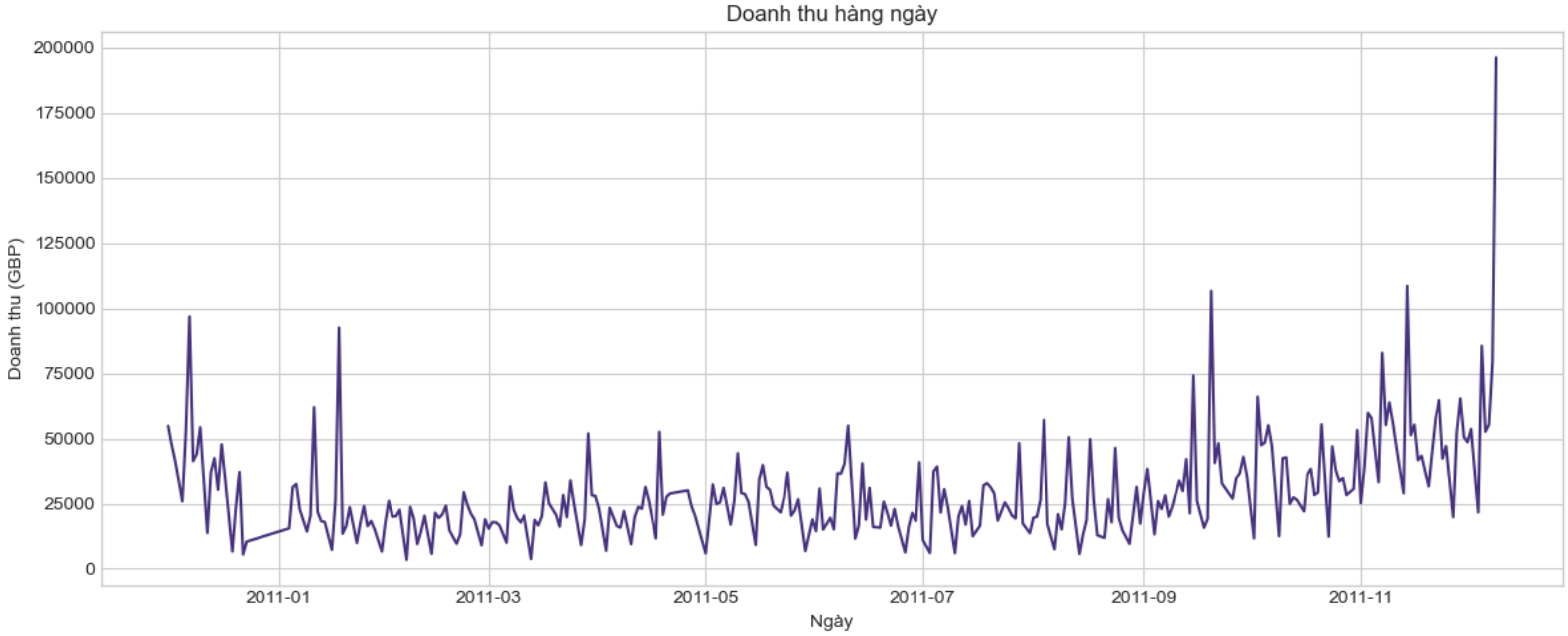
Phân Tích Khám Phá - Doanh Thu

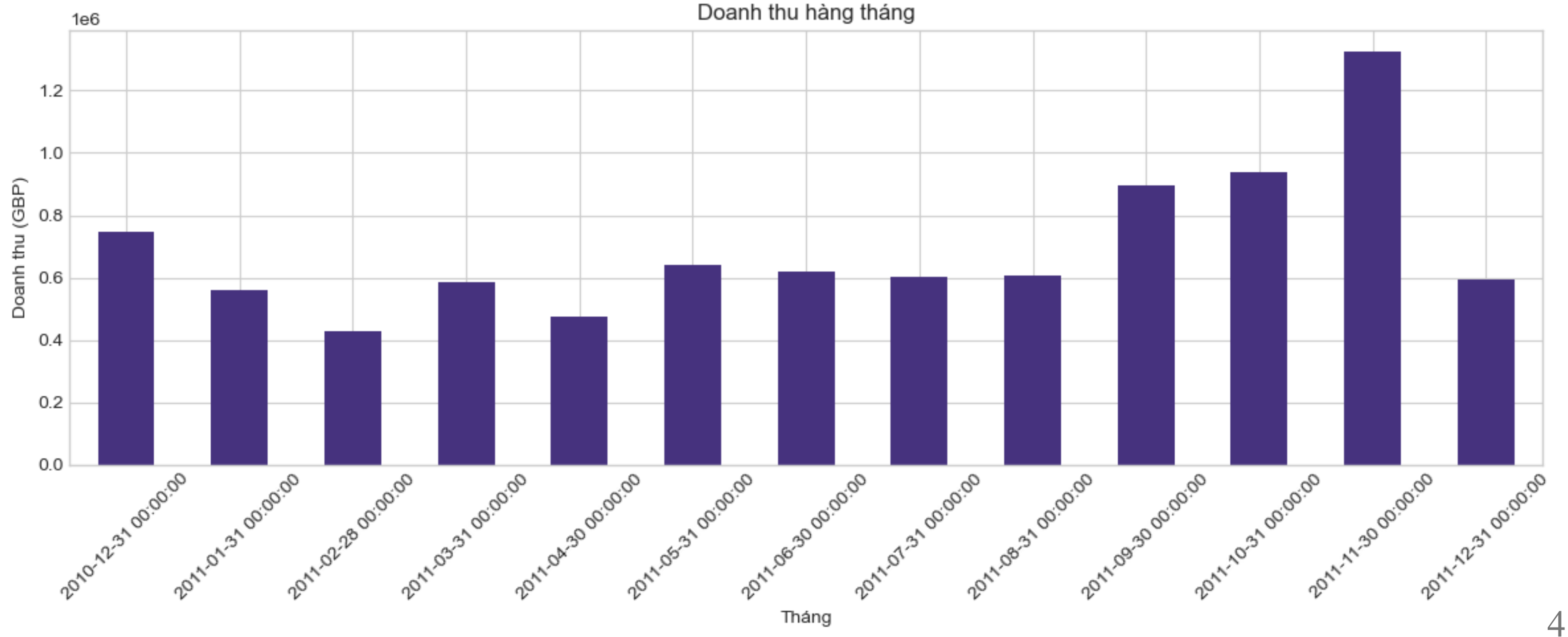
Doanh Thu Hàng Ngày

- **Xu hướng tăng dần:** Từ đầu năm đến cuối năm
- **Biến động cao:** Doanh thu không ổn định,
- **Doanh thu trung bình hàng ngày** dao động từ £20,000-£40,000 trong hầu hết các ngày
- **Ngày có doanh thu cao nhất** đạt gần £200,000 (cuối tháng 11/2011)

Doanh Thu Hàng Tháng

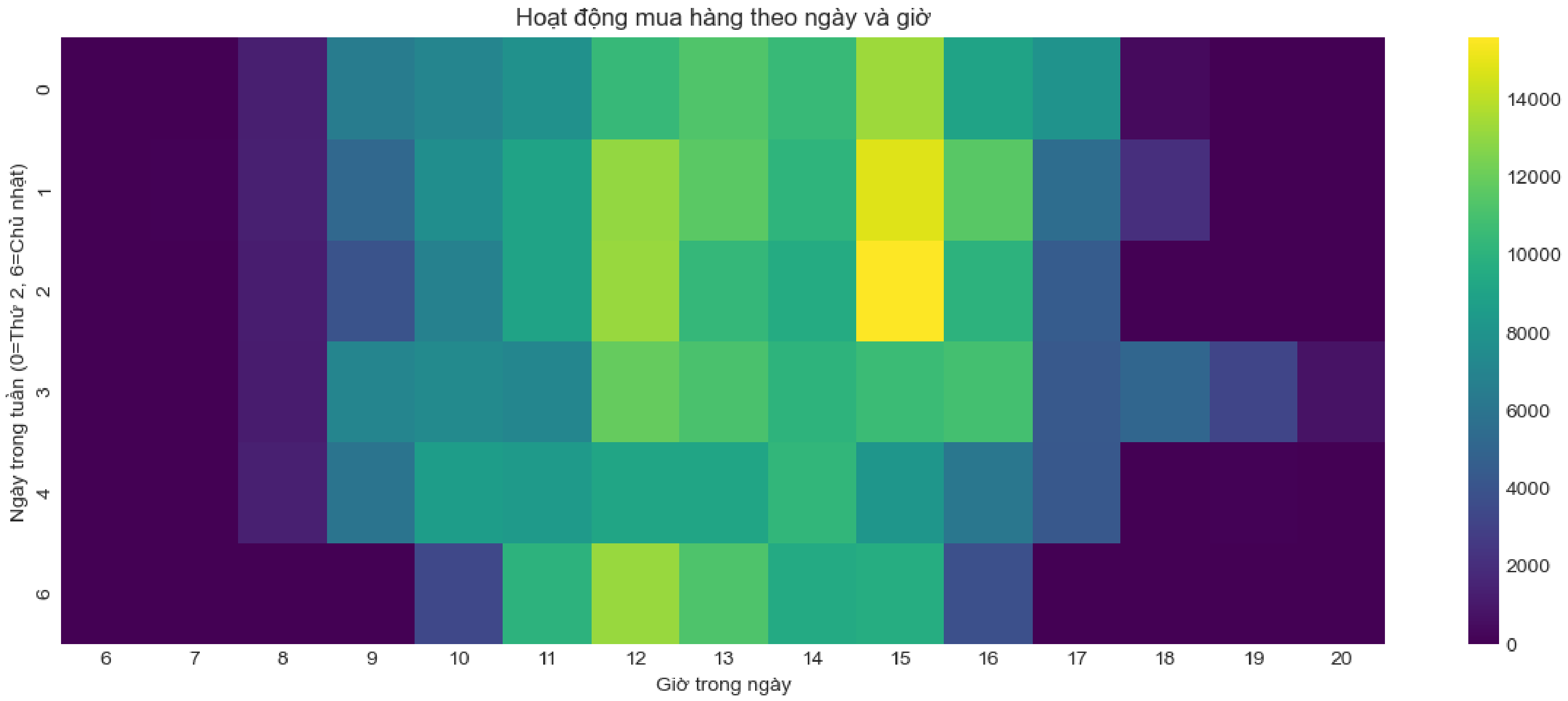
- **Xu hướng tăng trưởng:** Từ tháng 9-11/2011, doanh thu tăng vượt bậc từ ~£900,000 lên ~£1.35 triệu
- **Mùa thấp điểm:** Tháng 2-4/2011 duy trì doanh thu thấp quanh ~£420,000 đến ~£590,000
- Tháng 2/2011 có doanh thu **thấp nhất** ~£420,000, Tháng 11/2011 đạt doanh thu **cao nhất** ~£1.35 triệu





Kết quả EDA: Insights chính

Phân Tích Khám Phá - Mua Hàng Theo Ngày và Giờ



Hoạt Động Mua Hàng Theo Ngày và Giờ

- **Giờ vàng (12-15h):** Hoạt động mua hàng cao nhất với ~12,000-15,000 giao dịch,
- **Giờ sáng (9-11h) và chiều tối (16-17h):** Mức độ trung bình cao với ~8,000-10,000 giao dịch
- **Thứ 3 và thứ 4** có hoạt động cao nhất, đặc biệt vào khung 12-15h
- **Chủ nhật (Sunday)** có hoạt động thấp nhất trong tuần

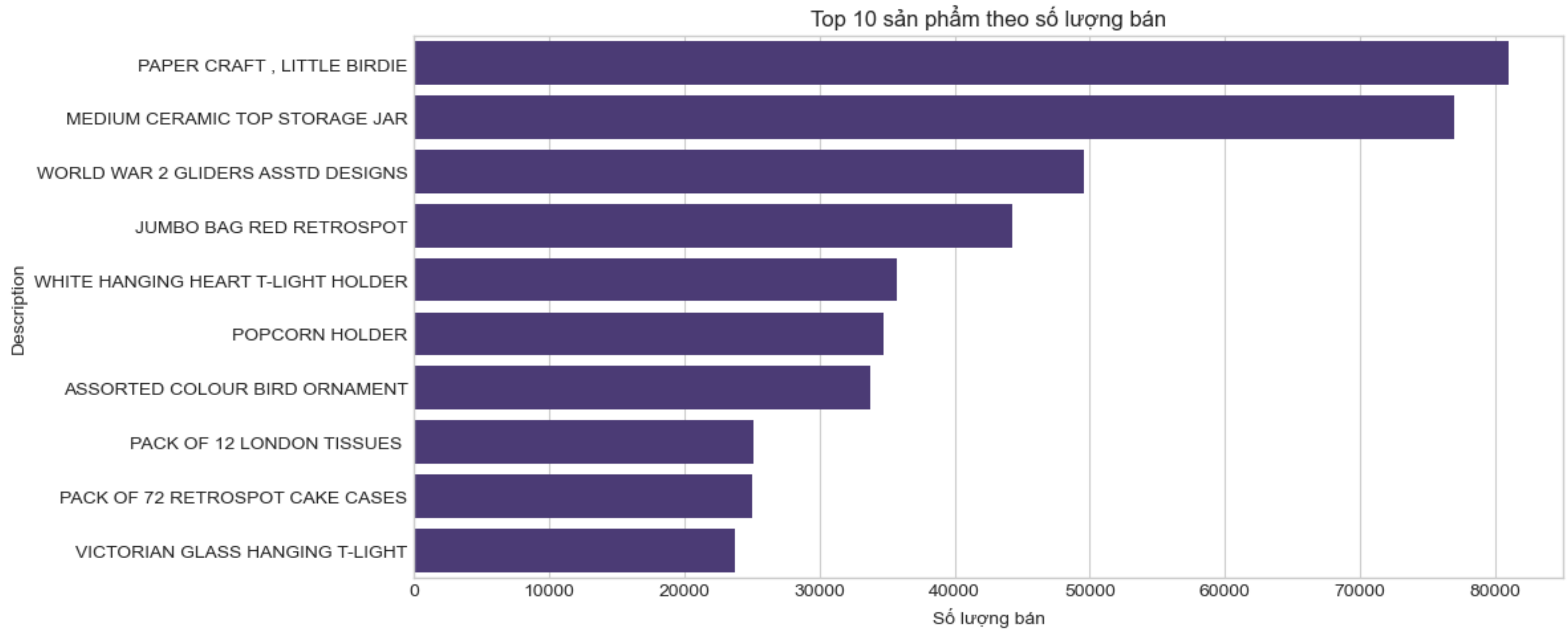
Kết quả EDA: Insights chính

Phân Tích Khám Phá - hiệu suất sản phẩm theo số lượng bán và doanh thu

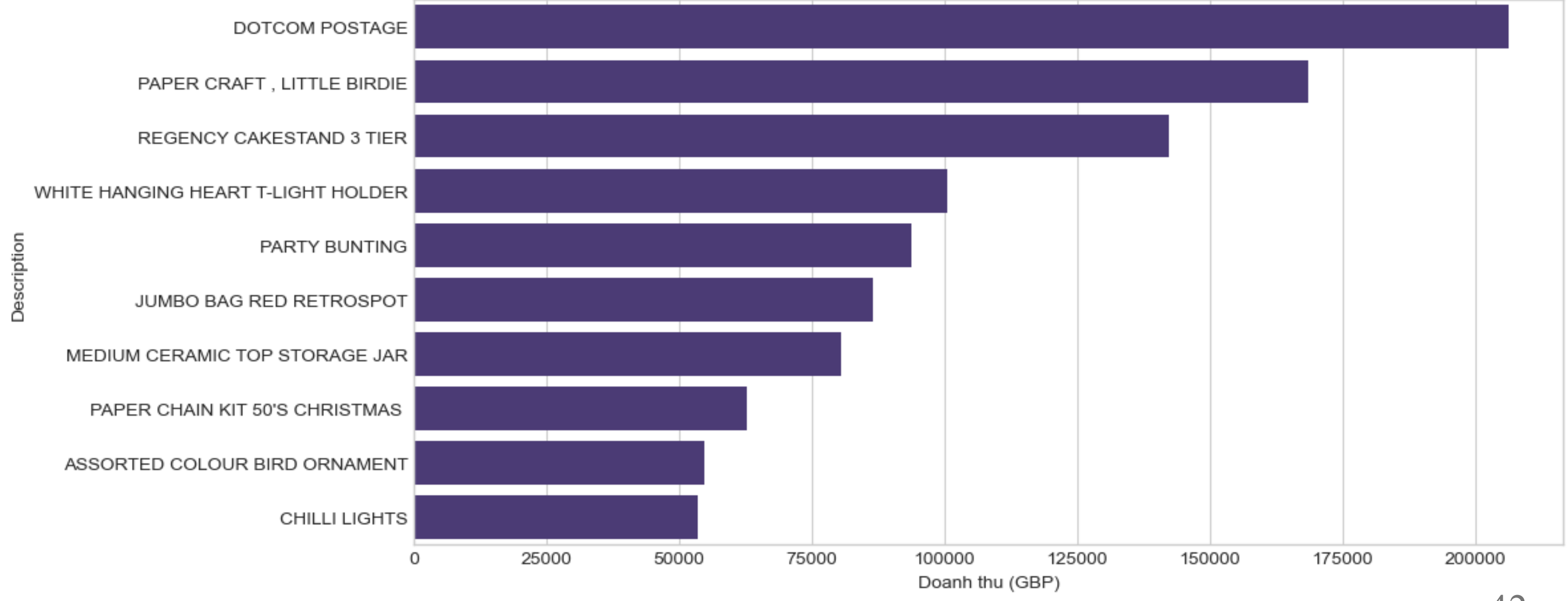
Sản Phẩm Theo Số Lượng Bán và Doanh Thu

- **Top 10 sản phẩm có số lượng** từ 25,000 đến 82,000 units
- **PAPER CRAFT, LITTLE BIRDIE** dẫn đầu với ~82,000 units bán ra
- **DOTCOM POSTAGE** sinh doanh thu cao nhất >200,000
- **Top 10 sản phẩm có doanh thu** từ ~51,000 đến >200,000
- **PAPER CRAFT, LITTLE BIRDIE** chiếm ưu thế: top 1 cả về số lượng và doanh thu

Top 10 sản phẩm theo số lượng bán



Top 10 sản phẩm theo doanh thu

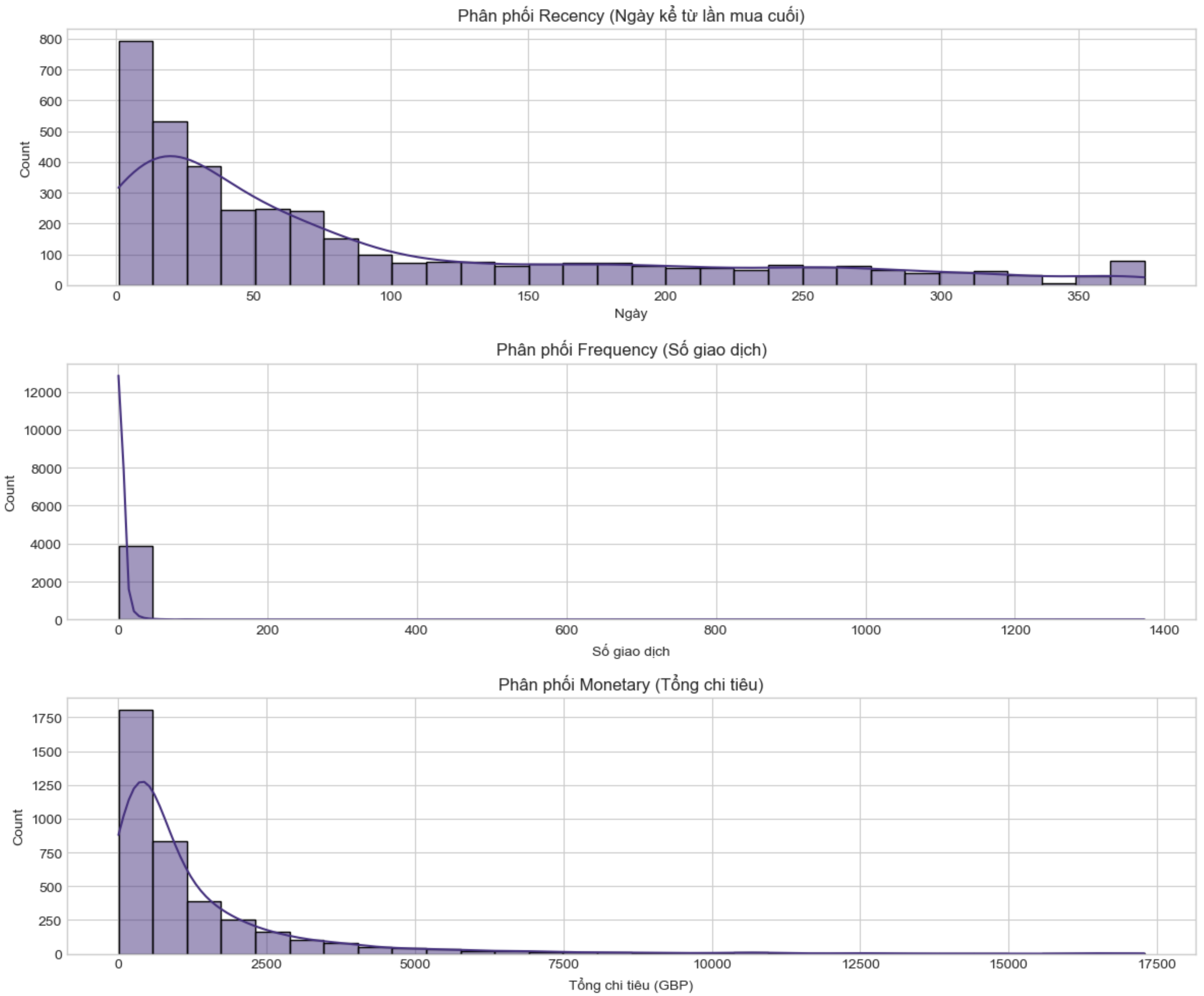


Kết quả EDA: Insights chính

Phân tích RFM

RECENCY

- Phần lớn khách nằm **dưới 50–60 ngày**, đỉnh tập trung ở vùng rất thấp (10–20 ngày)
- **Đường phân phối lệch phải mạnh:** một số khách đã rất lâu không mua (200–300+ ngày), nhưng số này ít
- Hệ thống vẫn có nhiều khách mua gần đây. Tuy nhiên đuôi dài cho thấy cũng có một lượng khách nguy cơ mất nếu không được kích hoạt lại
- **Có thể chia theo quantile:** top 20% khách có Recency nhỏ nhất, 20% tiếp theo thành 4 nhóm là các nhóm lâu rồi chưa quay lại



Kết quả EDA: Insights chính

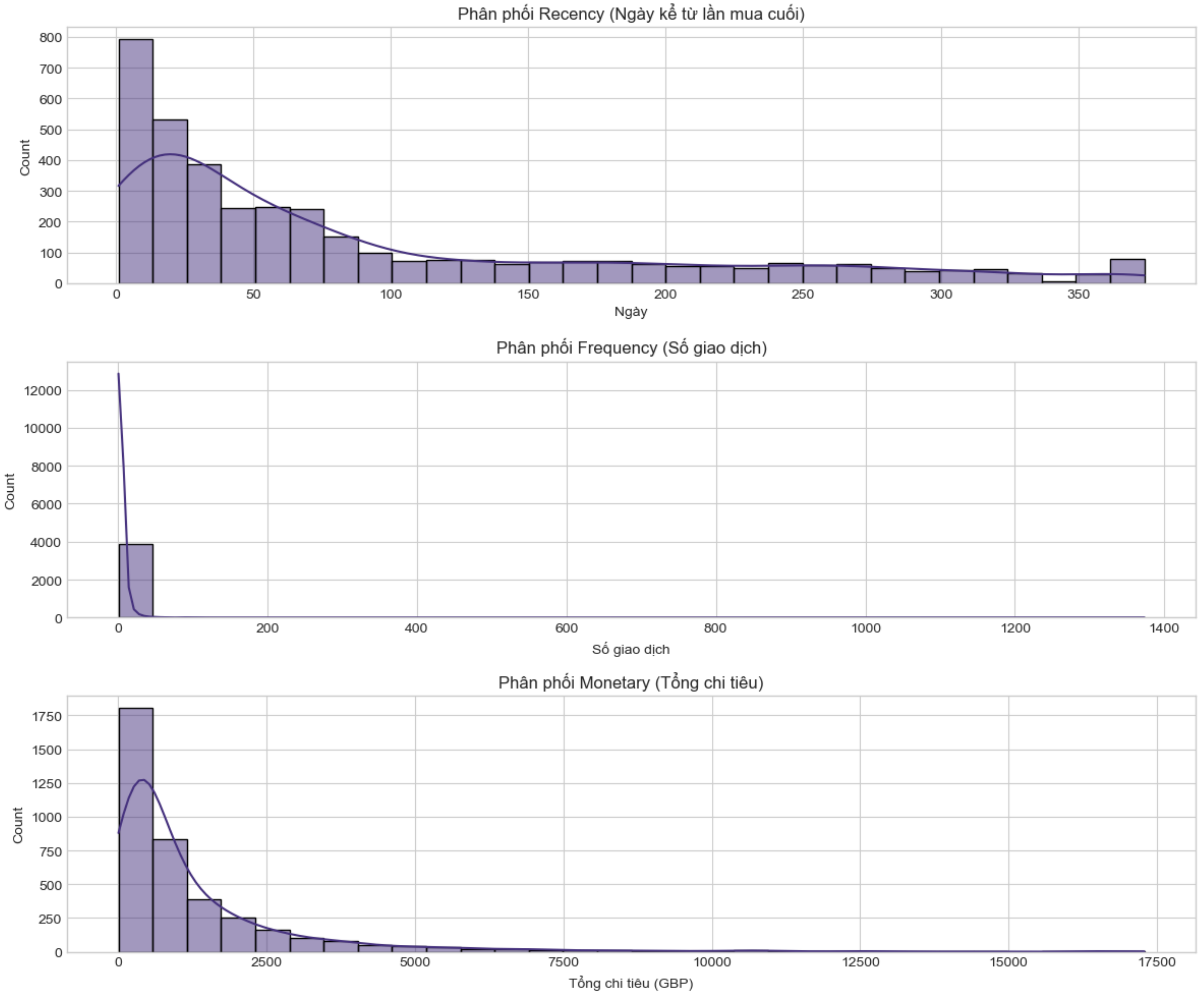
Phân tích RFM

Frequency

- Đa số khách có số giao dịch rất thấp. Phần cột từ 0–20 giao dịch chiếm gần hết chiều cao
- **Đường phân phối lệch phải mạnh:** Một vài khách hàng giao dịch cực nhiều (tới hàng trăm–nghìn lần) nên phân phối bị kéo dài ra bên phải..
- Phần khách “VIP” có F rất cao nhưng số lượng cực ít

Monetary

- Phần lớn khách chi tiêu trong vùng thấp–trung bình (0–2000 GBP).
- **Đường phân phối lệch phải mạnh:** Có một số ít khách chi cực lớn (tới hơn 10k–17k GBP) nên cũng lệch phải mạnh.
- Doanh thu tập trung vào một nhóm nhỏ khách high–value
- Phần đông khách mua nhỏ lẻ.



Kết quả Feature Engineering & Transform

Bộ 16 features ở cấp độ khách hàng nắm bắt các khía cạnh khác nhau của hành vi mua hàng, vượt xa RFM truyền thống.

Chỉ số cơ bản

- Sum_Quantity: Tổng số lượng sản phẩm đã mua
- Mean_UnitPrice: Giá trung bình trên mỗi đơn vị trong tất cả lần mua
- Mean_TotalPrice: Số tiền trung bình mỗi giao dịch
- Sum_TotalPrice: Tổng số tiền đã chi (giá trị vòng đời khách hàng)
- Count_Invoice: Số lượng giao dịch duy nhất
- Count_Stock: Số lượng sản phẩm duy nhất đã mua

Tổng hợp theo sản phẩm

- Mean_InvoiceCountPerStock: Tần suất mua trung bình trên mỗi sản phẩm
- Mean_StockCountPerInvoice: Số lượng SP khác nhau TB mỗi giao dịch

Tổng hợp theo hóa đơn

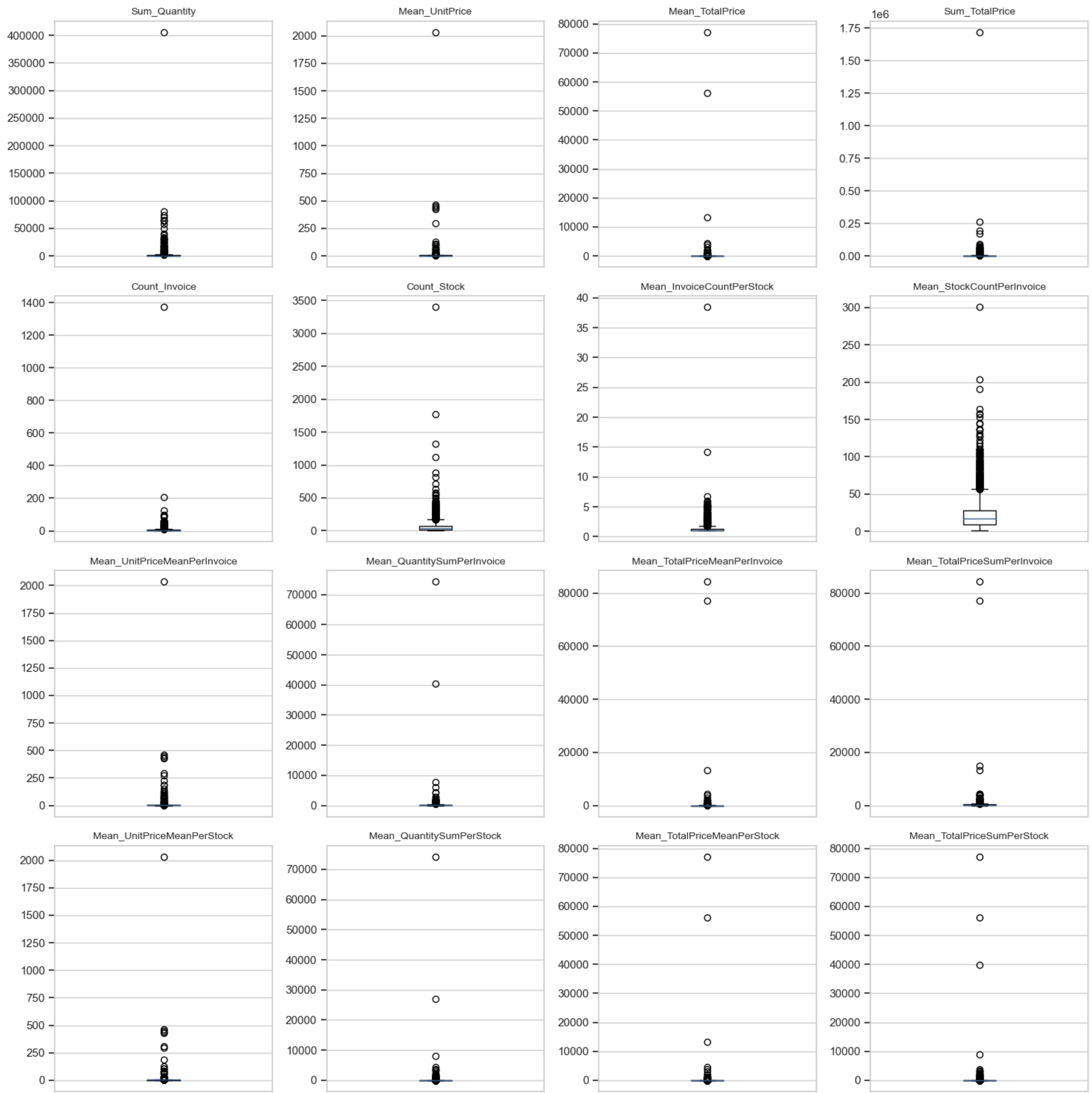
- Mean_UnitPriceMeanPerInvoice: Giá đơn vị trung bình mỗi giao dịch
- Mean_QuantitySumPerInvoice: số lượng TB mỗi GD
- Mean_TotalPriceMeanPerInvoice: Số tiền TB mỗi SP trong GD
- Mean_TotalPriceSumPerInvoice: Tổng chi tiêu TB mỗi GD

Tổng hợp theo loại sản phẩm

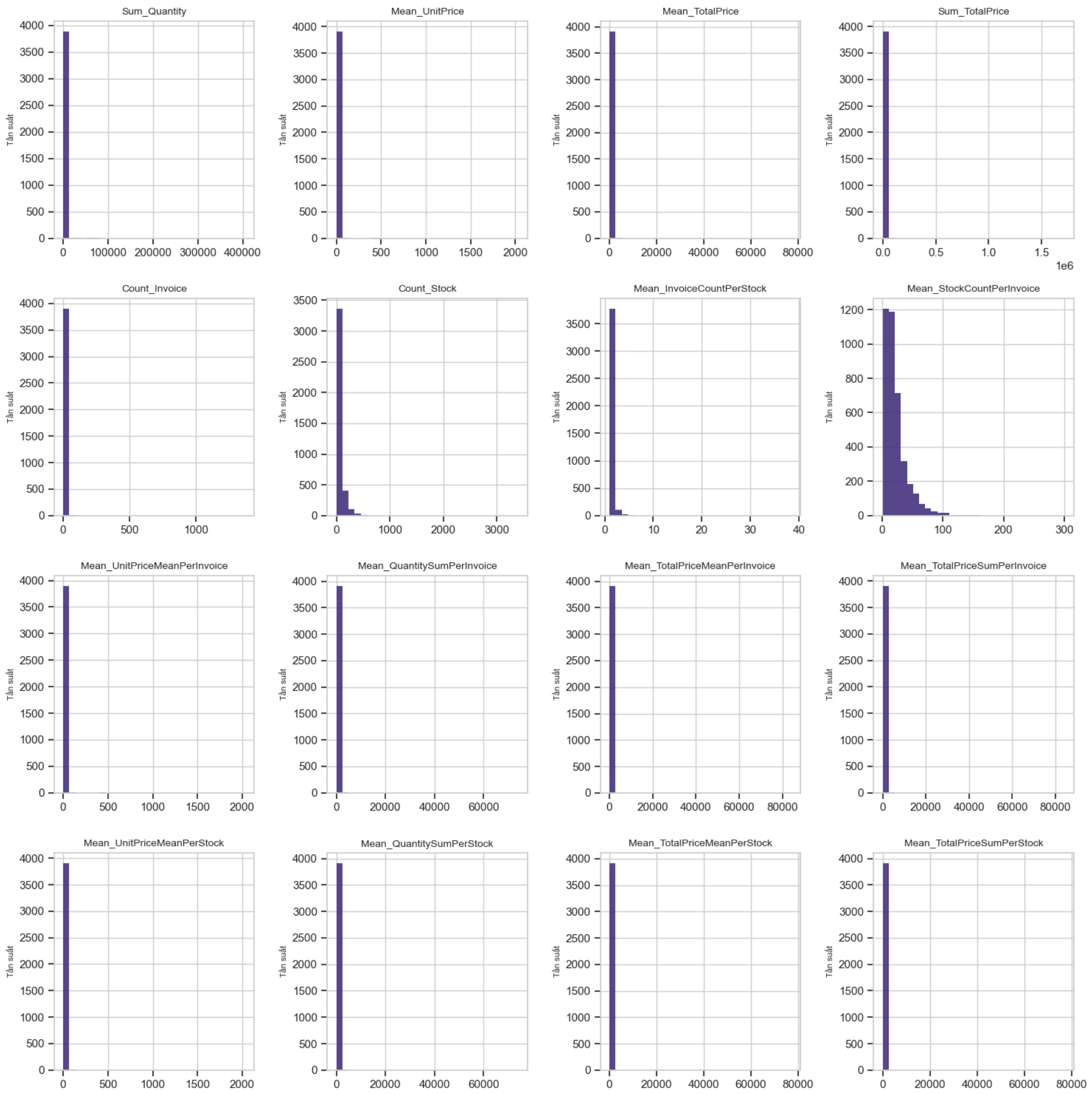
- Mean_UnitPriceMeanPerStock: ức giá trung bình trên mỗi sản phẩm
- Mean_QuantitySumPerStock: Số lượng TB đã mua trên mỗi SP
- Mean_TotalPriceMeanPerStock: Chi tiêu trung bình trên mỗi sản phẩm
- Mean_TotalPriceSumPerStock: Tổng chi tiêu TB trên mỗi SP

Phân phối của features trước biến đổi Box-Cox

Trực quan hóa tất cả features trước Box-Cox transformation

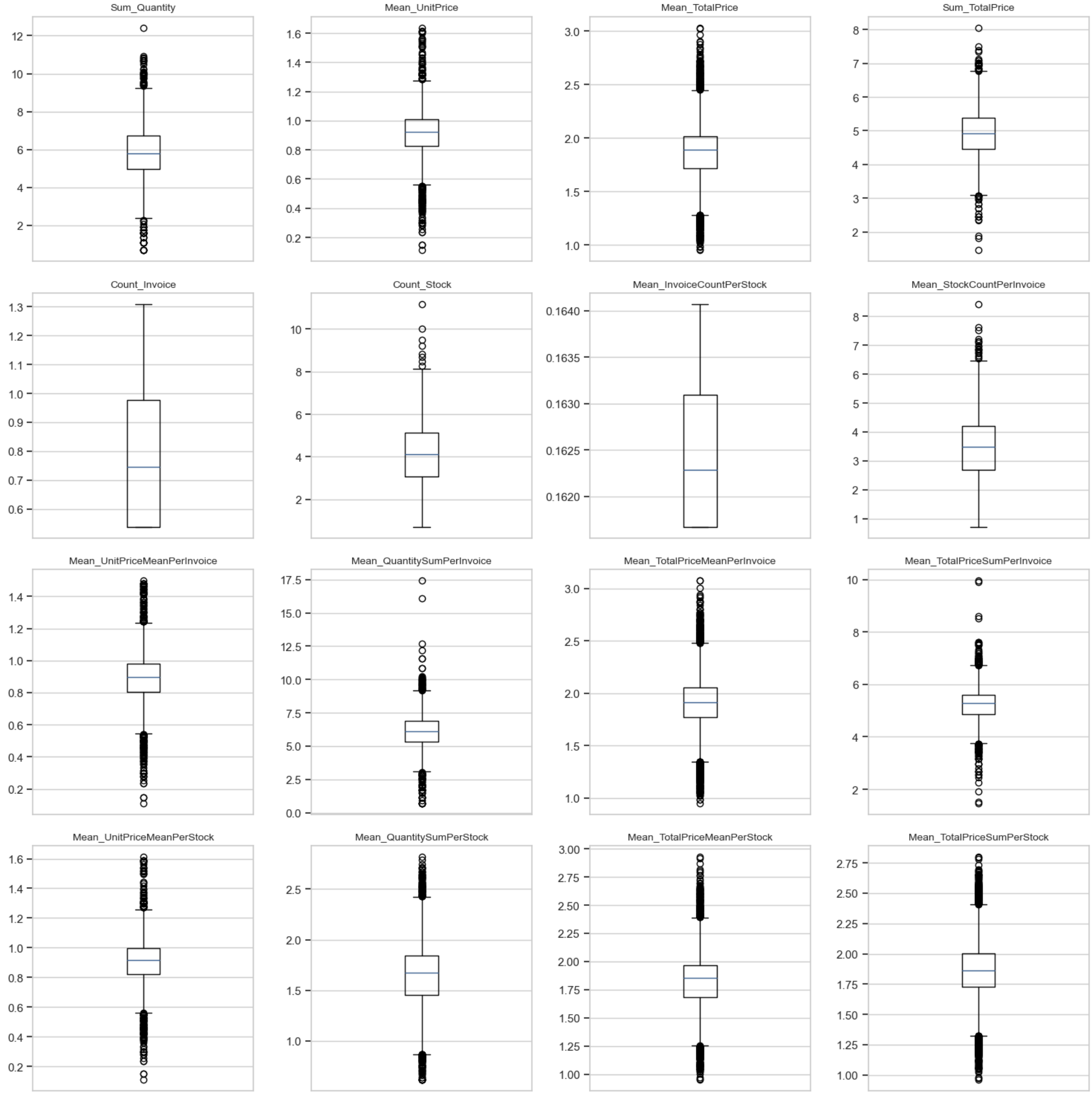


Trực quan hóa phân phối của tất cả features bằng histograms

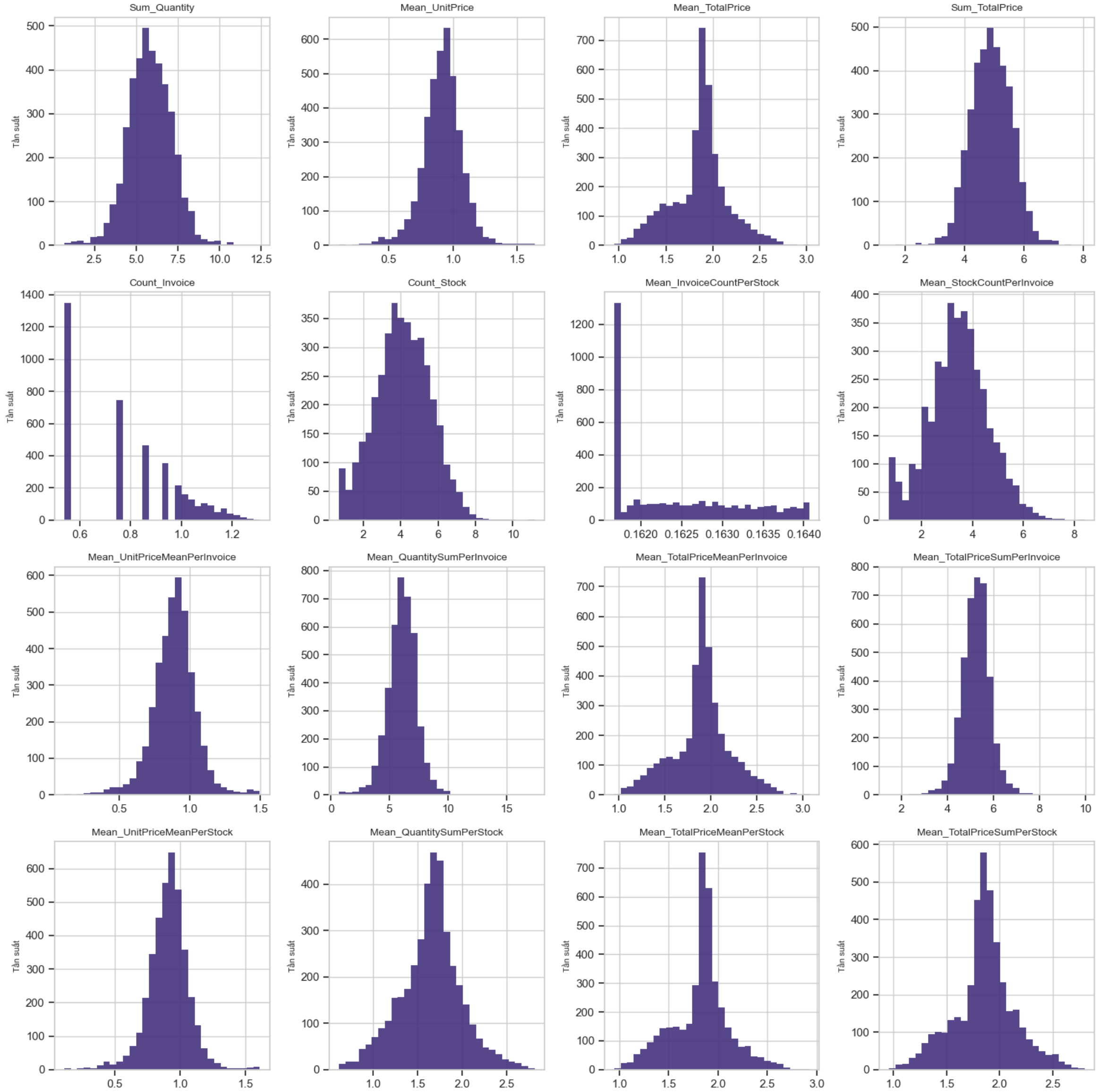


Phân phối của features trước biến đổi Box-Cox

Trực quan hóa tất cả features sau Box-Cox transformation



Trực quan hóa phân phối của features sau Box-Cox transformation bằng histograms

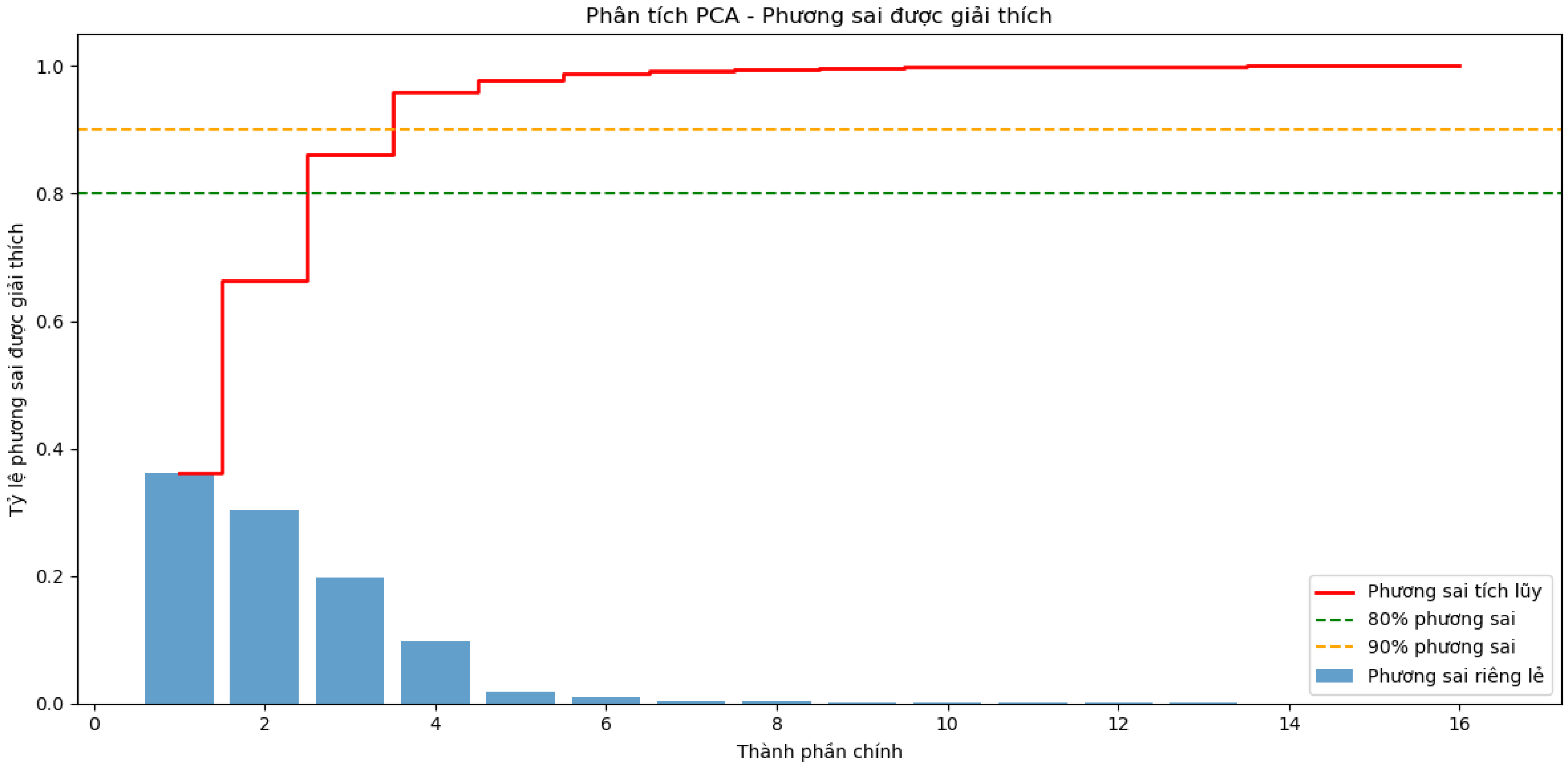


Kết quả Phân cụm: Tổng quan

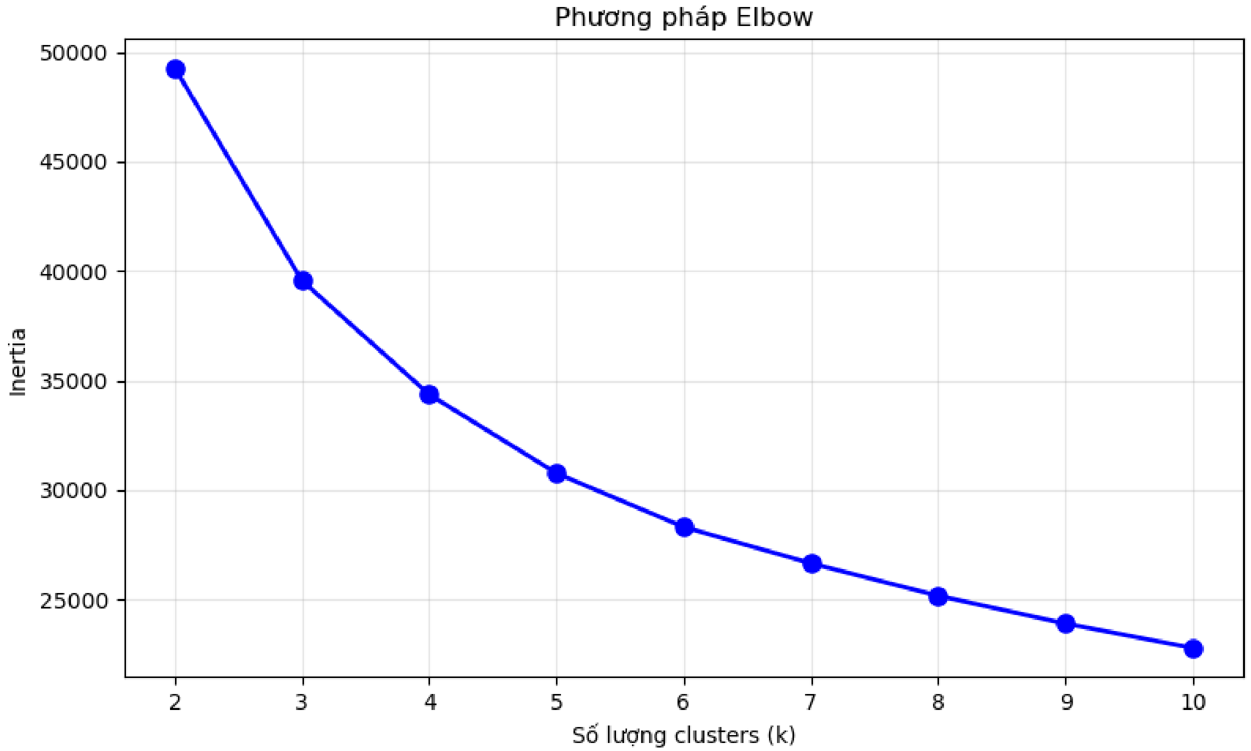
PCA là kỹ thuật giảm chiều dữ liệu, biến đổi tuyến tính các features tương quan thành tập các thành phần chính (principal components) không tương quan, sắp xếp theo độ quan trọng giảm dần.

Phương sai tích lũy:

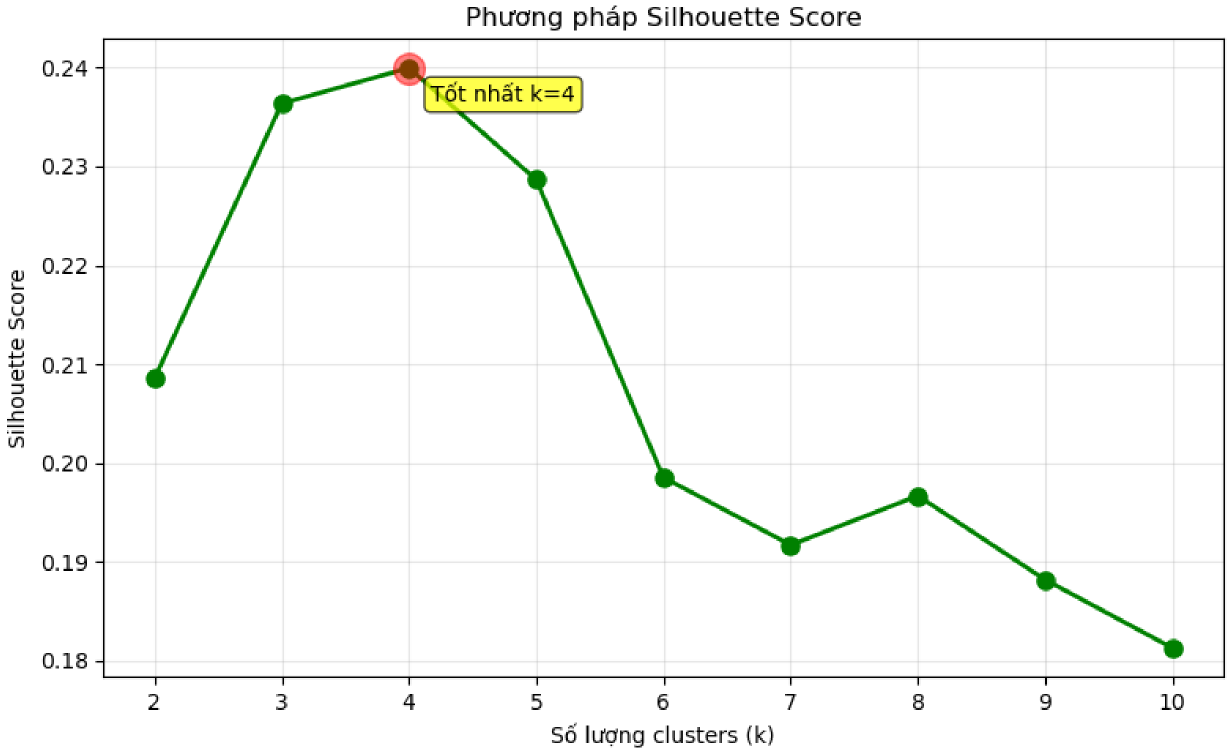
- PC1-PC1: 36.08%
- PC1-PC2: 66.41%
- PC1-PC3: 86.12%
- PC1-PC4: 95.95%
- PC1-PC5: 97.76%
- Cấu trúc dữ liệu chủ yếu nằm trên vài thành phần đầu, các thành phần sau gần như chỉ là nhiễu.
- Nếu bạn dùng tiêu chí ≥ 80% phương sai → chỉ cần 3 thành phần chính.
- Nếu muốn mô hình giữ ≥ 90% phương sai → nên chọn 4 thành phần chính.
- Trực quan hóa: bạn có thể vẽ scatter 2D/3D theo (PC1, PC2, PC3) để Nhìn phân cụm khách hàng, Phát hiện outlier,



Kết quả chọn cụm



Phương pháp Elbow



Phương pháp Silhouette Score

Elbow

- Trục X: số cụm *k*=2→10
- Trục Y: Inertia (tổng bình phương khoảng cách vào tâm cụm)
- Từ k=2 → k=3 → k=4: Inertia giảm rất mạnh
- Sau k=4: tốc độ giảm chậm lại
- “Khuỷu tay” (điểm đổi độ dốc rõ nhất) nằm khoảng k = 4 hoặc 5

Silhouette Score

- Trục X: số cụm *k*=2→10
- Trục Y: Silhouette score – đo độ “tách biệt” giữa các cụm (càng cao càng tốt)
- k=4: ~0.240 (cao nhất)
- Từ k=5 trở đi: điểm giảm dần, dao động ~0.18–0.23, thấp hơn nhiều so với k=4.
- Theo Silhouette, k = 4 cho các cụm có tách biệt nhất

Đánh giá kết quả phân cụm sử dụng PCA

Plot scatter theo (PC1, PC2) hoặc (PC1, PC2, PC3), ta đang nhìn phân bố khách hàng trong không gian đã giảm chiều, sau đó tô màu theo cụm K-Means (K=3 trái, K=4 phải)

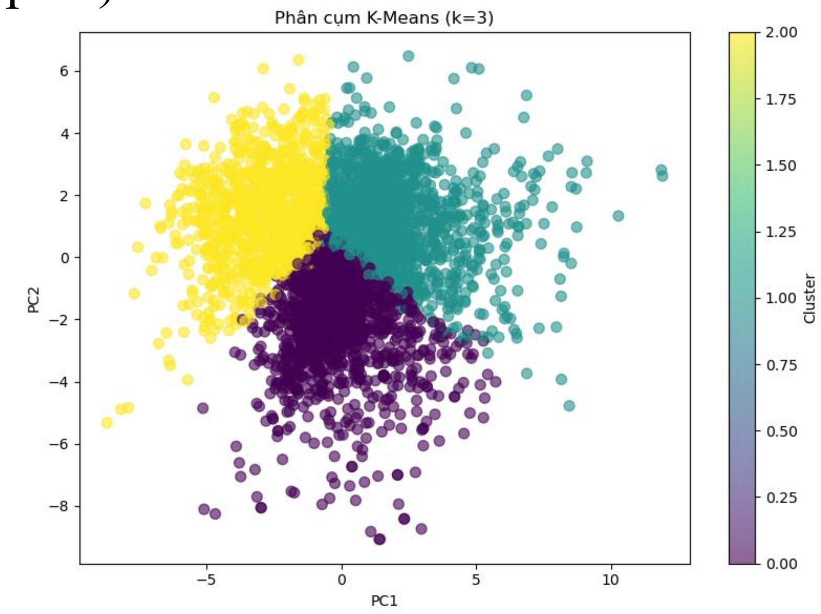
K=3 (Trái)

- Plot2D: Mô hình gom được 3 nhóm lớn, nhưng một nhóm “dưới” rất dày, chứa nhiều kiểu khách hàng khác nhau
- Plot2D: Ở vùng trung tâm (gần (0,0)) có khá nhiều điểm chen lẫn giữa các màu, biên cụm chưa rõ ràng
- Plot3D: Tương tự như Plot2D nhưng nhìn thấy rõ sự khác biệt của các cụm hơn Plot2D

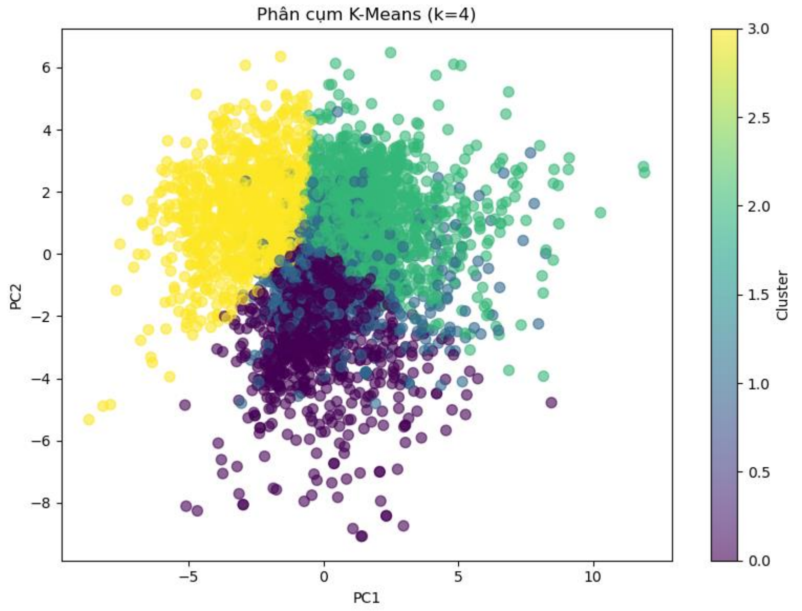
K=4 (Phải)

- Plot2D: Một nhóm “trung gian” quanh khu vực trung tâm gần cụm xanh là và tím
- Plot2D: Có sự overlap của khu vực mới với các vùng xanh lá và tím
- Plot3D: Cụm mới rõ ràng hơn nằm ở PC3 thấp, cụm khác nằm ở PC3 cao hơn

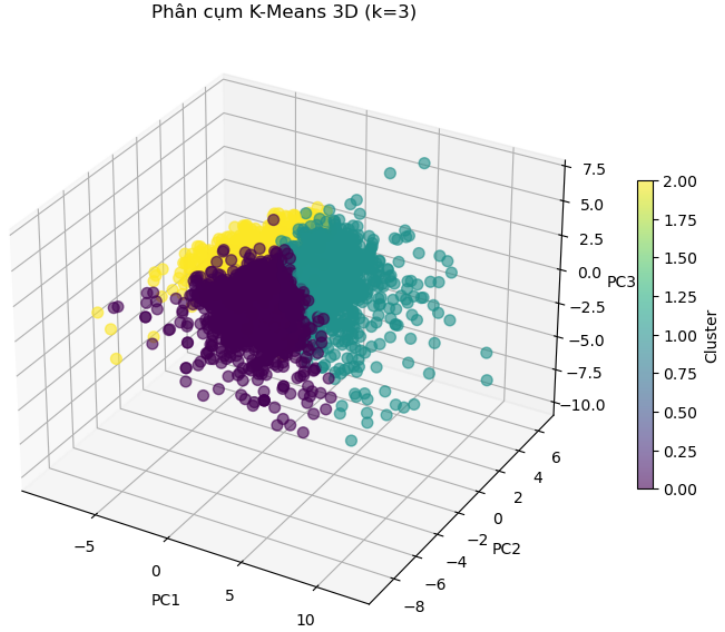
=> Điều này khớp với việc Silhouette score cao nhất ở **k=4**: khoảng cách giữa các cụm được cải thiện khi xét đủ 3 PC.



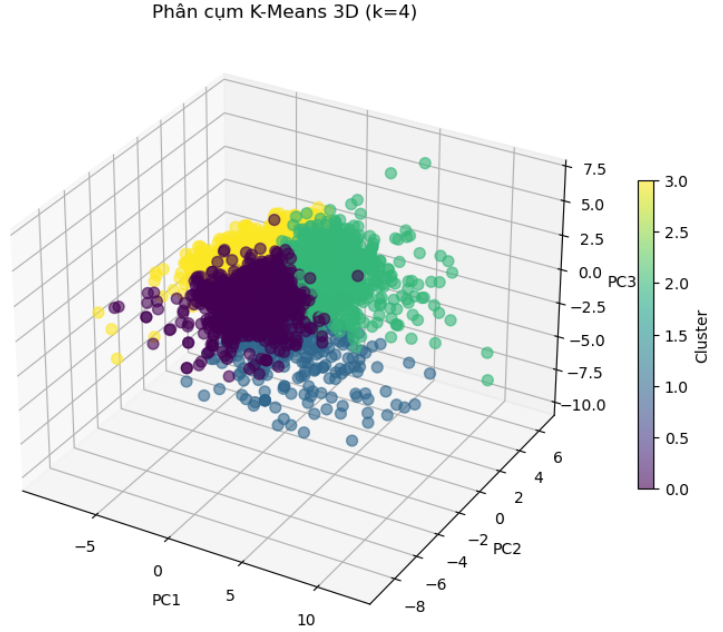
Phân cụm K-Means (k=3)



Phân cụm K-Means (k=4)

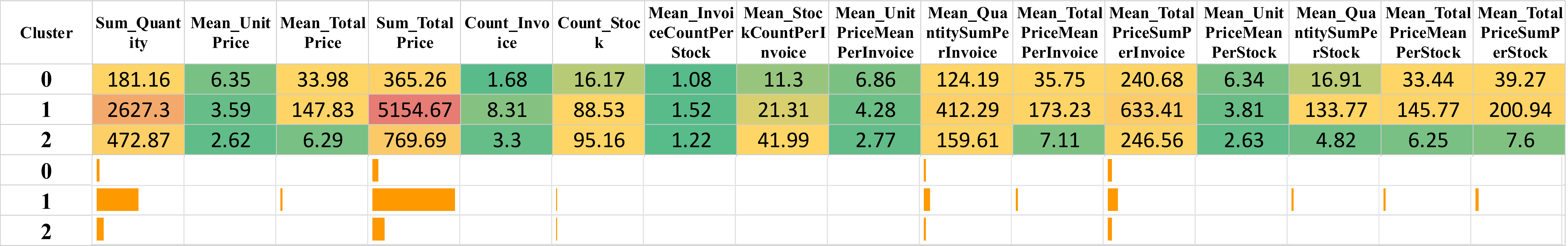


Phân cụm K-Means 3D (k=3)



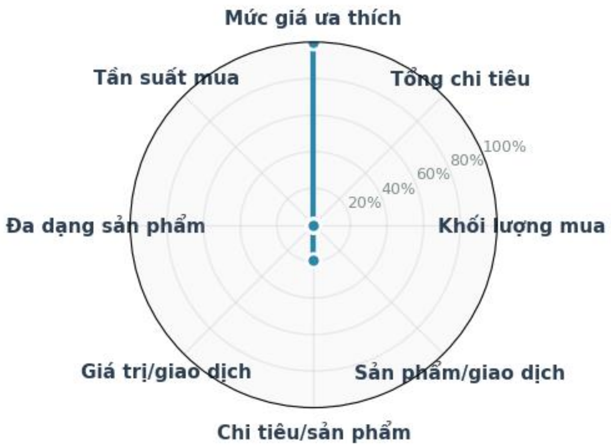
Phân cụm K-Means 3D (k=4)

Profile của các cụm (k=3)

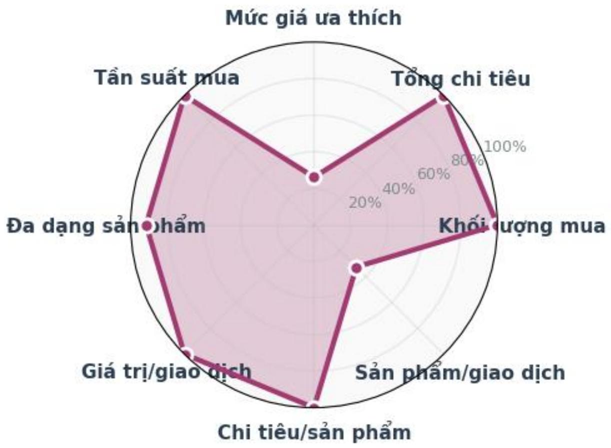


Cluster	Sum_Quantity	Mean_UnitPrice	Mean_TotalPrice	Sum_TotalPrice	Count_Invoice	Count_Stock	Mean_InvoiceCountPerStock	Mean_StockCountPerInvoice	Mean_UnitPriceMeanPerInvoice	Mean_QuantitySumPerInvoice	Mean_TotalPriceMeanPerInvoice	Mean_TotalPriceSumPerInvoice	Mean_UnitPriceMeanPerStock	Mean_QuantitySumPerStock	Mean_TotalPriceMeanPerStock	Mean_TotalPriceSumPerStock
0	181.16	6.35	33.98	365.26	1.68	16.17	1.08	11.3	6.86	124.19	35.75	240.68	6.34	16.91	33.44	39.27
1	2627.3	3.59	147.83	5154.67	8.31	88.53	1.52	21.31	4.28	412.29	173.23	633.41	3.81	133.77	145.77	200.94
2	472.87	2.62	6.29	769.69	3.3	95.16	1.22	41.99	2.77	159.61	7.11	246.56	2.63	4.82	6.25	7.6
0																
1																
2																

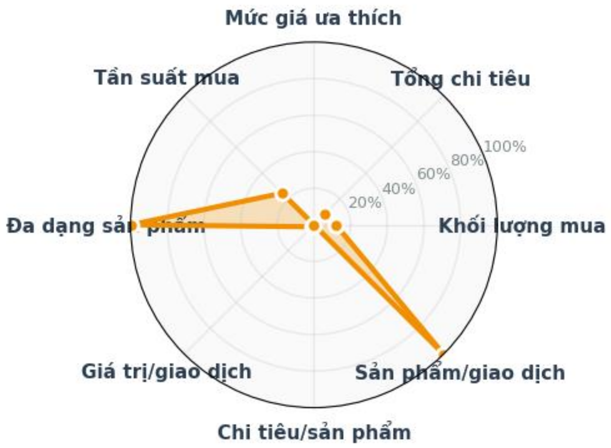
(0) Khách Premium ít mua



(1) Khách chủ lực mua số lượng lớn



(2) Khách săn hàng đa dạng giá rẻ

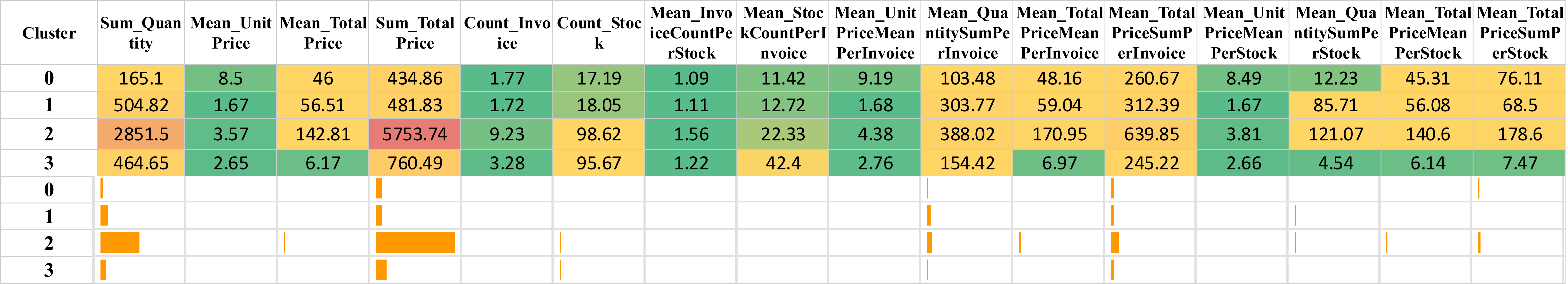


Cluster 0 – "Khách Premium ít mua"
- Ưa sản phẩm giá cao, nhưng mua ít lần và ít sản phẩm.

Cluster 1 – "Khách chủ lực mua số lượng lớn"
- Chi tiêu & khối lượng áp đảo, tần suất cao, mỗi giao dịch giá trị lớn.

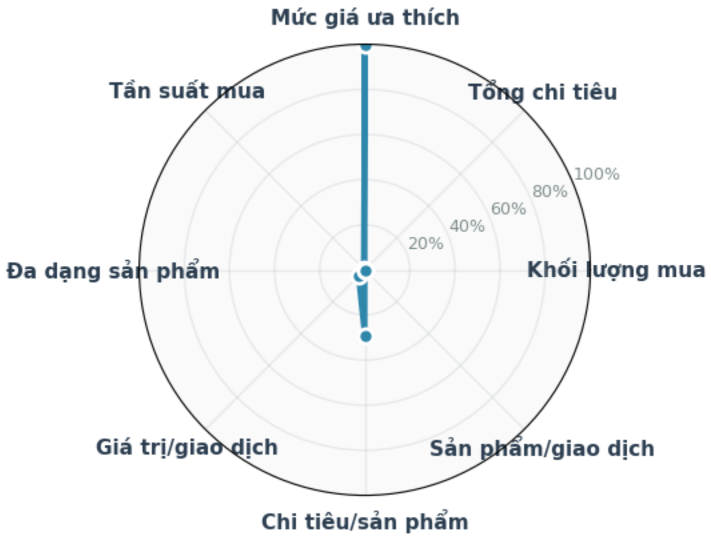
Cluster 2 – "Khách săn hàng đa dạng giá rẻ"
- Giá thấp, mỗi đơn nhiều mặt hàng khác nhau, chi tiêu/giao dịch không quá cao

Profile của các cụm (k=4)

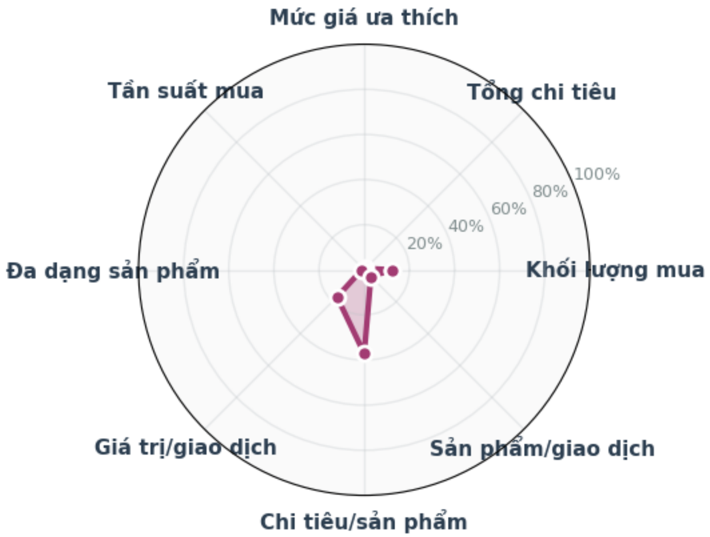


Cluster	Sum_Quantity	Mean_UnitPrice	Mean_TotalPrice	Sum_TotalPrice	Count_Invoice	Count_Stock	Mean_InvoiceCountPerStock	Mean_StockCountPerInvoice	Mean_UnitPriceMeanPerInvoice	Mean_QuantitySumPerInvoice	Mean_TotalPriceMeanPerInvoice	Mean_TotalPriceSumPerInvoice	Mean_UnitPriceMeanPerStock	Mean_QuantitySumPerStock	Mean_TotalPriceMeanPerStock	Mean_TotalPriceSumPerStock
0	165.1	8.5	46	434.86	1.77	17.19	1.09	11.42	9.19	103.48	48.16	260.67	8.49	12.23	45.31	76.11
1	504.82	1.67	56.51	481.83	1.72	18.05	1.11	12.72	1.68	303.77	59.04	312.39	1.67	85.71	56.08	68.5
2	2851.5	3.57	142.81	5753.74	9.23	98.62	1.56	22.33	4.38	388.02	170.95	639.85	3.81	121.07	140.6	178.6
3	464.65	2.65	6.17	760.49	3.28	95.67	1.22	42.4	2.76	154.42	6.97	245.22	2.66	4.54	6.14	7.47
0																
1																
2																
3																

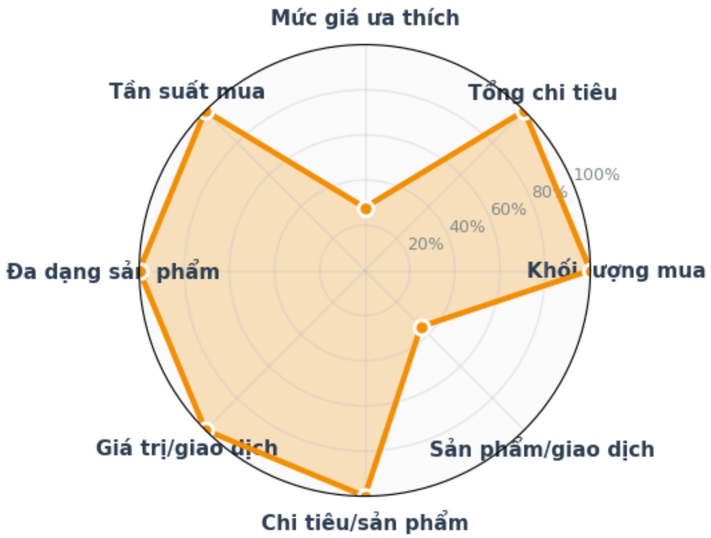
(0) Khách Premium ít mua



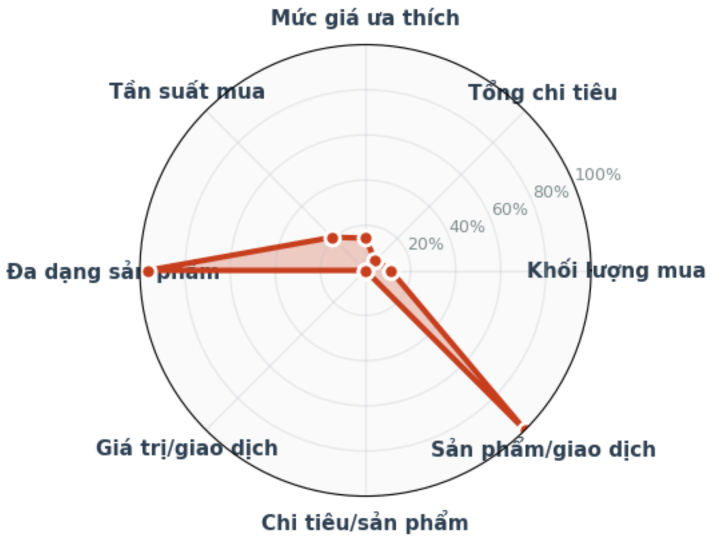
(1) Khách vãng lai



(2) Khách săn hàng đa dạng giá rẻ



(3) Khách săn hàng đa dạng giá rẻ



Cluster 0 – “Khách Premium ít mua”
- Ưa sản phẩm giá cao, nhưng mua ít lần và ít sản phẩm.

Cluster 1 – “Khách vãng lai”
- Nhóm khách có số lượng nhỏ, giá rẻ, mua ít

Cluster 2 – “Khách chủ lực mua số lượng lớn”
- Chi tiêu & khối lượng áp đảo, tần suất cao, mỗi giao dịch giá trị lớn

Cluster 3 – “Khách săn hàng đa dạng giá rẻ”
- Mỗi đơn nhiều mặt hàng khác nhau, chi tiêu/giao dịch không quá cao

Hướng ứng dụng Kinh doanh theo Cụm

Cụm 1: Khách Premium ít mua

Đặc trưng: Đơn giá cao nhất, Ít đơn, ít mã hàng

Chiến lược: Đẩy các gói dịch vụ cao cấp, limited, quà tặng sang. Mời tham gia membership hạng cao dù tần suất thấp, tập trung trải nghiệm hơn là giảm giá

KPI: Tăng tần suất mua mà không giảm giá quá sâu

Cụm 2: Khách vãng lai

Đặc trưng: Thích sản phẩm giá rất thấp

Chiến lược: Dùng promo đơn giản, voucher lần sau để kéo quay lại. Giảm giá sang sản phẩm giá cao hơn. Remarketing nhẹ, nhắc lại giỏ hàng

KPI: Tỷ lệ khách quay lại từ 1 → 2+ đơn, upgrade cụm 3 hoặc 4

Cụm 3: Khách chủ lực mua số lượng lớn

Đặc trưng: Mua thường xuyên, nhiều mã hàng, đơn & sản phẩm

Chiến lược: Đặt làm nhóm chiến lược: chương trình VIP, ưu đãi theo doanh số, account manager riêng. Khóa chặt bằng hợp đồng, subscription, ưu tiên tồn kho & giao hàng

KPI: Tỷ trọng doanh thu từ nhóm (duy trì hoặc tăng)

Cụm 4: Khách săn hàng đa dạng giá rẻ

Đặc trưng: Mỗi đơn có nhiều loại sản phẩm, đa dạng ngành hàng

Chiến lược: Gợi ý thêm sản phẩm liên quan, sản phẩm mới. Gói combo nhiều loại, subscription "hộp trải nghiệm"

KPI: Tăng số lượng ngành hàng/mã hàng trên mỗi khách

Hướng mở rộng trong tương lai

Model Enhancement

Thử các thuật toán khác

DBSCAN cho cụm phi tuyến, **Hierarchical clustering** cho cấu trúc lồng nhau, **Gaussian Mixture** cho soft clustering.

Mở rộng Features

Seasonal features (mùa vụ), product categories (danh mục chi tiết), geographic expansion, time decay weights

Phân cụm theo sản phẩm

Phân tích sâu nhóm sản phẩm đặc thù

Business Applications

Recommendation System

Product recommendations cá nhân hoá, collaborative filtering.

Price Optimization

Dynamic pricing, WTP modeling, promotional targeting.

Churn Prediction

Early warning system, supervised learning cho at-risk customers.

CLV Modeling

Customer Lifetime Value prediction & Resource allocation.

Multi-Agent Simulation

Mô phỏng behavior khách hàng, chiến lược marketing sâu.

Tổng kết

Dự án đã thành công phân khúc ~3,920 khách hàng thành 4 cụm rõ ràng. Mỗi cụm có đặc trưng business logic mạnh mẽ và actionable insights cụ thể.

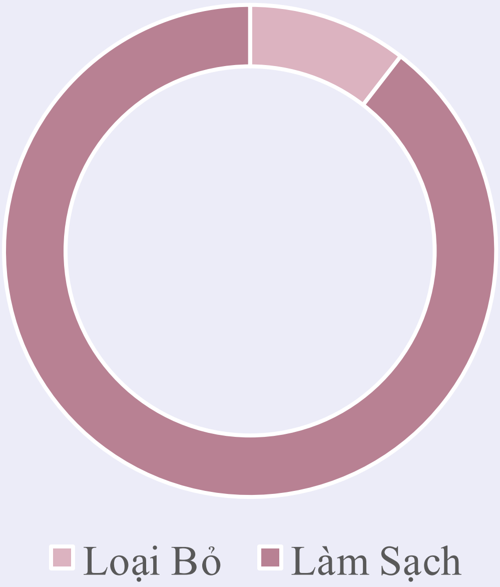
Tổng Giao Dịch Gốc
541,909
Dữ liệu thô ban đầu (Raw Data)

Giao Dịch Hợp Lệ (UK)
354,321
✓ Đã làm sạch

Khách Hàng (UK)
3,920
Số lượng khách hàng

Cụm Khách Hàng
4
3,920 khách hàng

Tỉ Lệ Dữ Liệu Sạch



Dữ liệu giảm 10.5% sau khi lọc nhiễu và giới hạn tại UK

Các Phát Hiện Chính (Key Findings)

Xu Hướng Doanh Thu

Doanh thu cho thấy xu hướng tăng dần theo thời gian trong giai đoạn khảo sát.

Thời Gian Mua Hàng

Hầu hết việc mua hàng diễn ra trong giờ làm việc (các ngày trong tuần).

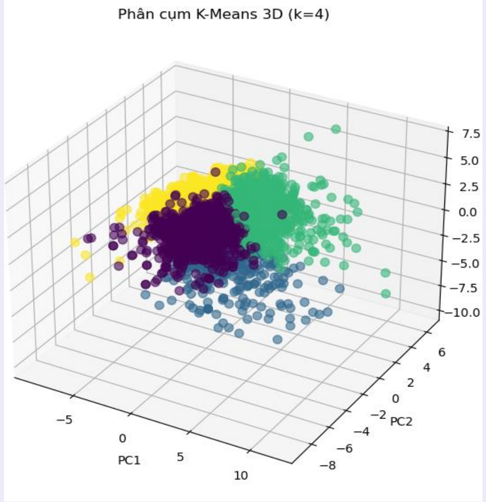
Hành Vi Khách Hàng

Chi tiêu và tần suất giao dịch của khách hàng rất đa dạng.

Quy Trình Xử Lý Đã Thực Hiện

- ✓ Loại bỏ bản ghi thiếu CustomerID
- ✓ Lọc dữ liệu quốc gia: UK
- ✓ Loại bỏ Quantity/Price âm
- ✓ Chuẩn hóa định dạng ngày tháng

Plot 3D
4 Cụm Khách Hàng



Appendix

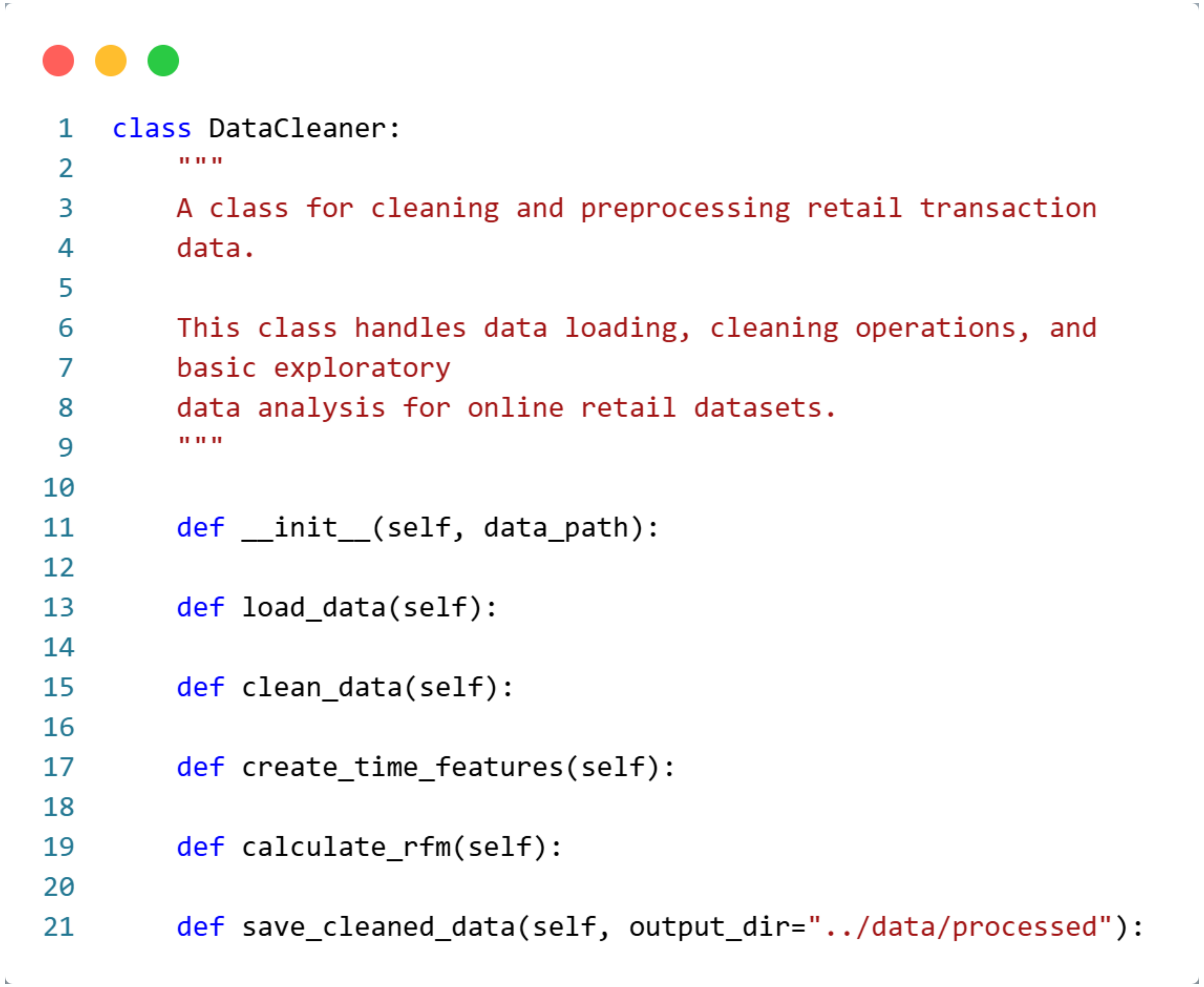
Giải thích các class sử dụng trong source code

Class 1: DataCleaner

Mục đích: Xử lý và làm sạch dữ liệu giao dịch bán lẻ từ file raw

Chức năng chính:

- Load dữ liệu từ file CSV
- Làm sạch dữ liệu (loại bỏ missing values, invalid records)
- Tạo các time features
- Tính toán RFM metrics (Recency, Frequency, Monetary)
- Save data đã được clean vào folder cụ thể

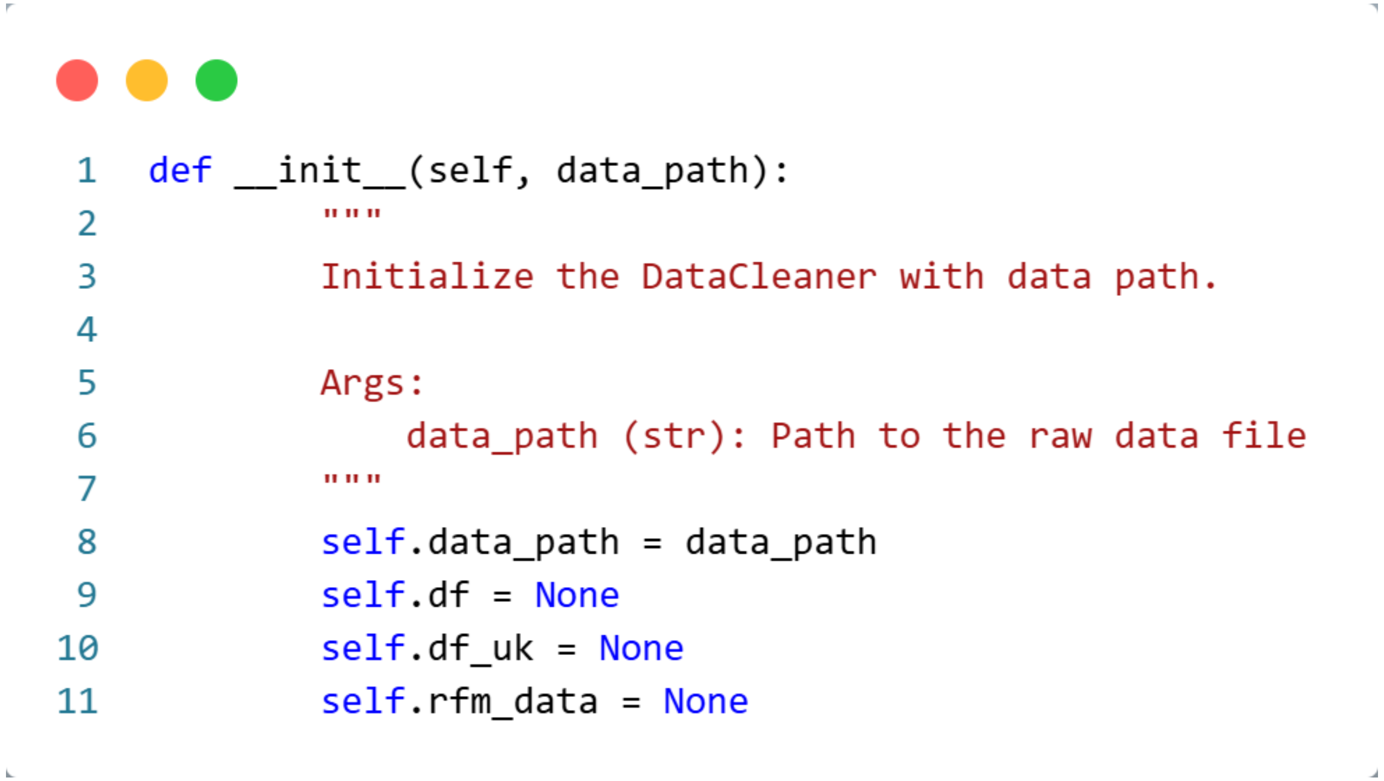


```python
class DataCleaner:
    """
    A class for cleaning and preprocessing retail transaction
    data.

    This class handles data loading, cleaning operations, and
    basic exploratory
    data analysis for online retail datasets.
    """

    def __init__(self, data_path):

    def load_data(self):

    def clean_data(self):

    def create_time_features(self):

    def calculate_rfm(self):

    def save_cleaned_data(self, output_dir="../data/processed"):
```

Class 1: DataCleaner

__init__(self, data_path): Khởi tạo object với đường dẫn file dữ liệu

- data_path: Lưu đường dẫn đến file dữ liệu gốc
- df: Sẽ chứa dataframe gốc sau khi load
- df_uk: Sẽ chứa dataframe đã lọc (chỉ UK customers)
- rfm_data: Sẽ chứa RFM metrics đã tính toán



```python
def __init__(self, data_path):
        """
        Initialize the DataCleaner with data path.

        Args:
            data_path (str): Path to the raw data file
        """
        self.data_path = data_path
        self.df = None
        self.df_uk = None
        self.rfm_data = None
```

Class 1: DataCleaner

load_data(self): Load dữ liệu từ CSV và format các columns

1. Định nghĩa kiểu dữ liệu cho từng cột để đảm bảo load đúng format

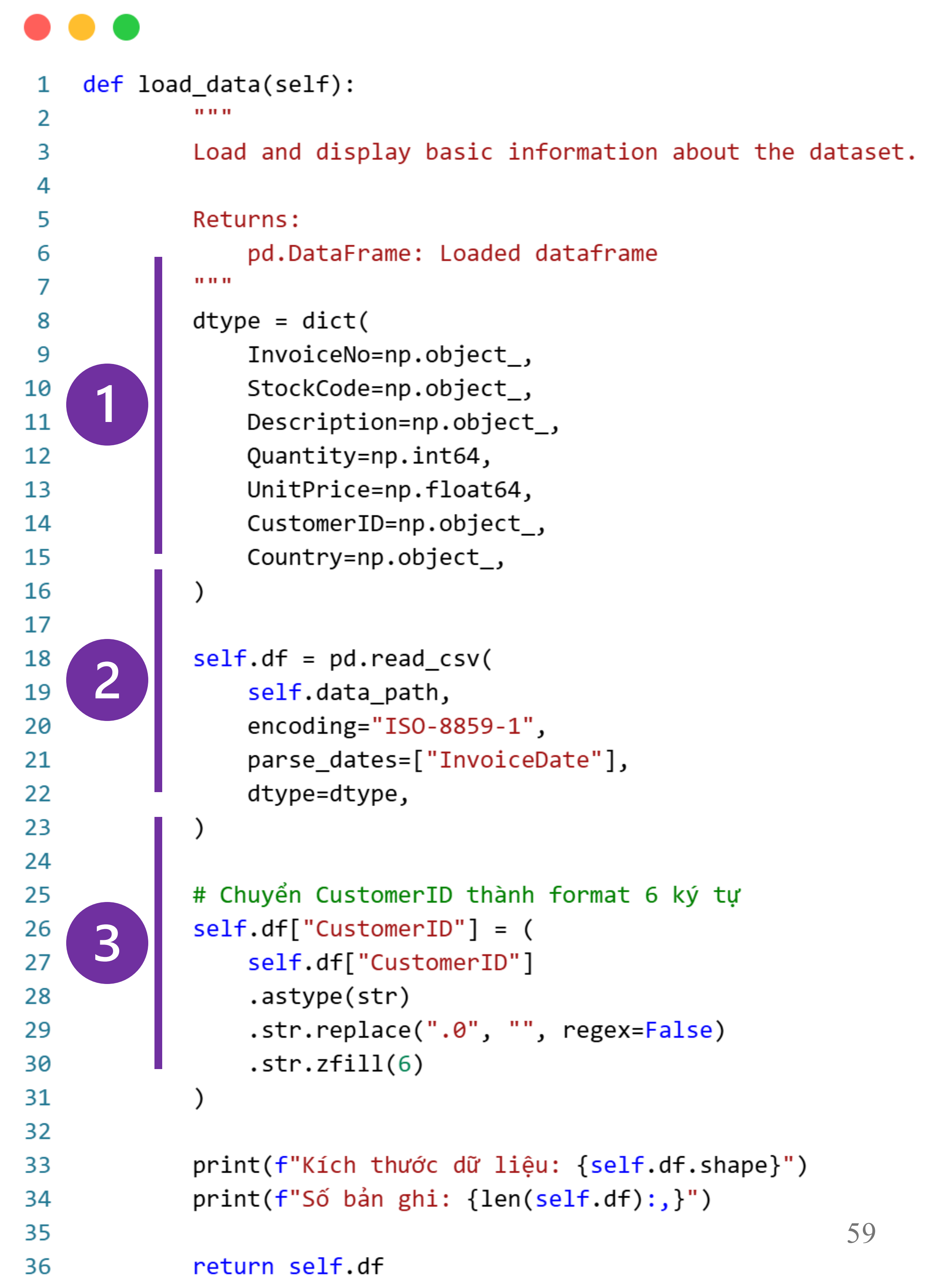
- object_: cho string/text columns
- int64: cho số nguyên
- float64: cho số thực

2. Đọc file CSV

- encoding="ISO-8859-1": Xử lý các ký tự đặc biệt trong dữ liệu
- parse_dates: Tự động convert cột InvoiceDate sang datetime
- dtype: Áp dụng các kiểu dữ liệu đã định nghĩa

3. Format CustomerID

- .astype(str): Convert sang string
- .str.replace(".0", ""): Xóa ".0" thừa
- .str.zfill(6): Padding thành 6 ký tự (ví dụ: "123" → "000123")



```python
def load_data(self):
        """
        Load and display basic information about the dataset.

        Returns:
            pd.DataFrame: Loaded dataframe
        """
        dtype = dict(
            InvoiceNo=np.object_,
            StockCode=np.object_,
            Description=np.object_,
            Quantity=np.int64,
            UnitPrice=np.float64,
            CustomerID=np.object_,
            Country=np.object_,
        )

        self.df = pd.read_csv(
            self.data_path,
            encoding="ISO-8859-1",
            parse_dates=["InvoiceDate"],
            dtype=dtype,
        )

        # Chuyển CustomerID thành format 6 ký tự
        self.df["CustomerID"] = (
            self.df["CustomerID"]
            .astype(str)
            .str.replace(".0", "", regex=False)
            .str.zfill(6)
        )

        print(f"Kích thước dữ liệu: {self.df.shape}")
        print(f"Số bản ghi: {len(self.df):,}")

        return self.df
```

Class 1: DataCleaner

clean_data(self): Làm sạch dữ liệu và lọc khách hàng UK

1. Tạo cột TotalPrice

- Tính tổng giá trị mỗi dòng giao dịch
- TotalPrice = Số lượng × Đơn giá

2. Loại bỏ hóa đơn bị hủy

- giữ lại các hóa đơn KHÔNG bắt đầu bằng "C"

3. Lọc khách hàng UK

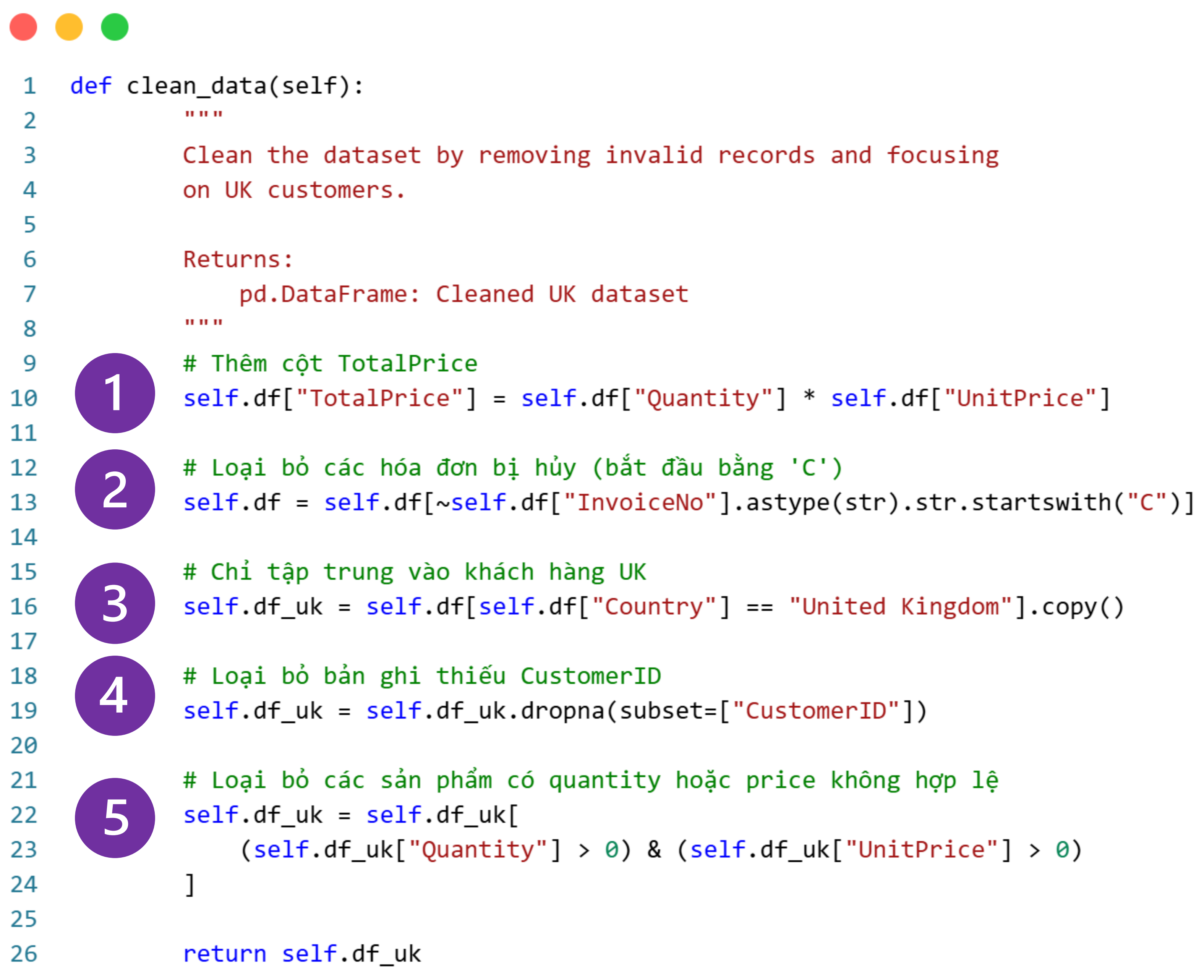
- Tạo bản sao độc lập chỉ giữ khách hàng UK

4. Xử lý missing values

- Loại bỏ sample không có CustomerID

5. Lọc dữ liệu hợp lệ

- Chỉ giữ transactions có Quantity và Price dương
- Loại bỏ các giá trị âm hoặc 0 (dữ liệu lỗi)

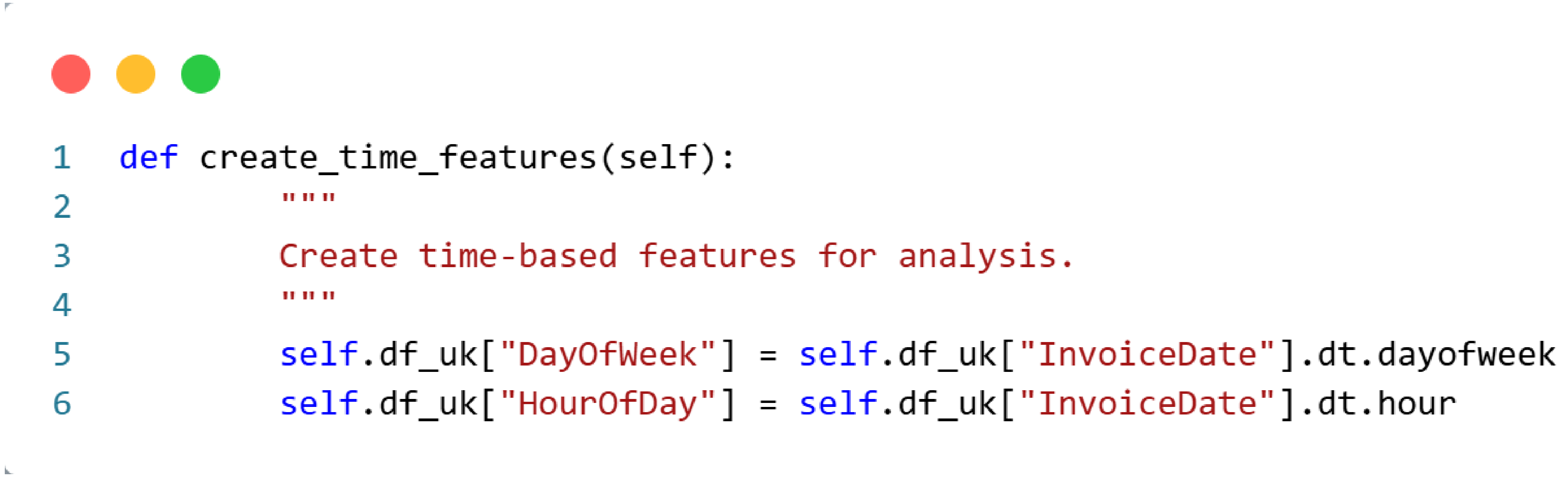


```python
def clean_data(self):
        """
        Clean the dataset by removing invalid records and focusing
        on UK customers.

        Returns:
            pd.DataFrame: Cleaned UK dataset
        """
        # Thêm cột TotalPrice
        self.df["TotalPrice"] = self.df["Quantity"] * self.df["UnitPrice"]

        # Loại bỏ các hóa đơn bị hủy (bắt đầu bằng 'C')
        self.df = self.df[~self.df["InvoiceNo"].astype(str).str.startswith("C")]

        # Chỉ tập trung vào khách hàng UK
        self.df_uk = self.df[self.df["Country"] == "United Kingdom"].copy()

        # Loại bỏ bản ghi thiếu CustomerID
        self.df_uk = self.df_uk.dropna(subset=["CustomerID"])

        # Loại bỏ các sản phẩm có quantity hoặc price không hợp lệ
        self.df_uk = self.df_uk[
            (self.df_uk["Quantity"] > 0) & (self.df_uk["UnitPrice"] > 0)
        ]

        return self.df_uk
```

Class 1: DataCleaner

create_time_features(self): Tạo các features dựa trên thời gian

- Extract ngày trong tuần và giờ trong ngày
- Hữu ích cho phân tích seasonal patterns



```python
def create_time_features(self):
        """
        Create time-based features for analysis.
        """
        self.df_uk["DayOfWeek"] = self.df_uk["InvoiceDate"].dt.dayofweek
        self.df_uk["HourOfDay"] = self.df_uk["InvoiceDate"].dt.hour
```

Class 1: DataCleaner

calculate_rfm(self): Tính toán RFM metrics cho mỗi khách hàng

1. Định nghĩa snapshot date

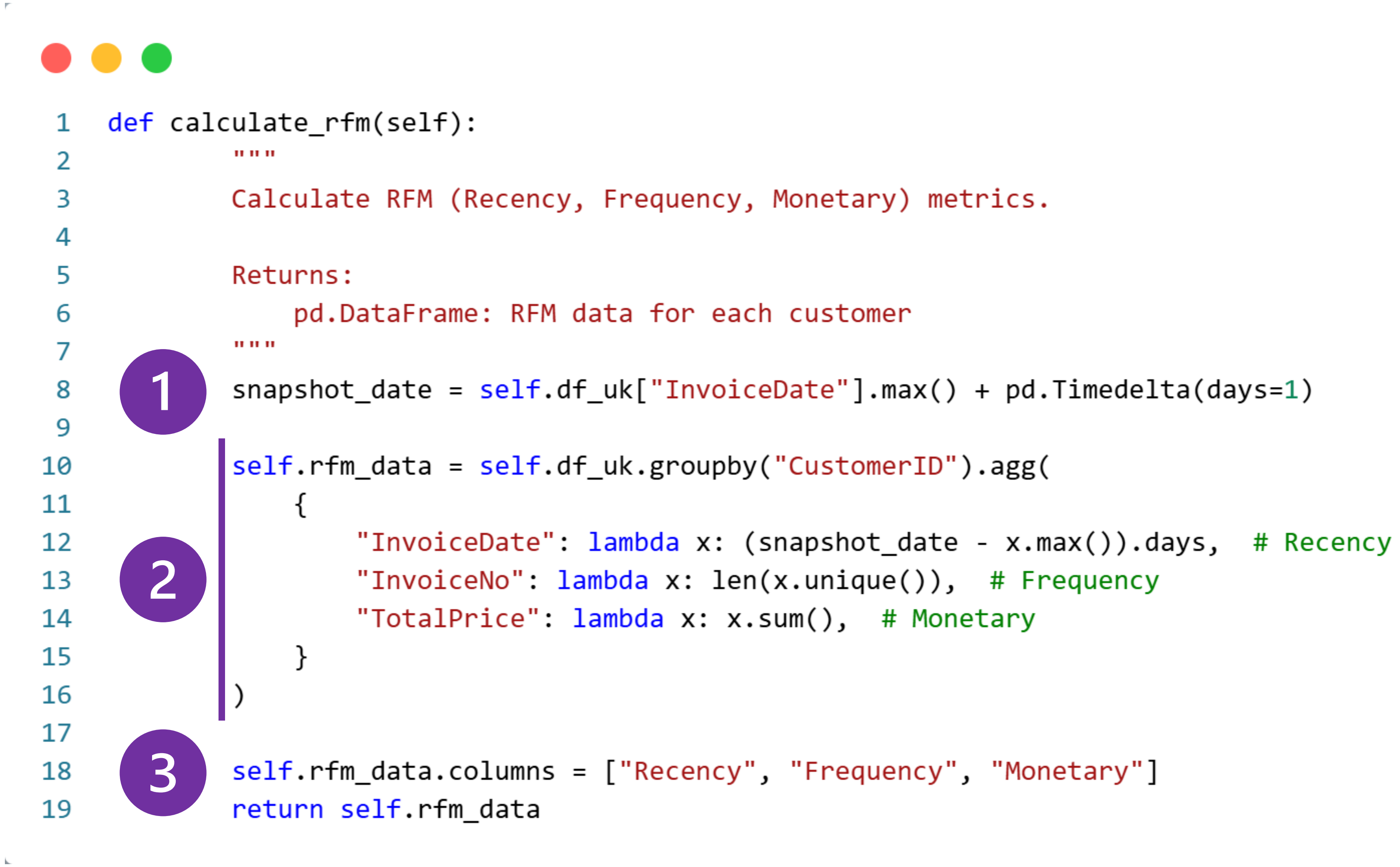
- Lấy ngày sau cùng trong dataset + 1 ngày
- Dùng làm mốc để tính Recency

2. Tính RFM

- Recency: Số ngày kể từ lần mua cuối đến snapshot_date
- Frequency: Số hóa đơn unique (số lần mua)
- Monetary: Tổng tiền đã chi tiêu

3. Đổi tên columns

- Đặt tên columns rõ ràng

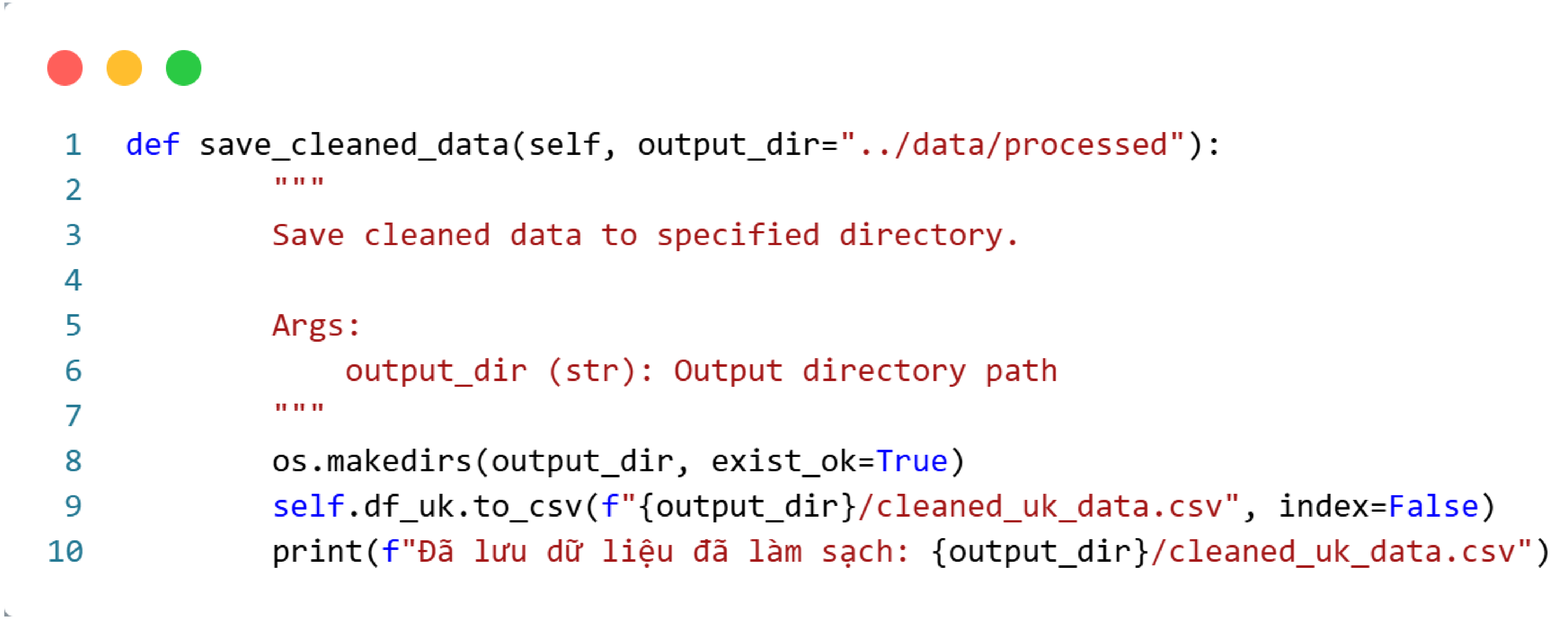


```python
def calculate_rfm(self):
        """
        Calculate RFM (Recency, Frequency, Monetary) metrics.

        Returns:
            pd.DataFrame: RFM data for each customer
        """
        snapshot_date = self.df_uk["InvoiceDate"].max() + pd.Timedelta(days=1)

        self.rfm_data = self.df_uk.groupby("CustomerID").agg(
            {
                "InvoiceDate": lambda x: (snapshot_date - x.max()).days,  # Recency
                "InvoiceNo": lambda x: len(x.unique()),  # Frequency
                "TotalPrice": lambda x: x.sum(),  # Monetary
            }
        )

        self.rfm_data.columns = ["Recency", "Frequency", "Monetary"]
        return self.rfm_data
```

Class 1: DataCleaner

save_cleaned_data(self, output_dir): Lưu dữ liệu đã làm sạch ra file

- exist_ok=True: Không báo lỗi nếu folder đã tồn tại
- index=False: Không lưu row index vào CSV



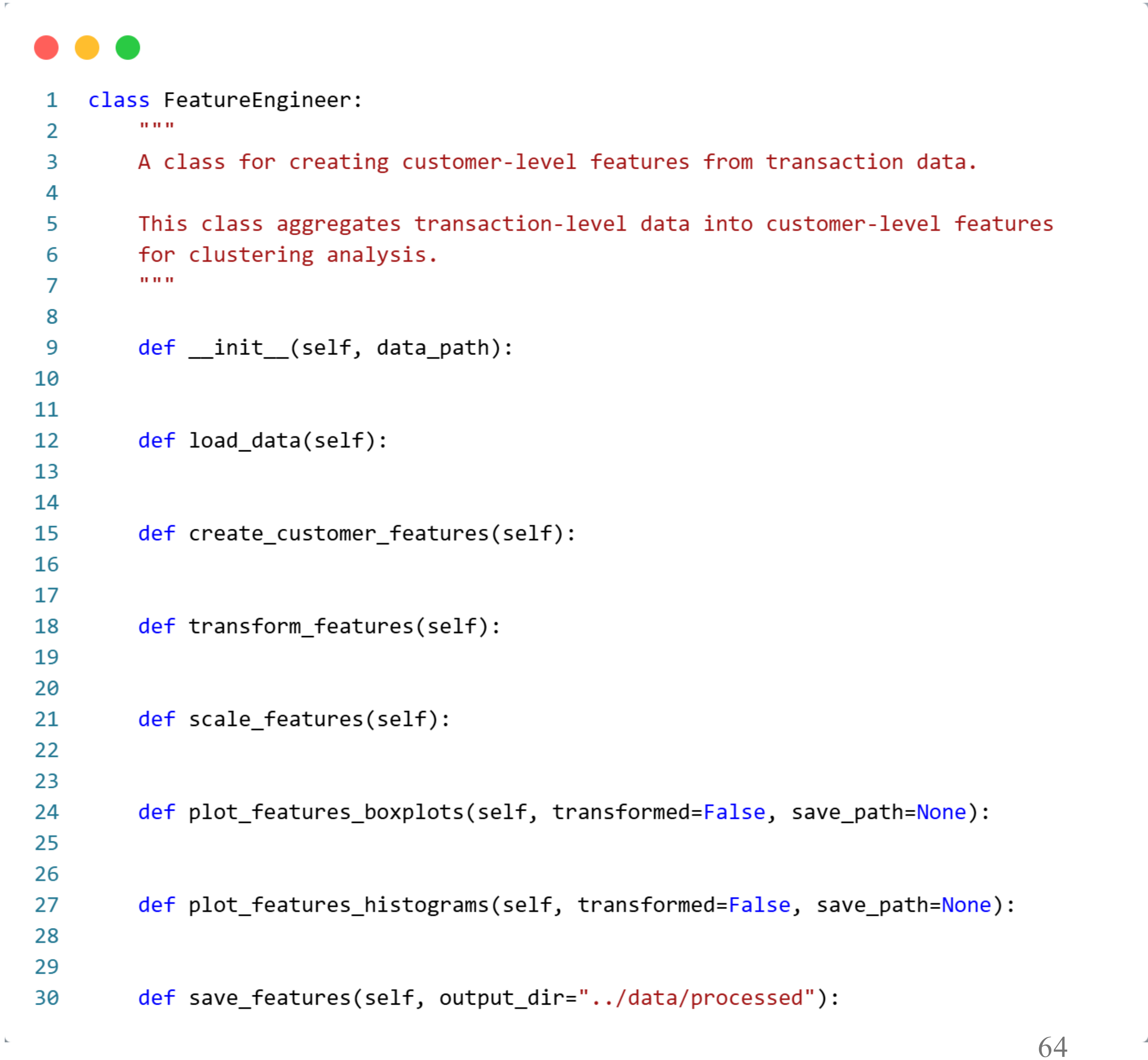
```python
def save_cleaned_data(self, output_dir="../data/processed"):
        """
        Save cleaned data to specified directory.

        Args:
            output_dir (str): Output directory path
        """
        os.makedirs(output_dir, exist_ok=True)
        self.df_uk.to_csv(f"{output_dir}/cleaned_uk_data.csv", index=False)
        print(f"Đã lưu dữ liệu đã làm sạch: {output_dir}/cleaned_uk_data.csv")
```

Class 2: FeatureEngineer

Mục đích: Tạo các features tổng hợp ở mức khách hàng từ dữ liệu giao dịch

Chức năng chính:

- Load clean data và chuẩn bị cho feature engineering
- Tổng hợp features hành vi mua sắm cho mỗi khách hàng
- Áp dụng Box-Cox và standardization để chuẩn hóa phân phối
- Vẽ box plots và histograms cho các features
- Save các feature đã được xử lý



```python
class FeatureEngineer:
    """
    A class for creating customer-level features from transaction data.

    This class aggregates transaction-level data into customer-level features
    for clustering analysis.
    """

    def __init__(self, data_path):


    def load_data(self):


    def create_customer_features(self):


    def transform_features(self):


    def scale_features(self):


    def plot_features_boxplots(self, transformed=False, save_path=None):


    def plot_features_histograms(self, transformed=False, save_path=None):


    def save_features(self, output_dir="../data/processed"):
```

Class 2: FeatureEngineer

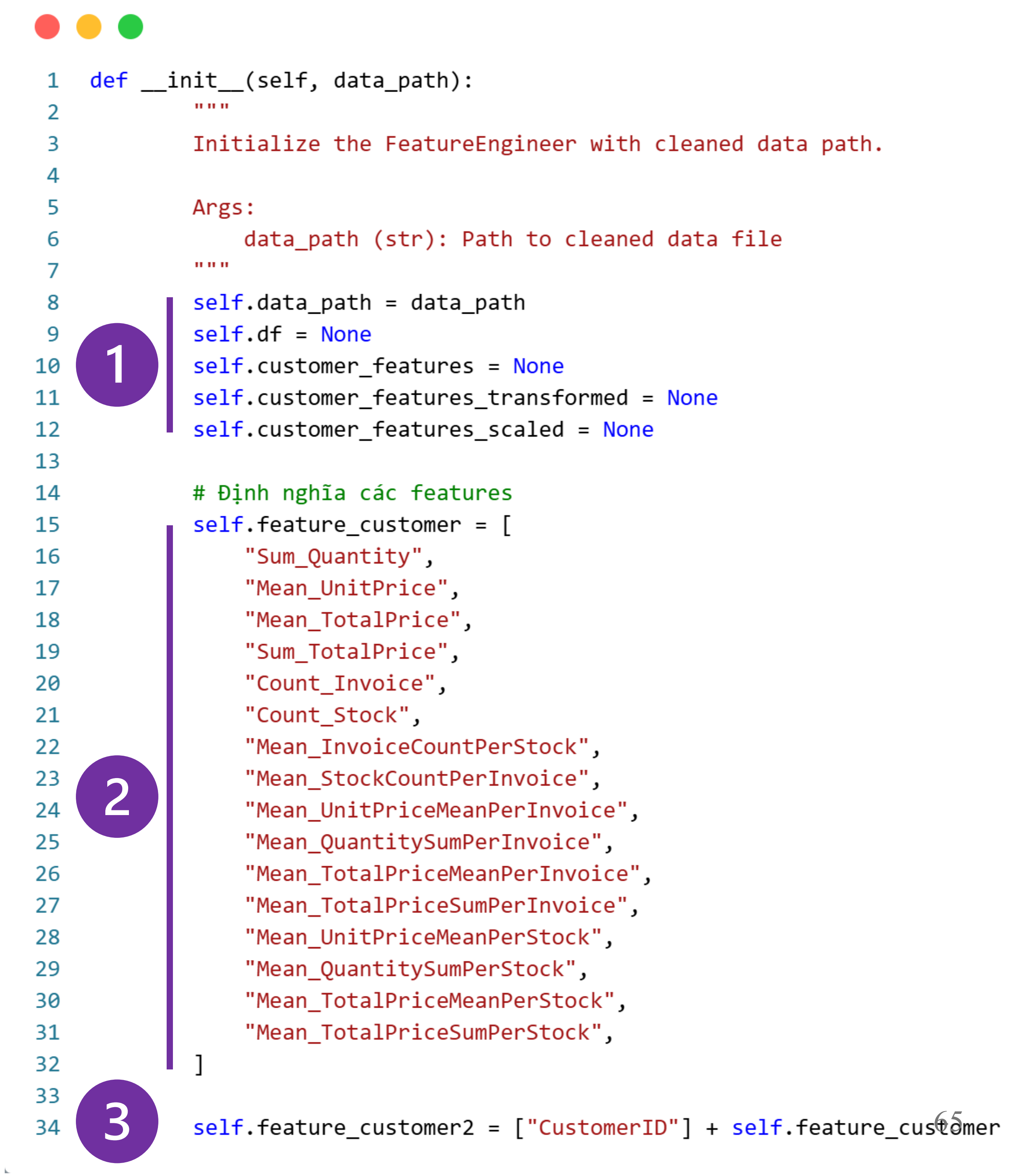
__init__(self, data_path): Khởi tạo với các biến và định nghĩa features

1. Khởi tạo dataframes

2 Định nghĩa danh sách features

3. Thêm CustomerID vào list

- Tạo list đầy đủ bao gồm cả CustomerID column

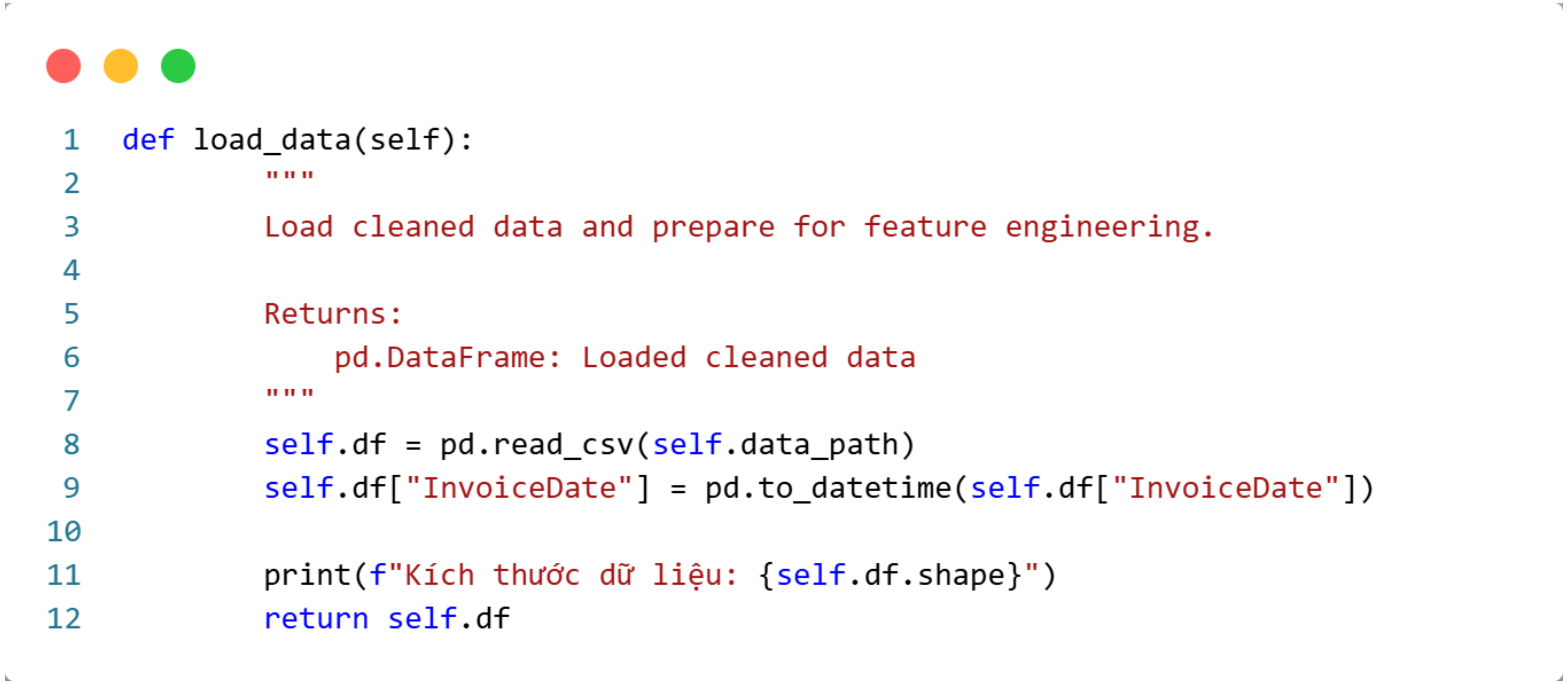


```python
def __init__(self, data_path):
        """
        Initialize the FeatureEngineer with cleaned data path.

        Args:
            data_path (str): Path to cleaned data file
        """
        self.data_path = data_path
        self.df = None
        self.customer_features = None
        self.customer_features_transformed = None
        self.customer_features_scaled = None

        # Định nghĩa các features
        self.feature_customer = [
            "Sum_Quantity",
            "Mean_UnitPrice",
            "Mean_TotalPrice",
            "Sum_TotalPrice",
            "Count_Invoice",
            "Count_Stock",
            "Mean_InvoiceCountPerStock",
            "Mean_StockCountPerInvoice",
            "Mean_UnitPriceMeanPerInvoice",
            "Mean_QuantitySumPerInvoice",
            "Mean_TotalPriceMeanPerInvoice",
            "Mean_TotalPriceSumPerInvoice",
            "Mean_UnitPriceMeanPerStock",
            "Mean_QuantitySumPerStock",
            "Mean_TotalPriceMeanPerStock",
            "Mean_TotalPriceSumPerStock",
        ]

        self.feature_customer2 = ["CustomerID"] + self.feature_customer
```

Class 2: FeatureEngineer

load_data(self): Load dữ liệu đã làm sạch

- Load dữ liệu đã làm sạch và convert InvoiceDate từ string → datetime để xử lý



```python
def load_data(self):
        """
        Load cleaned data and prepare for feature engineering.

        Returns:
            pd.DataFrame: Loaded cleaned data
        """
        self.df = pd.read_csv(self.data_path)
        self.df["InvoiceDate"] = pd.to_datetime(self.df["InvoiceDate"])

        print(f"Kích thước dữ liệu: {self.df.shape}")
        return self.df
```

Class 2: FeatureEngineer

create_customer_features(self): Tạo 16 features tổng hợp cho mỗi khách hàng

1. Khởi tạo DataFrame rỗng

- Tạo DataFrame với số rows = số khách hàng
- Khởi tạo tất cả giá trị = 0

2. Set CustomerID column type

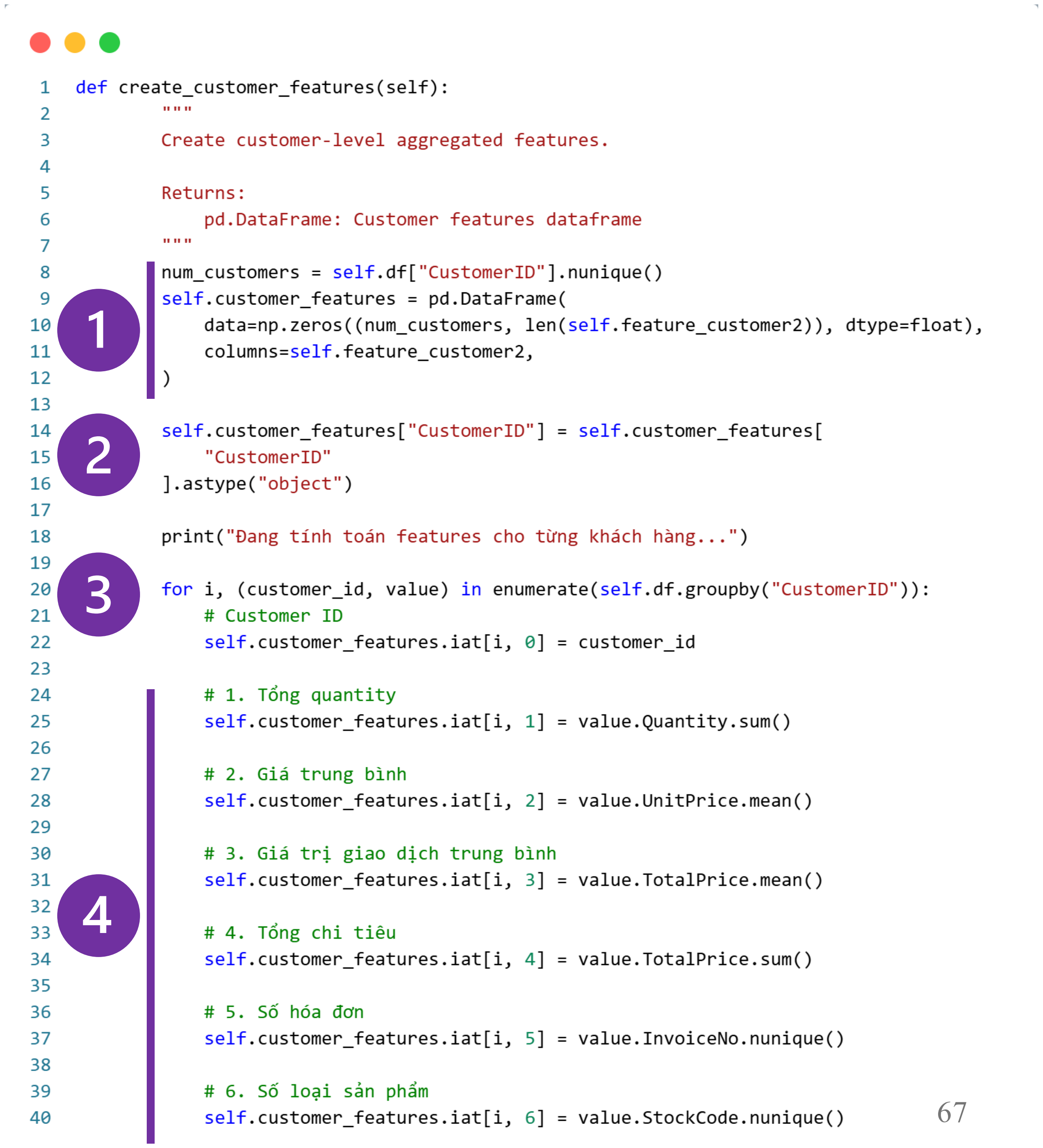
- Convert CustomerID column sang kiểu object (string)

3. Loop qua từng khách hàng

- Nhóm tất cả transactions theo từng khách hàng

4. Tính các features đơn giản (1-6)

- Truy cập nhanh cell theo vị trí (row i, column col)
- Các phép tính trực tiếp: sum(), mean(), nunique()



```python
def create_customer_features(self):
        """
        Create customer-level aggregated features.

        Returns:
            pd.DataFrame: Customer features dataframe
        """
        num_customers = self.df["CustomerID"].nunique()
        self.customer_features = pd.DataFrame(
            data=np.zeros((num_customers, len(self.feature_customer2)), dtype=float),
            columns=self.feature_customer2,
        )

        self.customer_features["CustomerID"] = self.customer_features[
            "CustomerID"
        ].astype("object")

        print("Đang tính toán features cho từng khách hàng...")

        for i, (customer_id, value) in enumerate(self.df.groupby("CustomerID")):
            # Customer ID
            self.customer_features.iat[i, 0] = customer_id

            # 1. Tổng quantity
            self.customer_features.iat[i, 1] = value.Quantity.sum()

            # 2. Giá trung bình
            self.customer_features.iat[i, 2] = value.UnitPrice.mean()

            # 3. Giá trị giao dịch trung bình
            self.customer_features.iat[i, 3] = value.TotalPrice.mean()

            # 4. Tổng chi tiêu
            self.customer_features.iat[i, 4] = value.TotalPrice.sum()

            # 5. Số hóa đơn
            self.customer_features.iat[i, 5] = value.InvoiceNo.nunique()

            # 6. Số loại sản phẩm
            self.customer_features.iat[i, 6] = value.StockCode.nunique()
```

Class 2: FeatureEngineer

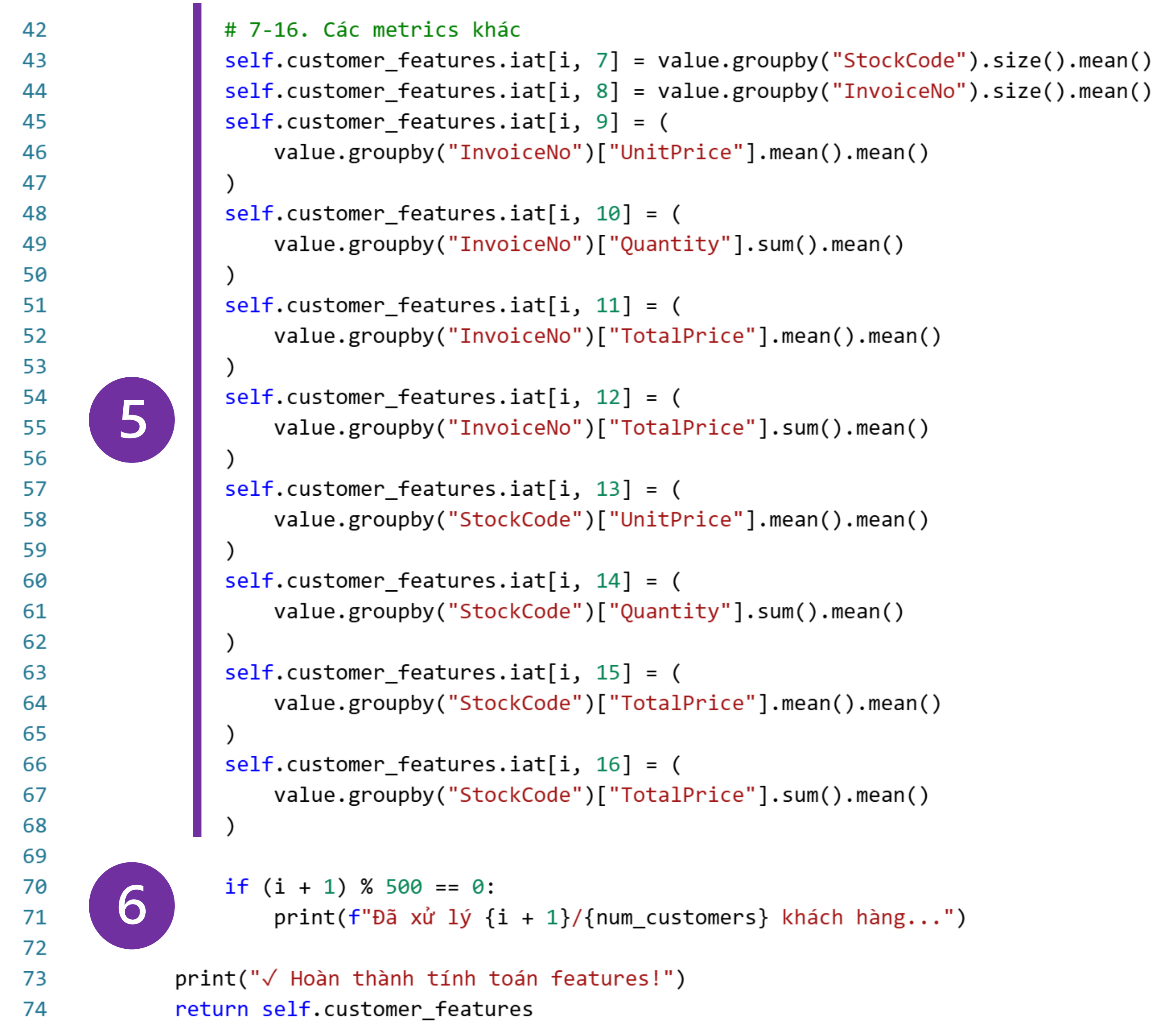
create_customer_features(self): Tạo 16 features tổng hợp cho mỗi khách hàng

5. Tính các features phức tạp (7-16)

- Tính trung bình của các giá trị trung bình

6. Progress tracking

- In thông báo tiến độ mỗi 500 khách hàng



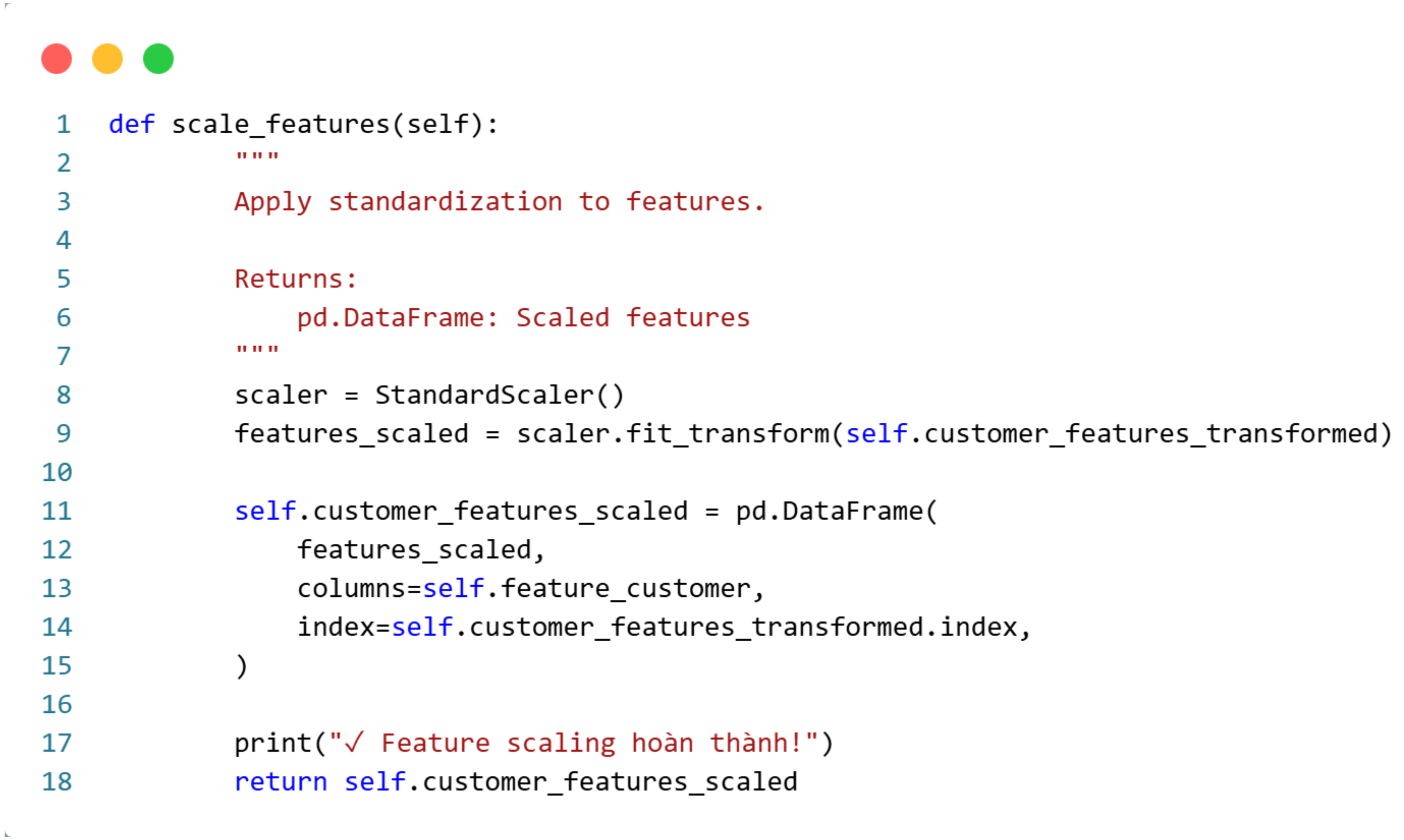
```python
            # 7-16. Các metrics khác
            self.customer_features.iat[i, 7] = value.groupby("StockCode").size().mean()
            self.customer_features.iat[i, 8] = value.groupby("InvoiceNo").size().mean()
            self.customer_features.iat[i, 9] = (
                value.groupby("InvoiceNo")["UnitPrice"].mean().mean()
            )
            self.customer_features.iat[i, 10] = (
                value.groupby("InvoiceNo")["Quantity"].sum().mean()
            )
            self.customer_features.iat[i, 11] = (
                value.groupby("InvoiceNo")["TotalPrice"].mean().mean()
            )
            self.customer_features.iat[i, 12] = (
                value.groupby("InvoiceNo")["TotalPrice"].sum().mean()
            )
            self.customer_features.iat[i, 13] = (
                value.groupby("StockCode")["UnitPrice"].mean().mean()
            )
            self.customer_features.iat[i, 14] = (
                value.groupby("StockCode")["Quantity"].sum().mean()
            )
            self.customer_features.iat[i, 15] = (
                value.groupby("StockCode")["TotalPrice"].mean().mean()
            )
            self.customer_features.iat[i, 16] = (
                value.groupby("StockCode")["TotalPrice"].sum().mean()
            )

            if (i + 1) % 500 == 0:
                print(f"Đã xử lý {i + 1}/{num_customers} khách hàng...")

        print("✓ Hoàn thành tính toán features!")
        return self.customer_features
```

Class 2: FeatureEngineer

scale_features(self): Standardization (mean=0, std=1)

Standardization

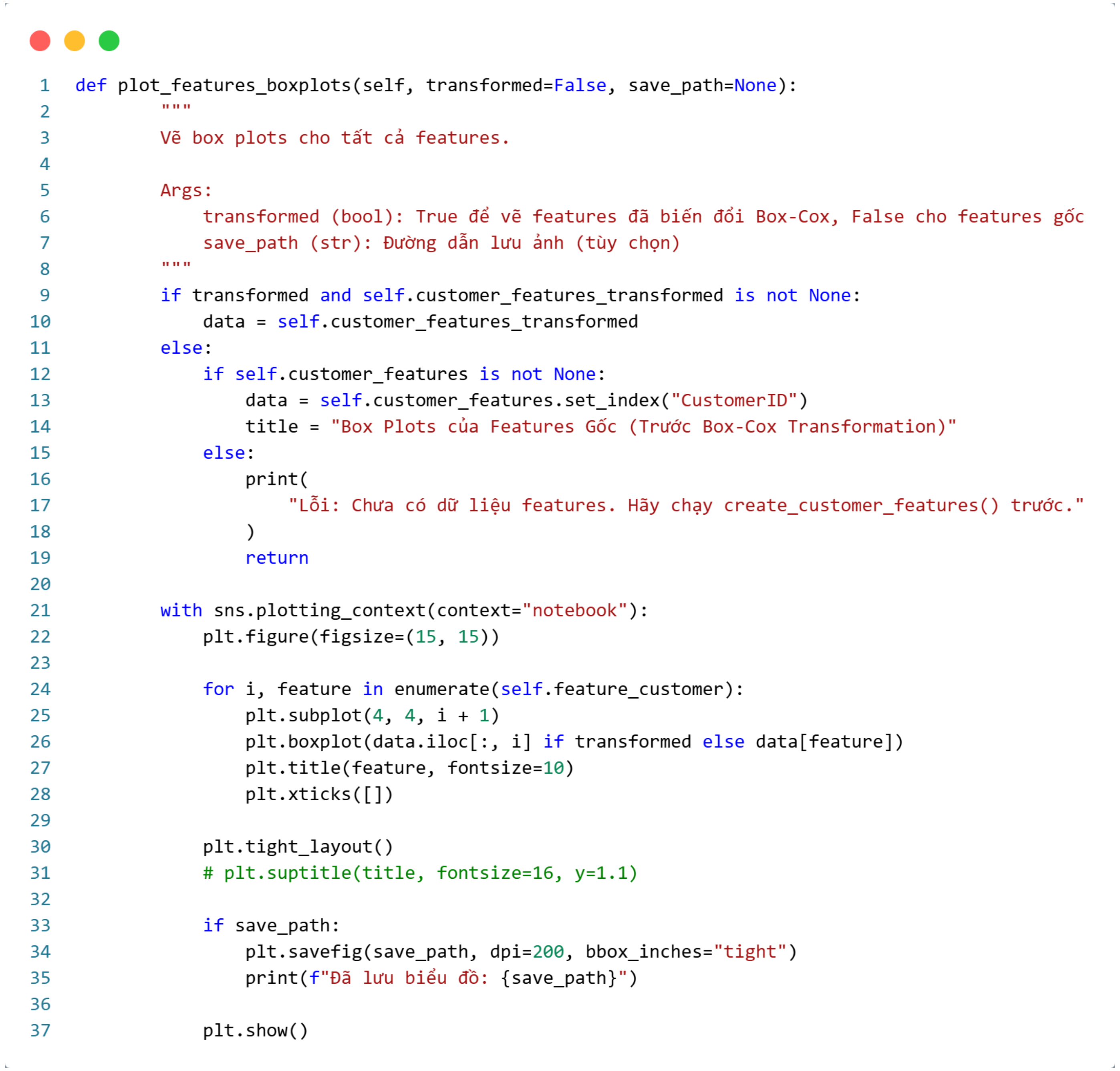
- Công thức: (x - mean) / std
- Đưa tất cả features về cùng scale (mean=0, std=1)
- Quan trọng cho clustering algorithms



```python
def scale_features(self):
        """
        Apply standardization to features.

        Returns:
            pd.DataFrame: Scaled features
        """
        scaler = StandardScaler()
        features_scaled = scaler.fit_transform(self.customer_features_transformed)

        self.customer_features_scaled = pd.DataFrame(
            features_scaled,
            columns=self.feature_customer,
            index=self.customer_features_transformed.index,
        )

        print("✓ Feature scaling hoàn thành!")
        return self.customer_features_scaled
```

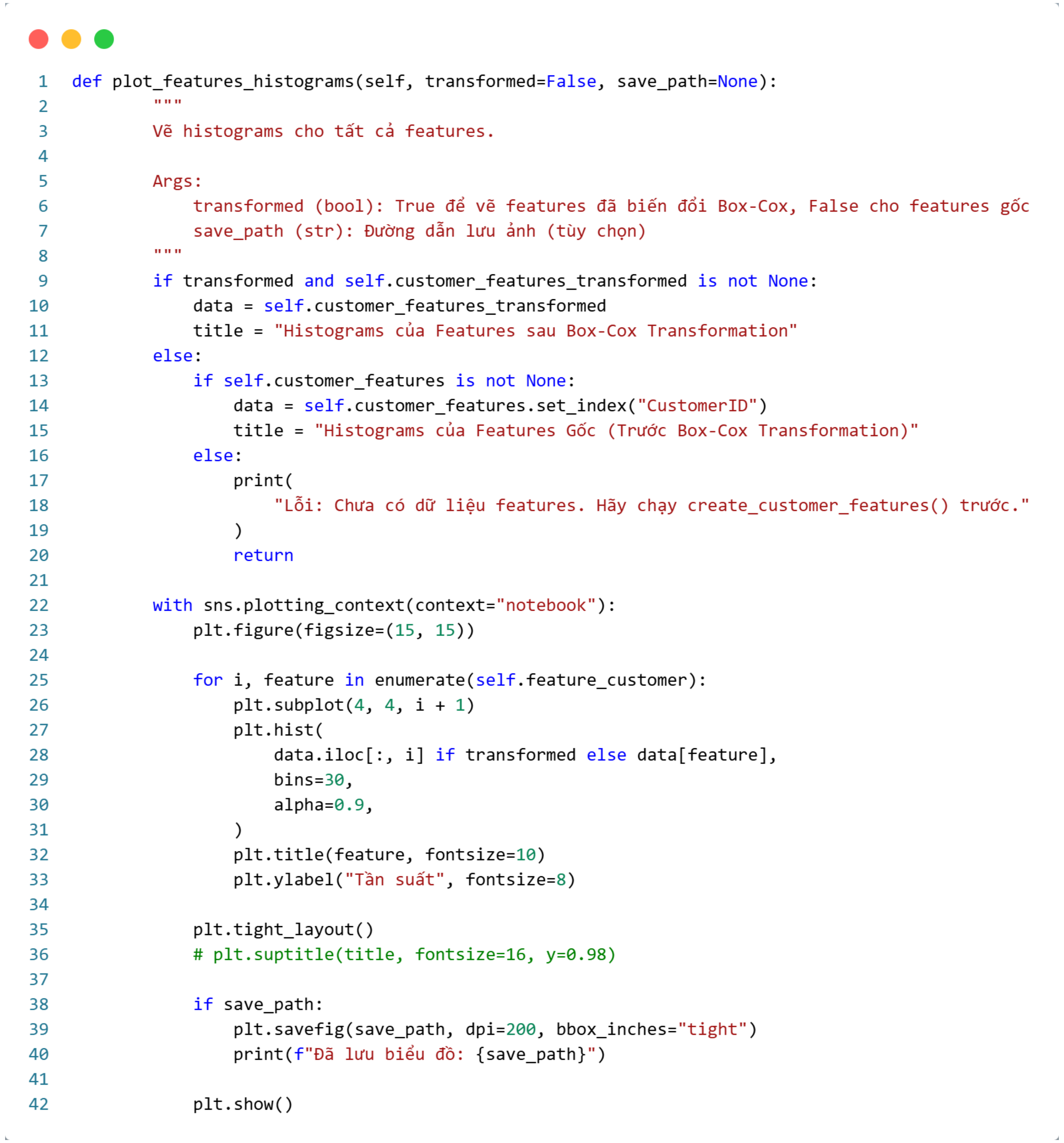
Class 2: FeatureEngineer

plot_features_boxplots(self, transformed, save_path):
Vẽ box plots cho tất cả features



```python
def plot_features_boxplots(self, transformed=False, save_path=None):
        """
        Vẽ box plots cho tất cả features.

        Args:
            transformed (bool): True để vẽ features đã biến đổi Box-Cox, False cho features gốc
            save_path (str): Đường dẫn lưu ảnh (tùy chọn)
        """
        if transformed and self.customer_features_transformed is not None:
            data = self.customer_features_transformed
        else:
            if self.customer_features is not None:
                data = self.customer_features.set_index("CustomerID")
                title = "Box Plots của Features Gốc (Trước Box-Cox Transformation)"
            else:
                print(
                    "Lỗi: Chưa có dữ liệu features. Hãy chạy create_customer_features() trước."
                )
                return

        with sns.plotting_context(context="notebook"):
            plt.figure(figsize=(15, 15))

            for i, feature in enumerate(self.feature_customer):
                plt.subplot(4, 4, i + 1)
                plt.boxplot(data.iloc[:, i] if transformed else data[feature])
                plt.title(feature, fontsize=10)
                plt.xticks([])

            plt.tight_layout()
            # plt.suptitle(title, fontsize=16, y=1.1)

            if save_path:
                plt.savefig(save_path, dpi=200, bbox_inches="tight")
                print(f"Đã lưu biểu đồ: {save_path}")

            plt.show()
```

plot_features_histograms(self, transformed, save_path):
Vẽ histograms để xem phân phối của features



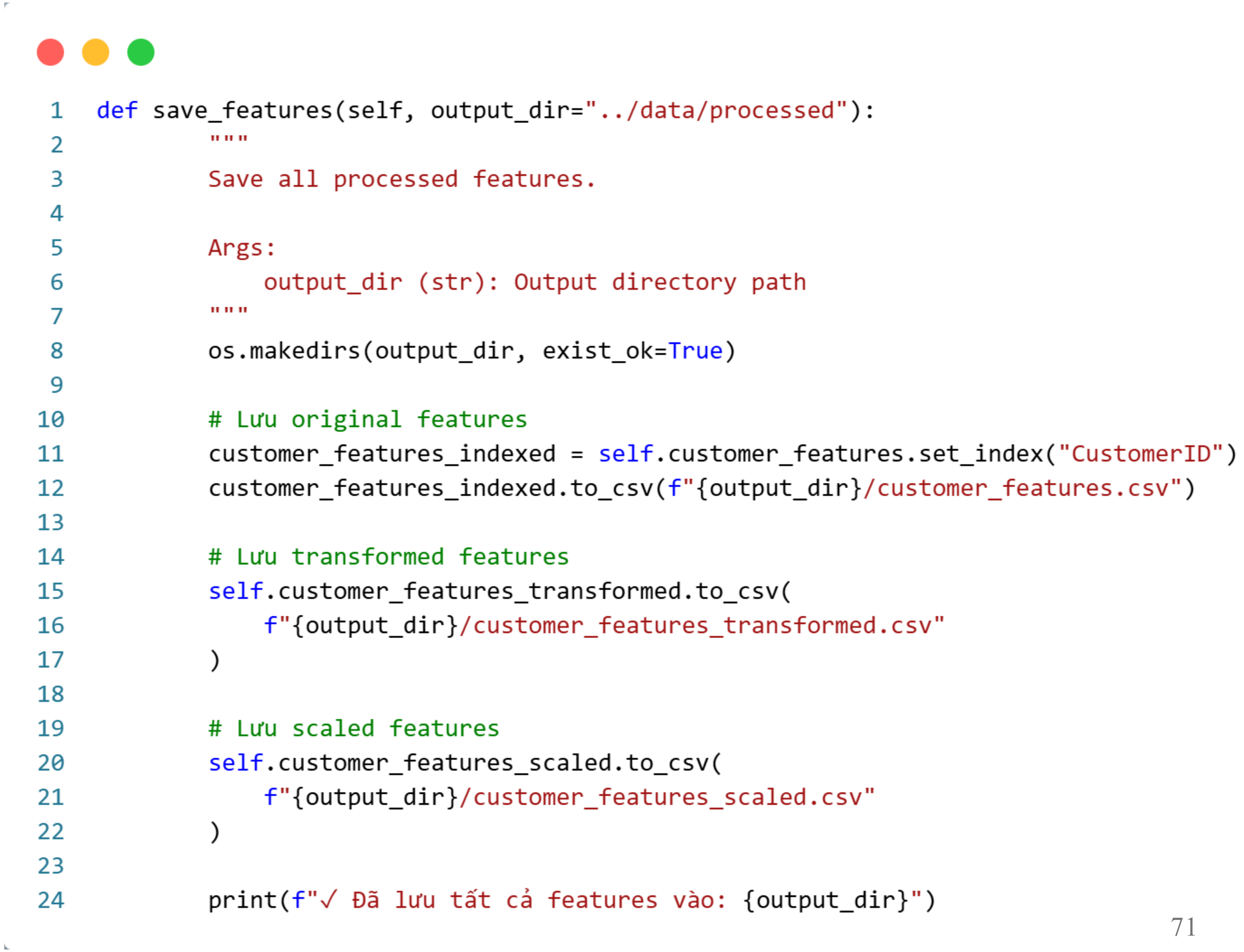
```python
def plot_features_histograms(self, transformed=False, save_path=None):
        """
        Vẽ histograms cho tất cả features.

        Args:
            transformed (bool): True để vẽ features đã biến đổi Box-Cox, False cho features gốc
            save_path (str): Đường dẫn lưu ảnh (tùy chọn)
        """
        if transformed and self.customer_features_transformed is not None:
            data = self.customer_features_transformed
            title = "Histograms của Features sau Box-Cox Transformation"
        else:
            if self.customer_features is not None:
                data = self.customer_features.set_index("CustomerID")
                title = "Histograms của Features Gốc (Trước Box-Cox Transformation)"
            else:
                print(
                    "Lỗi: Chưa có dữ liệu features. Hãy chạy create_customer_features() trước."
                )
                return

        with sns.plotting_context(context="notebook"):
            plt.figure(figsize=(15, 15))

            for i, feature in enumerate(self.feature_customer):
                plt.subplot(4, 4, i + 1)
                plt.hist(
                    data.iloc[:, i] if transformed else data[feature],
                    bins=30,
                    alpha=0.9,
                )
                plt.title(feature, fontsize=10)
                plt.ylabel("Tần suất", fontsize=8)

            plt.tight_layout()
            # plt.suptitle(title, fontsize=16, y=0.98)

            if save_path:
                plt.savefig(save_path, dpi=200, bbox_inches="tight")
                print(f"Đã lưu biểu đồ: {save_path}")

            plt.show()
```

Class 2: FeatureEngineer

save_features(self, output_dir): Lưu tất cả versions của features

Lưu 3 versions

- Lưu 3 versions để có thể sử dụng tùy nhu cầu
- Scaled version dùng cho clustering



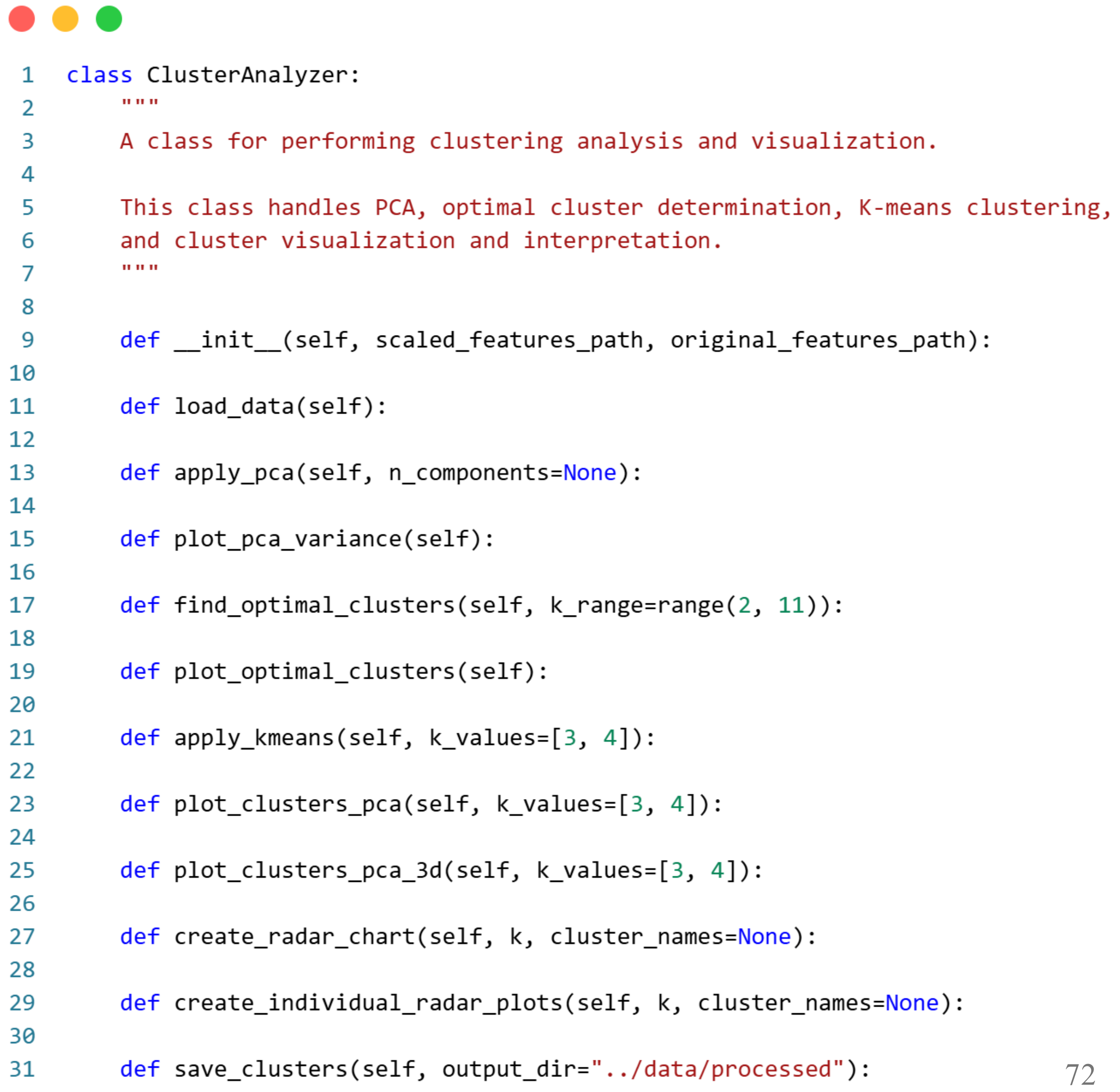
```
def save_features(self, output_dir="../data/processed"):
        """
        Save all processed features.

        Args:
            output_dir (str): Output directory path
        """
        os.makedirs(output_dir, exist_ok=True)

        # Lưu original features
        customer_features_indexed = self.customer_features.set_index("CustomerID")
        customer_features_indexed.to_csv(f"{output_dir}/customer_features.csv")

        # Lưu transformed features
        self.customer_features_transformed.to_csv(
            f"{output_dir}/customer_features_transformed.csv"
        )

        # Lưu scaled features
        self.customer_features_scaled.to_csv(
            f"{output_dir}/customer_features_scaled.csv"
        )

        print(f"✓ Đã lưu tất cả features vào: {output_dir}")
```

Class 3: ClusterAnalyzer

Mục đích: Thực hiện phân tích clustering và visualization

Chức năng chính:

- Áp dụng PCA để giảm chiều dữ liệu
- Tìm số cluster tối ưu (Elbow method, Silhouette score)
- Thực hiện K-means clustering
- Visualization clusters trong không gian 2D/3D
- Tạo radar charts để phân tích đặc điểm clusters

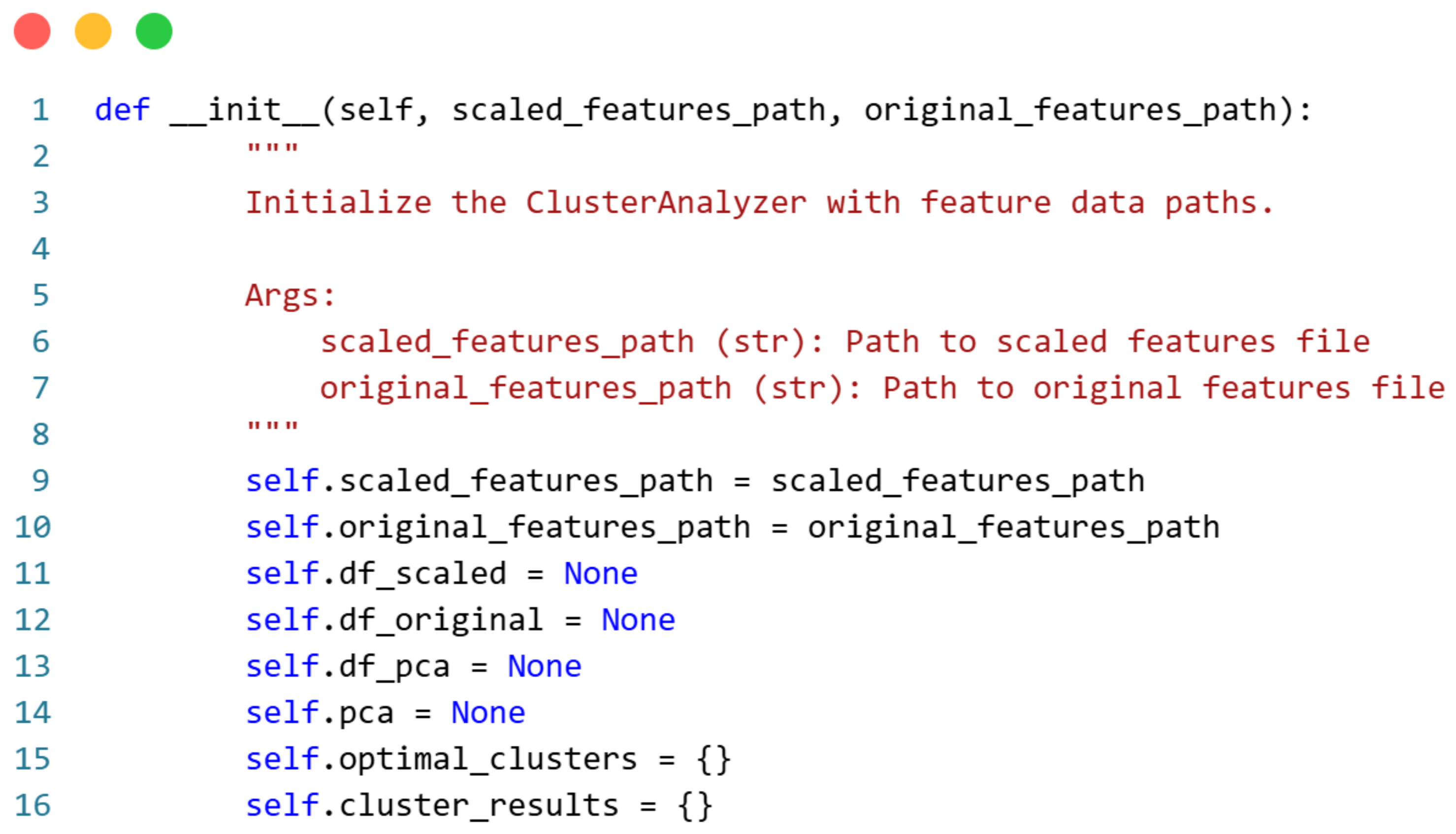


```python
class ClusterAnalyzer:
    """
    A class for performing clustering analysis and visualization.

    This class handles PCA, optimal cluster determination, K-means clustering,
    and cluster visualization and interpretation.
    """

    def __init__(self, scaled_features_path, original_features_path):

    def load_data(self):

    def apply_pca(self, n_components=None):

    def plot_pca_variance(self):

    def find_optimal_clusters(self, k_range=range(2, 11)):

    def plot_optimal_clusters(self):

    def apply_kmeans(self, k_values=[3, 4]):

    def plot_clusters_pca(self, k_values=[3, 4]):

    def plot_clusters_pca_3d(self, k_values=[3, 4]):

    def create_radar_chart(self, k, cluster_names=None):

    def create_individual_radar_plots(self, k, cluster_names=None):

    def save_clusters(self, output_dir="../data/processed"):
```

Class 3: ClusterAnalyzer

__init__(self, scaled_features_path, original_features_path): Khởi tạo với đường dẫn features đã scaled và original

- df_scaled: Features đã scaled (dùng cho clustering)
- df_original: Features gốc
- df_pca: Features sau PCA
- pca: PCA object
- optimal_clusters: Kết quả phân tích số cluster tối ưu
- cluster_results: Kết quả clustering cho mỗi k

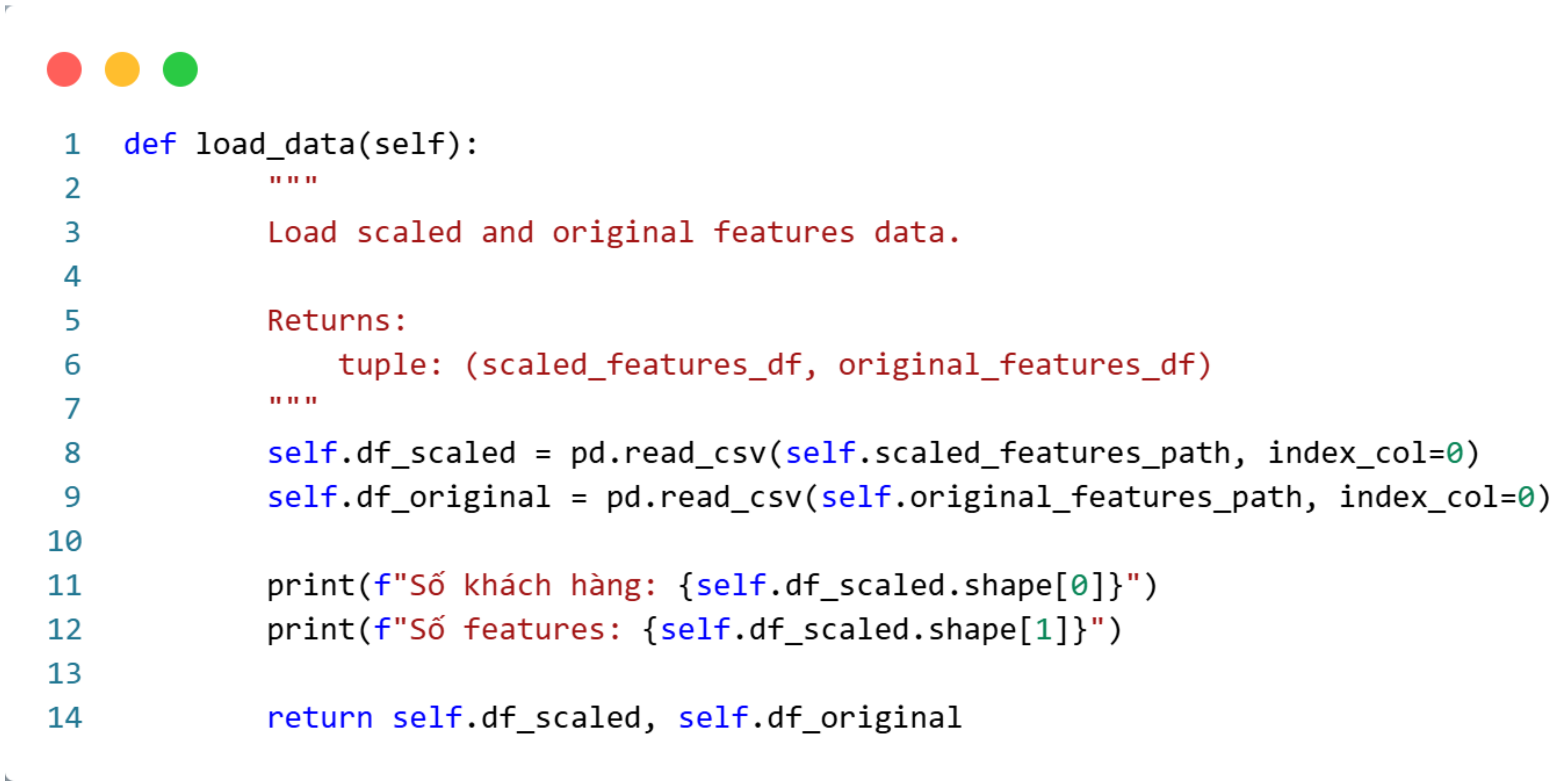


```python
def __init__(self, scaled_features_path, original_features_path):
        """
        Initialize the ClusterAnalyzer with feature data paths.

        Args:
            scaled_features_path (str): Path to scaled features file
            original_features_path (str): Path to original features file
        """
        self.scaled_features_path = scaled_features_path
        self.original_features_path = original_features_path
        self.df_scaled = None
        self.df_original = None
        self.df_pca = None
        self.pca = None
        self.optimal_clusters = {}
        self.cluster_results = {}
```

Class 3: ClusterAnalyzer

load_data(self): Load scaled và original features

- Scaled: dùng cho PCA và clustering
- Original: dùng cho interpret kết quả



```python
def load_data(self):
        """
        Load scaled and original features data.

        Returns:
            tuple: (scaled_features_df, original_features_df)
        """
        self.df_scaled = pd.read_csv(self.scaled_features_path, index_col=0)
        self.df_original = pd.read_csv(self.original_features_path, index_col=0)

        print(f"Số khách hàng: {self.df_scaled.shape[0]}")
        print(f"Số features: {self.df_scaled.shape[1]}")

        return self.df_scaled, self.df_original
```

Class 3: ClusterAnalyzer

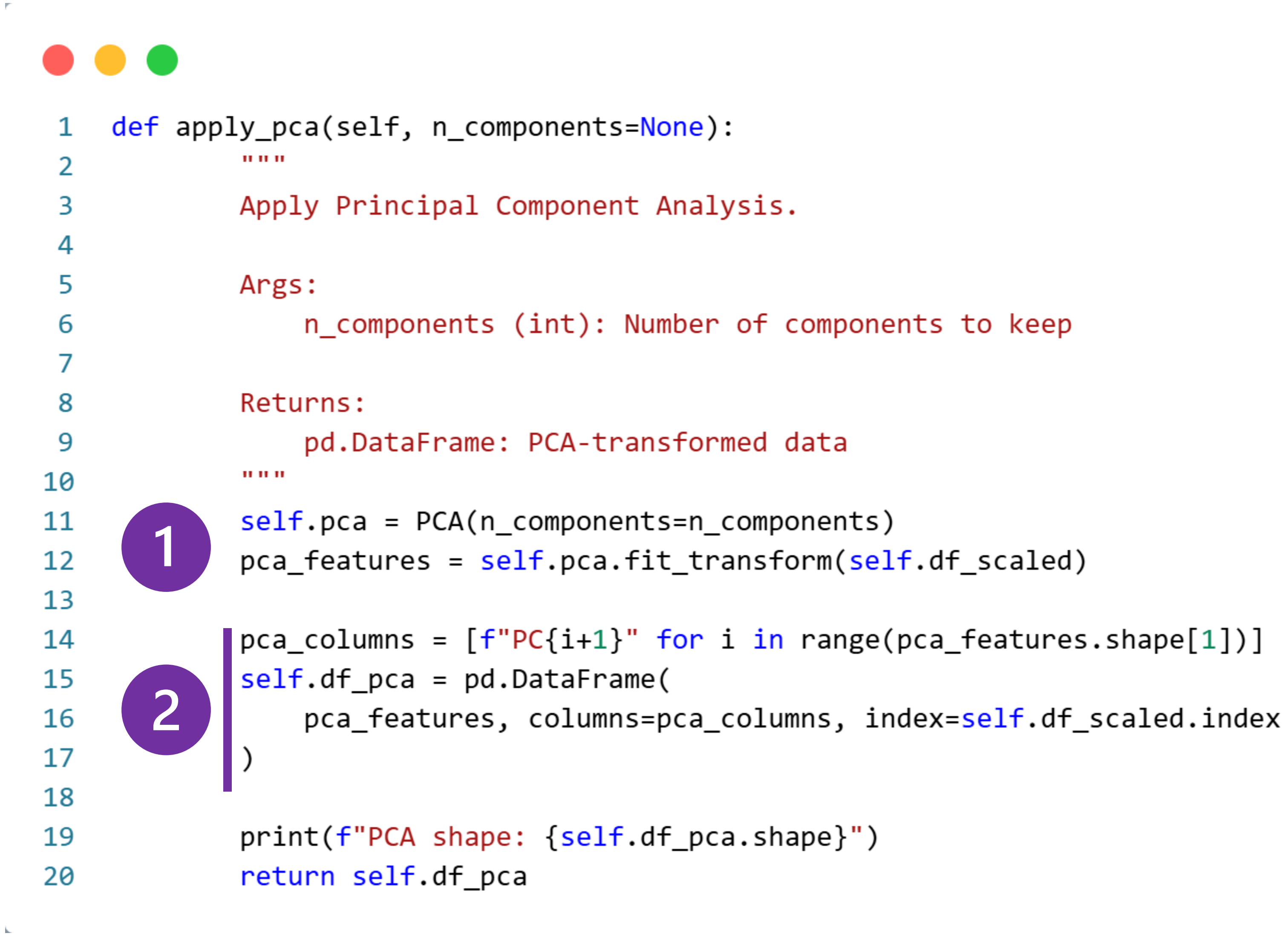
apply_pca(self, n_components): Giảm chiều dữ liệu bằng PCA

1. Fit PCA

- n_components: Giữ tất cả components
- fit_transform: Học transformation và áp dụng PCA

2. Tạo DataFrame

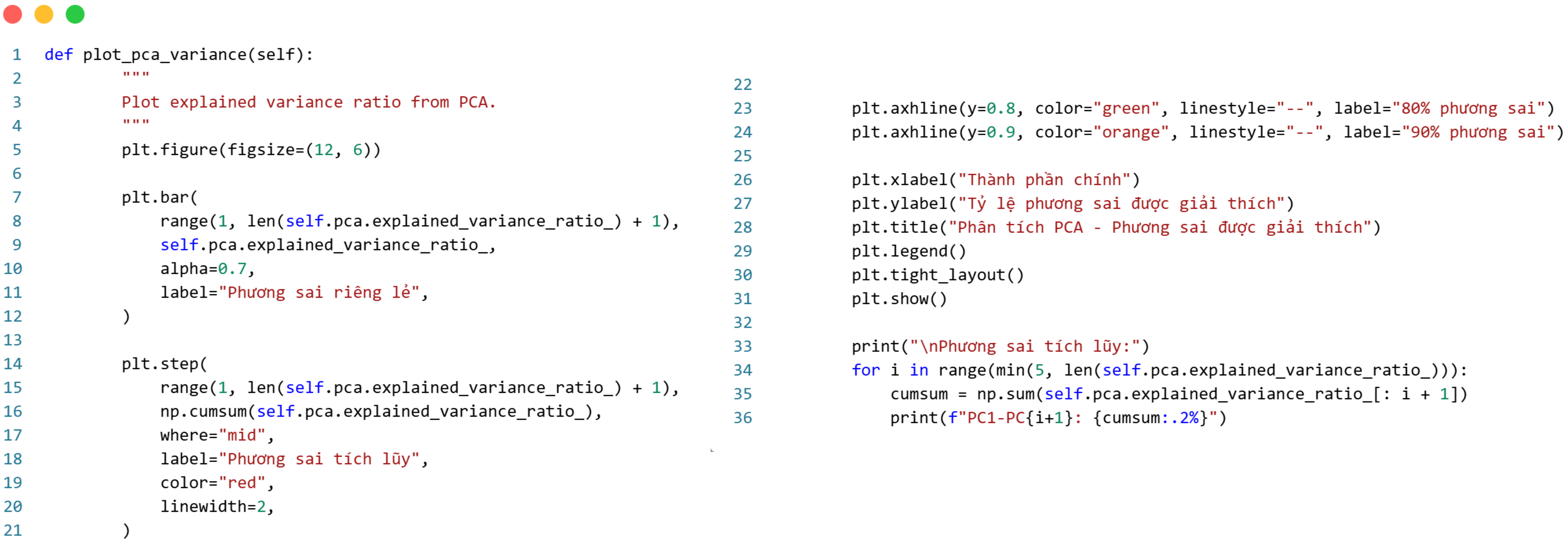
- Đặt tên columns: PC1, PC2, PC3, ...
- Giữ nguyên CustomerID index



```python
def apply_pca(self, n_components=None):
        """
        Apply Principal Component Analysis.

        Args:
            n_components (int): Number of components to keep

        Returns:
            pd.DataFrame: PCA-transformed data
        """
        self.pca = PCA(n_components=n_components)
        pca_features = self.pca.fit_transform(self.df_scaled)

        pca_columns = [f"PC{i+1}" for i in range(pca_features.shape[1])]
        self.df_pca = pd.DataFrame(
            pca_features, columns=pca_columns, index=self.df_scaled.index
        )

        print(f"PCA shape: {self.df_pca.shape}")
        return self.df_pca
```

Class 3: ClusterAnalyzer

plot_pca_variance(self): Visualize explained variance của các principal components. Vẽ Mỗi bar = % variance giải thích bởi 1 PC. Vẽ cumulative variance giúp xác định cần bao nhiêu PC để giữ X% variance



```
def plot_pca_variance(self):
        """
        Plot explained variance ratio from PCA.
        """
        plt.figure(figsize=(12, 6))

        plt.bar(
            range(1, len(self.pca.explained_variance_ratio_) + 1),
            self.pca.explained_variance_ratio_,
            alpha=0.7,
            label="Phương sai riêng lẻ",
        )

        plt.step(
            range(1, len(self.pca.explained_variance_ratio_) + 1),
            np.cumsum(self.pca.explained_variance_ratio_),
            where="mid",
            label="Phương sai tích lũy",
            color="red",
            linewidth=2,
        )

        plt.axhline(y=0.8, color="green", linestyle="--", label="80% phương sai")
        plt.axhline(y=0.9, color="orange", linestyle="--", label="90% phương sai")

        plt.xlabel("Thành phần chính")
        plt.ylabel("Tỷ lệ phương sai được giải thích")
        plt.title("Phân tích PCA - Phương sai được giải thích")
        plt.legend()
        plt.tight_layout()
        plt.show()

        print("\nPhương sai tích lũy:")
        for i in range(min(5, len(self.pca.explained_variance_ratio_))):
            cumsum = np.sum(self.pca.explained_variance_ratio_[: i + 1])
            print(f"PC1-PC{i+1}: {cumsum:.2%}")
```

Class 3: ClusterAnalyzer

find_optimal_clusters(self, k_range): Tìm số cluster tối ưu bằng Elbow và Silhouette methods

1. Loop qua các giá trị k (để chọn được k tốt nhất)

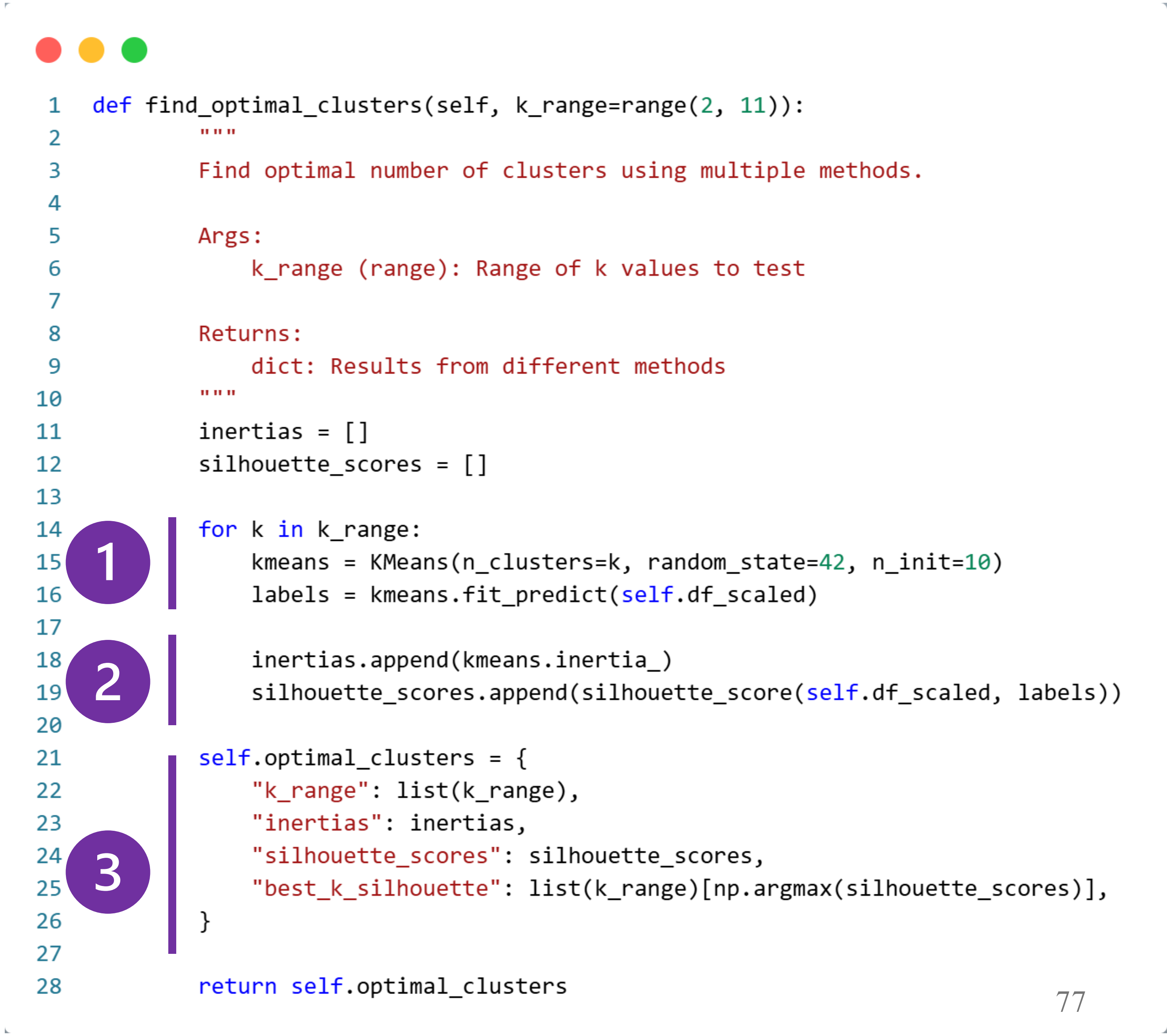
- Set seed = 4, để đảm bảo reproducibility
- Mỗi k chạy 10 lần với initial centers khác nhau, lấy kết quả tốt nhất

2. Tính metrics

- Inertia: Tổng khoảng cách bình phương từ điểm đến centroid (càng thấp càng tốt)
- Silhouette Score: Đo độ phân tách giữa các clusters (-1 đến 1, càng cao càng tốt)

3. Lưu kết quả

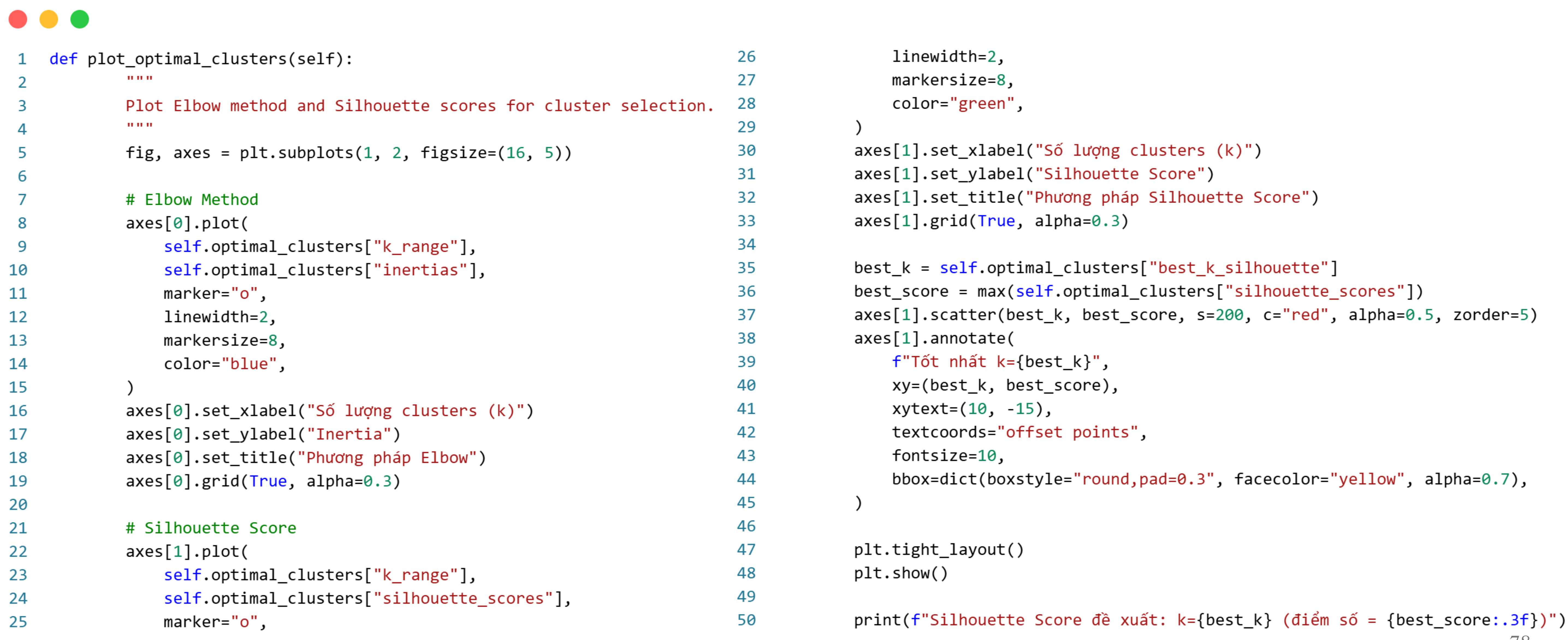
- Lưu tất cả metrics và k tốt nhất theo Silhouette



```python
def find_optimal_clusters(self, k_range=range(2, 11)):
        """
        Find optimal number of clusters using multiple methods.

        Args:
            k_range (range): Range of k values to test

        Returns:
            dict: Results from different methods
        """
        inertias = []
        silhouette_scores = []

        for k in k_range:
            kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
            labels = kmeans.fit_predict(self.df_scaled)

            inertias.append(kmeans.inertia_)
            silhouette_scores.append(silhouette_score(self.df_scaled, labels))

        self.optimal_clusters = {
            "k_range": list(k_range),
            "inertias": inertias,
            "silhouette_scores": silhouette_scores,
            "best_k_silhouette": list(k_range)[np.argmax(silhouette_scores)],
        }

        return self.optimal_clusters
```

Class 3: ClusterAnalyzer

plot_optimal_clusters(self): Vẽ biểu đồ Elbow và Silhouette. Vẽ đường inertia theo k. Tìm "elbow" (điểm uốn) → số cluster tối ưu. Highlight điểm có Silhouette score cao nhất



```python
def plot_optimal_clusters(self):
    """
    Plot Elbow method and Silhouette scores for cluster selection.
    """
    fig, axes = plt.subplots(1, 2, figsize=(16, 5))

    # Elbow Method
    axes[0].plot(
        self.optimal_clusters["k_range"],
        self.optimal_clusters["inertias"],
        marker="o",
        linewidth=2,
        markersize=8,
        color="blue",
    )
    axes[0].set_xlabel("Số lượng clusters (k)")
    axes[0].set_ylabel("Inertia")
    axes[0].set_title("Phương pháp Elbow")
    axes[0].grid(True, alpha=0.3)

    # Silhouette Score
    axes[1].plot(
        self.optimal_clusters["k_range"],
        self.optimal_clusters["silhouette_scores"],
        marker="o",
        linewidth=2,
        markersize=8,
        color="green",
    )
    axes[1].set_xlabel("Số lượng clusters (k)")
    axes[1].set_ylabel("Silhouette Score")
    axes[1].set_title("Phương pháp Silhouette Score")
    axes[1].grid(True, alpha=0.3)

    best_k = self.optimal_clusters["best_k_silhouette"]
    best_score = max(self.optimal_clusters["silhouette_scores"])
    axes[1].scatter(best_k, best_score, s=200, c="red", alpha=0.5, zorder=5)
    axes[1].annotate(
        f"Tốt nhất k={best_k}",
        xy=(best_k, best_score),
        xytext=(10, -15),
        textcoords="offset points",
        fontsize=10,
        bbox=dict(boxstyle="round,pad=0.3", facecolor="yellow", alpha=0.7),
    )

    plt.tight_layout()
    plt.show()

    print(f"Silhouette Score đề xuất: k={best_k} (điểm số = {best_score:.3f})")
```

Class 3: ClusterAnalyzer

apply_kmeans(self, k_values): Áp dụng K-means với các giá trị k khác nhau

0. Loop k values

1. Fit K-means

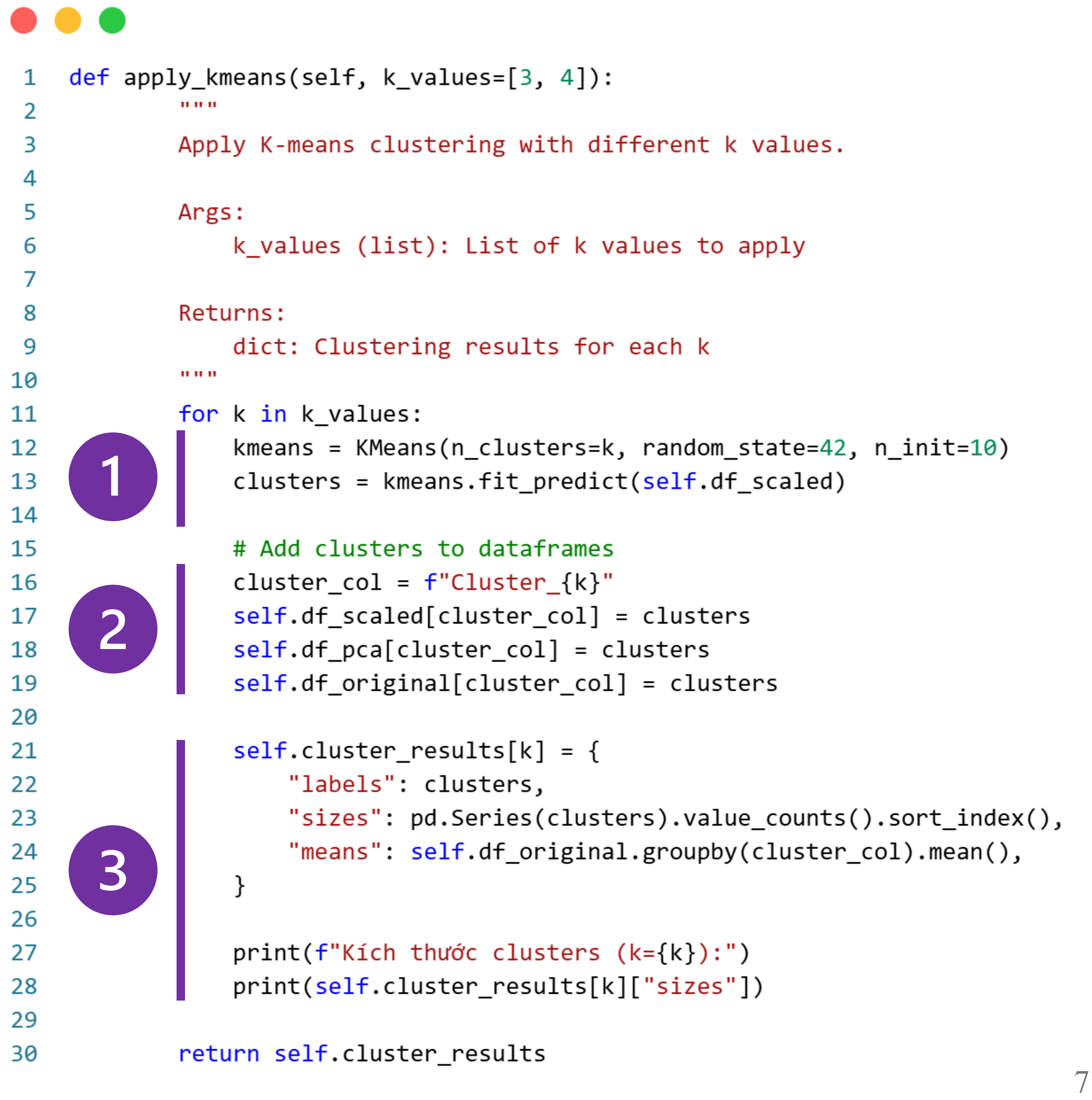
- Chạy K-means với k clusters
- clusters: là array chứa cluster label cho mỗi khách hàng

2. Tính metrics

- Thêm cột mới vào cả 3 dataframes
- Tên cột: "Cluster_3", "Cluster_4", ...

3. Lưu kết quả

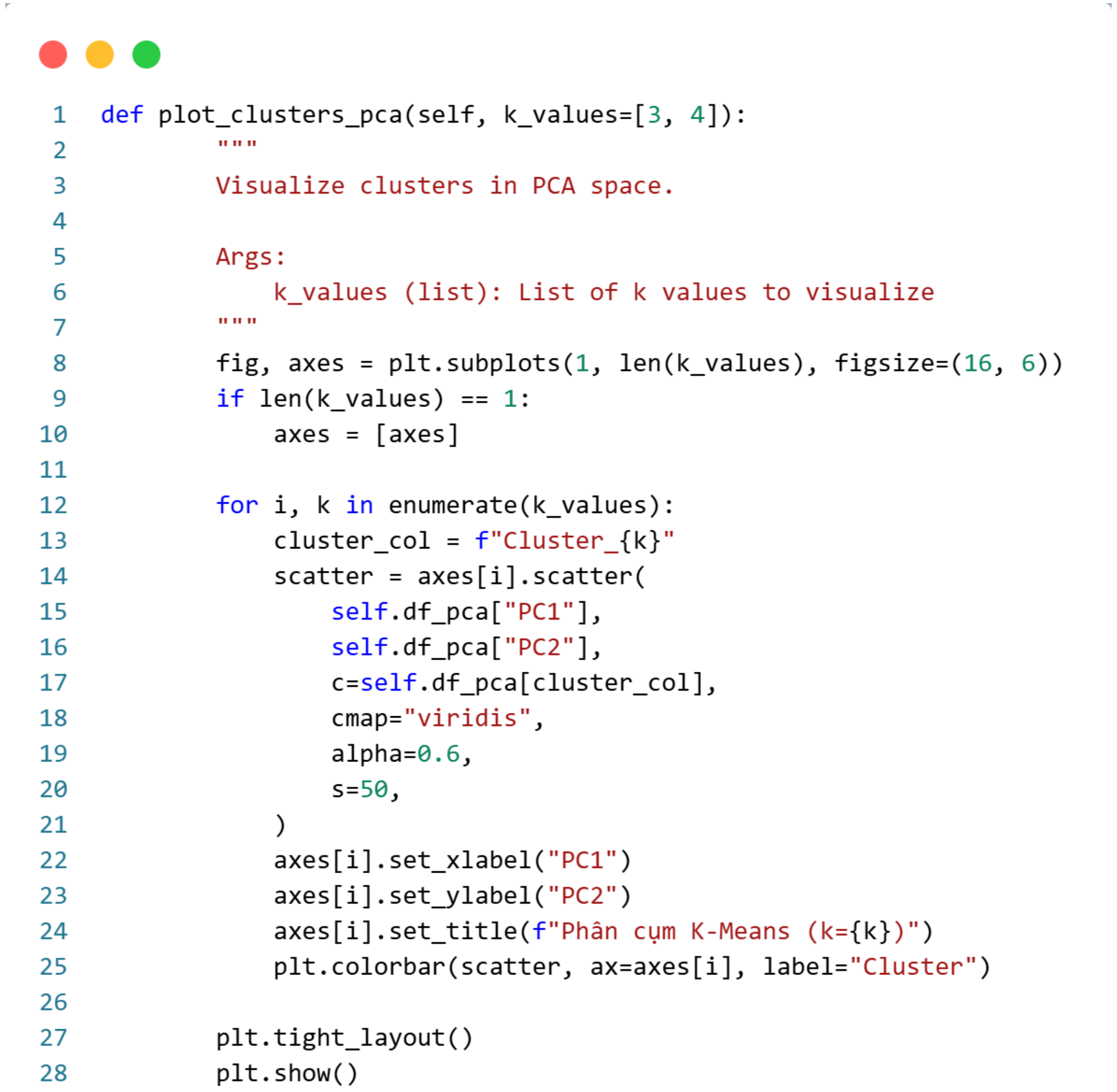
- labels: Cluster assignments
- sizes: Số khách hàng trong mỗi cluster
- means: Giá trị trung bình của các features theo cluster



```python
def apply_kmeans(self, k_values=[3, 4]):
        """
        Apply K-means clustering with different k values.

        Args:
            k_values (list): List of k values to apply

        Returns:
            dict: Clustering results for each k
        """
        for k in k_values:
            kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
            clusters = kmeans.fit_predict(self.df_scaled)

            # Add clusters to dataframes
            cluster_col = f"Cluster_{k}"
            self.df_scaled[cluster_col] = clusters
            self.df_pca[cluster_col] = clusters
            self.df_original[cluster_col] = clusters

            self.cluster_results[k] = {
                "labels": clusters,
                "sizes": pd.Series(clusters).value_counts().sort_index(),
                "means": self.df_original.groupby(cluster_col).mean(),
            }

            print(f"Kích thước clusters (k={k}):")
            print(self.cluster_results[k]["sizes"])

        return self.cluster_results
```

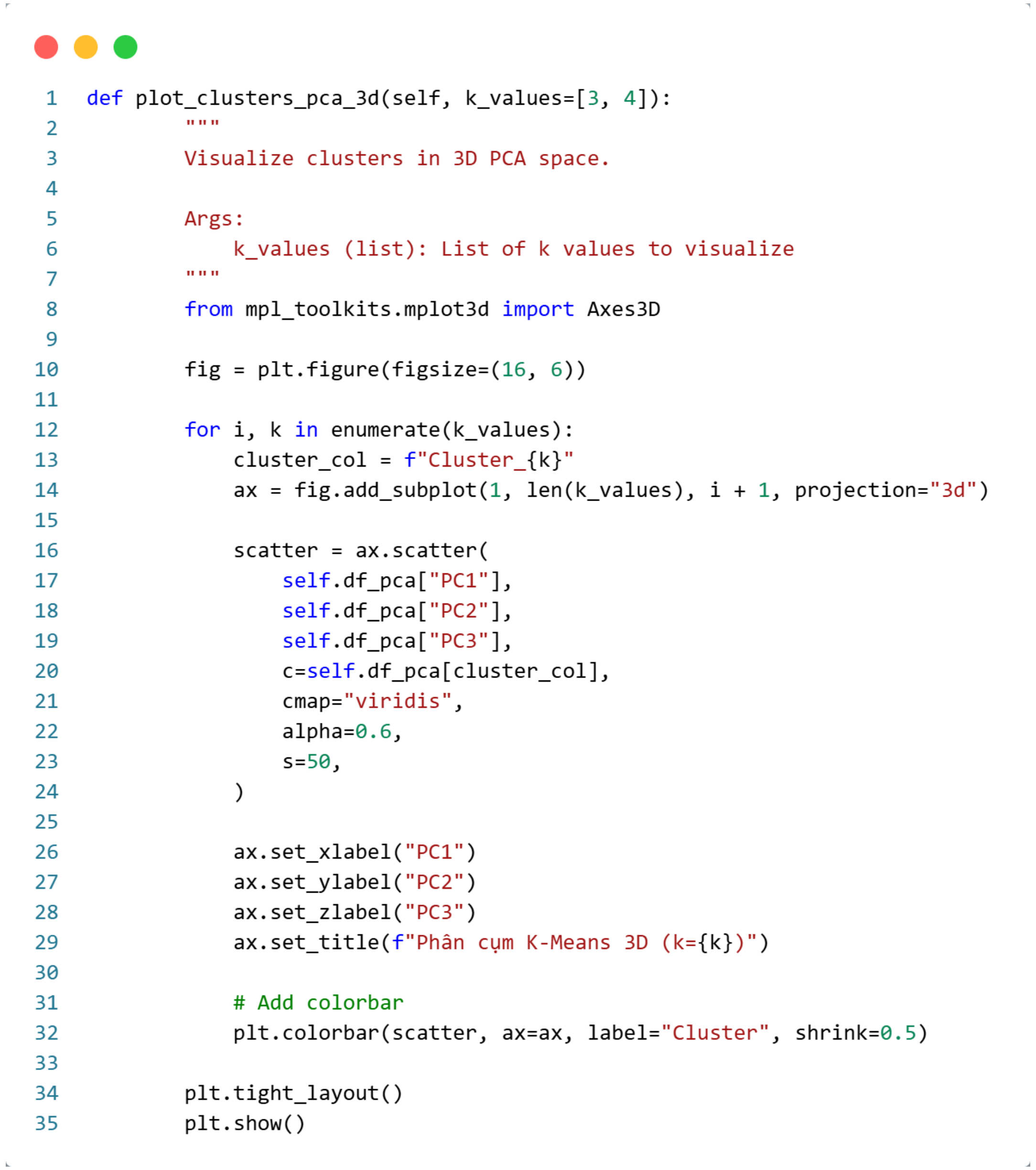
Class 3: ClusterAnalyzer

plot_clusters_pca(self, k_values): Visualize clusters trong không gian 2D PCA. Sử dụng 2 principal components đầu tiên. X-axis: PC1 (principal component 1). Y-axis: PC2 (principal component 2)



```python
def plot_clusters_pca(self, k_values=[3, 4]):
        """
        Visualize clusters in PCA space.

        Args:
            k_values (list): List of k values to visualize
        """
        fig, axes = plt.subplots(1, len(k_values), figsize=(16, 6))
        if len(k_values) == 1:
            axes = [axes]

        for i, k in enumerate(k_values):
            cluster_col = f"Cluster_{k}"
            scatter = axes[i].scatter(
                self.df_pca["PC1"],
                self.df_pca["PC2"],
                c=self.df_pca[cluster_col],
                cmap="viridis",
                alpha=0.6,
                s=50,
            )
            axes[i].set_xlabel("PC1")
            axes[i].set_ylabel("PC2")
            axes[i].set_title(f"Phân cụm K-Means (k={k})")
            plt.colorbar(scatter, ax=axes[i], label="Cluster")

        plt.tight_layout()
        plt.show()
```

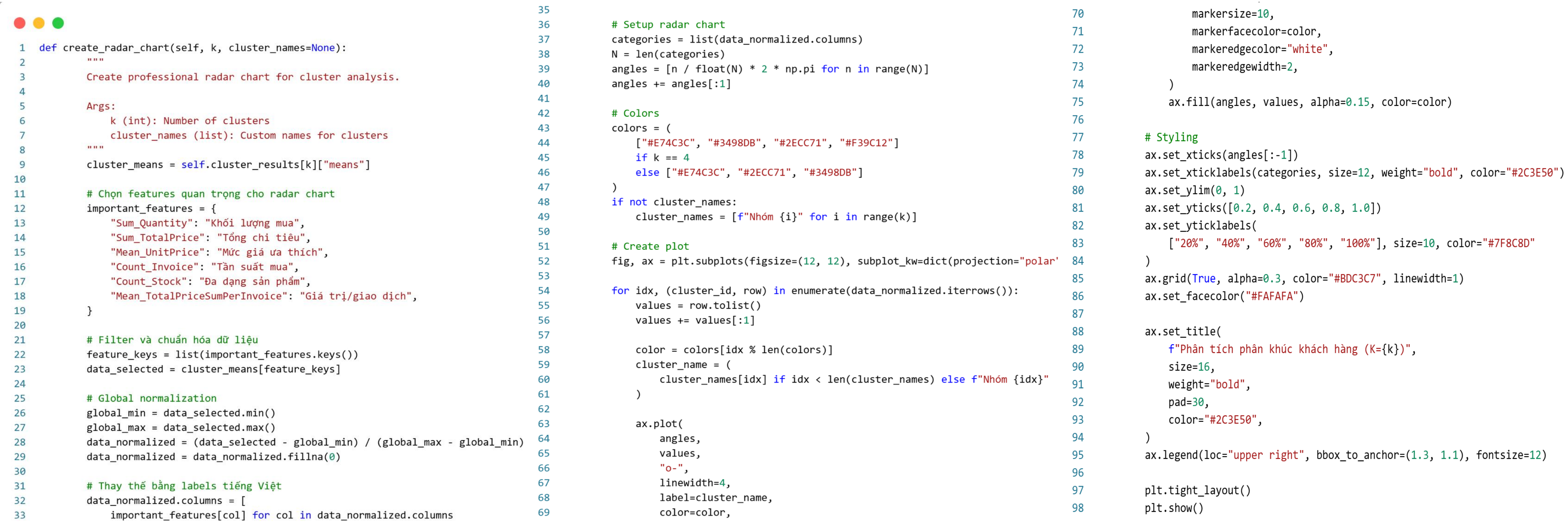
plot_clusters_pca_3d(self, k_values): Visualize clusters trong không gian 3D. Sử dụng 3 principal components đầu tiên. X-axis: PC1 (principal component 1). Y-axis: PC2 (principal component 2). Z-axis: PC3 (principal component 3)



```python
def plot_clusters_pca_3d(self, k_values=[3, 4]):
        """
        Visualize clusters in 3D PCA space.

        Args:
            k_values (list): List of k values to visualize
        """
        from mpl_toolkits.mplot3d import Axes3D

        fig = plt.figure(figsize=(16, 6))

        for i, k in enumerate(k_values):
            cluster_col = f"Cluster_{k}"
            ax = fig.add_subplot(1, len(k_values), i + 1, projection="3d")

            scatter = ax.scatter(
                self.df_pca["PC1"],
                self.df_pca["PC2"],
                self.df_pca["PC3"],
                c=self.df_pca[cluster_col],
                cmap="viridis",
                alpha=0.6,
                s=50,
            )

            ax.set_xlabel("PC1")
            ax.set_ylabel("PC2")
            ax.set_zlabel("PC3")
            ax.set_title(f"Phân cụm K-Means 3D (k={k})")

            # Add colorbar
            plt.colorbar(scatter, ax=ax, label="Cluster", shrink=0.5)

        plt.tight_layout()
        plt.show()
```

Class 3: ClusterAnalyzer

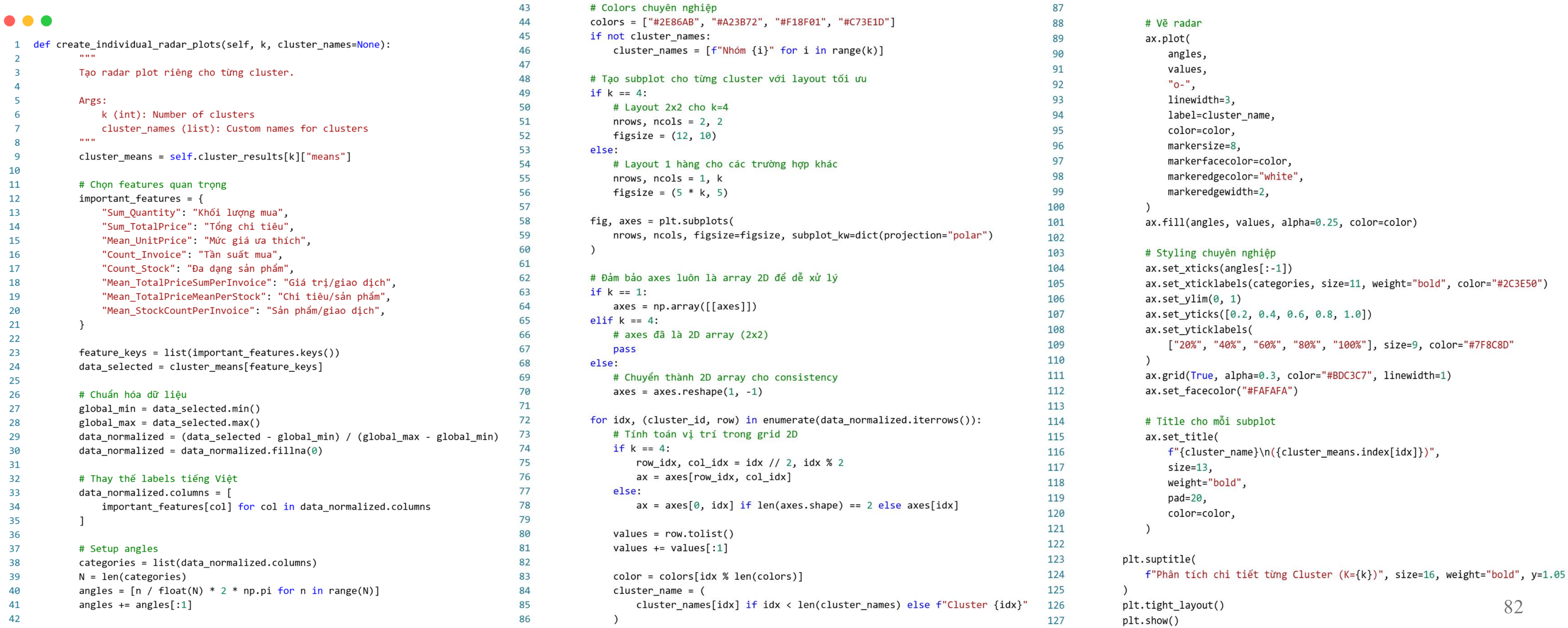
create_radar_chart(self, k, cluster_names): Tạo radar chart để so sánh profiles của các clusters. Chọn 6 features quan trọng nhất để visualize. Đưa tất cả features về scale 0-1 để so sánh trên cùng 1 radar chart. Tính góc cho mỗi feature trên vòng tròn (360 độ). Mỗi cluster là 1 đường trên radar



```python
def create_radar_chart(self, k, cluster_names=None):
        """
        Create professional radar chart for cluster analysis.

        Args:
            k (int): Number of clusters
            cluster_names (list): Custom names for clusters
        """
        cluster_means = self.cluster_results[k]["means"]

        # Chọn features quan trọng cho radar chart
        important_features = {
            "Sum_Quantity": "Khối lượng mua",
            "Sum_TotalPrice": "Tổng chi tiêu",
            "Mean_UnitPrice": "Mức giá ưa thích",
            "Count_Invoice": "Tần suất mua",
            "Count_Stock": "Đa dạng sản phẩm",
            "Mean_TotalPriceSumPerInvoice": "Giá trị/giao dịch",
        }

        # Filter và chuẩn hóa dữ liệu
        feature_keys = list(important_features.keys())
        data_selected = cluster_means[feature_keys]

        # Global normalization
        global_min = data_selected.min()
        global_max = data_selected.max()
        data_normalized = (data_selected - global_min) / (global_max - global_min)
        data_normalized = data_normalized.fillna(0)

        # Thay thế bằng labels tiếng Việt
        data_normalized.columns = [
            important_features[col] for col in data_normalized.columns

        # Setup radar chart
        categories = list(data_normalized.columns)
        N = len(categories)
        angles = [n / float(N) * 2 * np.pi for n in range(N)]
        angles += angles[:1]

        # Colors
        colors = (
            ["#E74C3C", "#3498DB", "#2ECC71", "#F39C12"]
            if k == 4
            else ["#E74C3C", "#2ECC71", "#3498DB"]
        )
        if not cluster_names:
            cluster_names = [f"Nhóm {i}" for i in range(k)]

        # Create plot
        fig, ax = plt.subplots(figsize=(12, 12), subplot_kw=dict(projection="polar'

        for idx, (cluster_id, row) in enumerate(data_normalized.iterrows()):
            values = row.tolist()
            values += values[:1]

            color = colors[idx % len(colors)]
            cluster_name = (
                cluster_names[idx] if idx < len(cluster_names) else f"Nhóm {idx}"
            )

            ax.plot(
                angles,
                values,
                "o-",
                linewidth=4,
                label=cluster_name,
                color=color,
                markersize=10,
                markerfacecolor=color,
                markeredgecolor="white",
                markeredgewidth=2,
            )
            ax.fill(angles, values, alpha=0.15, color=color)

        # Styling
        ax.set_xticks(angles[:-1])
        ax.set_xticklabels(categories, size=12, weight="bold", color="#2C3E50")
        ax.set_ylim(0, 1)
        ax.set_yticks([0.2, 0.4, 0.6, 0.8, 1.0])
        ax.set_yticklabels(
            ["20%", "40%", "60%", "80%", "100%"], size=10, color="#7F8C8D"
        )
        ax.grid(True, alpha=0.3, color="#BDC3C7", linewidth=1)
        ax.set_facecolor("#FAFAFA")

        ax.set_title(
            f"Phân tích phân khúc khách hàng (K={k})",
            size=16,
            weight="bold",
            pad=30,
            color="#2C3E50",
        )
        ax.legend(loc="upper right", bbox_to_anchor=(1.3, 1.1), fontsize=12)

        plt.tight_layout()
        plt.show()
```

Class 3: ClusterAnalyzer

create_individual_radar_plots(self, k, cluster_names): Tạo radar plot riêng cho TỪNG cluster. Chọn 6 features quan trọng nhất để visualize. Đưa tất cả features về scale 0-1 để so sánh trên cùng 1 radar chart. Tính góc cho mỗi feature trên vòng tròn (360 độ). Mỗi cluster là 1 đường trên radar



```python
def create_individual_radar_plots(self, k, cluster_names=None):
        """
        Tạo radar plot riêng cho từng cluster.

        Args:
            k (int): Number of clusters
            cluster_names (list): Custom names for clusters
        """
        cluster_means = self.cluster_results[k]["means"]

        # Chọn features quan trọng
        important_features = {
            "Sum_Quantity": "Khối lượng mua",
            "Sum_TotalPrice": "Tổng chi tiêu",
            "Mean_UnitPrice": "Mức giá ưa thích",
            "Count_Invoice": "Tần suất mua",
            "Count_Stock": "Đa dạng sản phẩm",
            "Mean_TotalPriceSumPerInvoice": "Giá trị/giao dịch",
            "Mean_TotalPriceMeanPerStock": "Chi tiêu/sản phẩm",
            "Mean_StockCountPerInvoice": "Sản phẩm/giao dịch",
        }

        feature_keys = list(important_features.keys())
        data_selected = cluster_means[feature_keys]

        # Chuẩn hóa dữ liệu
        global_min = data_selected.min()
        global_max = data_selected.max()
        data_normalized = (data_selected - global_min) / (global_max - global_min)
        data_normalized = data_normalized.fillna(0)

        # Thay thế labels tiếng Việt
        data_normalized.columns = [
            important_features[col] for col in data_normalized.columns
        ]

        # Setup angles
        categories = list(data_normalized.columns)
        N = len(categories)
        angles = [n / float(N) * 2 * np.pi for n in range(N)]
        angles += angles[:1]

        # Colors chuyên nghiệp
        colors = ["#2E86AB", "#A23B72", "#F18F01", "#C73E1D"]
        if not cluster_names:
            cluster_names = [f"Nhóm {i}" for i in range(k)]

        # Tạo subplot cho từng cluster với layout tối ưu
        if k == 4:
            # Layout 2x2 cho k=4
            nrows, ncols = 2, 2
            figsize = (12, 10)
        else:
            # Layout 1 hàng cho các trường hợp khác
            nrows, ncols = 1, k
            figsize = (5 * k, 5)

        fig, axes = plt.subplots(
            nrows, ncols, figsize=figsize, subplot_kw=dict(projection="polar")
        )

        # Đảm bảo axes luôn là array 2D để dễ xử lý
        if k == 1:
            axes = np.array([[axes]])
        elif k == 4:
            # axes đã là 2D array (2x2)
            pass
        else:
            # Chuyển thành 2D array cho consistency
            axes = axes.reshape(1, -1)

        for idx, (cluster_id, row) in enumerate(data_normalized.iterrows()):
            # Tính toán vị trí trong grid 2D
            if k == 4:
                row_idx, col_idx = idx // 2, idx % 2
                ax = axes[row_idx, col_idx]
            else:
                ax = axes[0, idx] if len(axes.shape) == 2 else axes[idx]

            values = row.tolist()
            values += values[:1]

            color = colors[idx % len(colors)]
            cluster_name = (
                cluster_names[idx] if idx < len(cluster_names) else f"Cluster {idx}"
            )

            # Vẽ radar
            ax.plot(
                angles,
                values,
                "o-",
                linewidth=3,
                label=cluster_name,
                color=color,
                markersize=8,
                markerfacecolor=color,
                markeredgecolor="white",
                markeredgewidth=2,
            )
            ax.fill(angles, values, alpha=0.25, color=color)

            # Styling chuyên nghiệp
            ax.set_xticks(angles[:-1])
            ax.set_xticklabels(categories, size=11, weight="bold", color="#2C3E50")
            ax.set_ylim(0, 1)
            ax.set_yticks([0.2, 0.4, 0.6, 0.8, 1.0])
            ax.set_yticklabels(
                ["20%", "40%", "60%", "80%", "100%"], size=9, color="#7F8C8D"
            )
            ax.grid(True, alpha=0.3, color="#BDC3C7", linewidth=1)
            ax.set_facecolor("#FAFAFA")

            # Title cho mỗi subplot
            ax.set_title(
                f"{cluster_name}\n({cluster_means.index[idx]})",
                size=13,
                weight="bold",
                pad=20,
                color=color,
            )

        plt.suptitle(
            f"Phân tích chi tiết từng Cluster (K={k})", size=16, weight="bold", y=1.05
        )
        plt.tight_layout()
        plt.show()
```

Class 3: ClusterAnalyzer

save_clusters(self, output_dir): Lưu kết quả phân cụm ra file CSV

0. Loop k values

1. Chuẩn bị dữ liệu

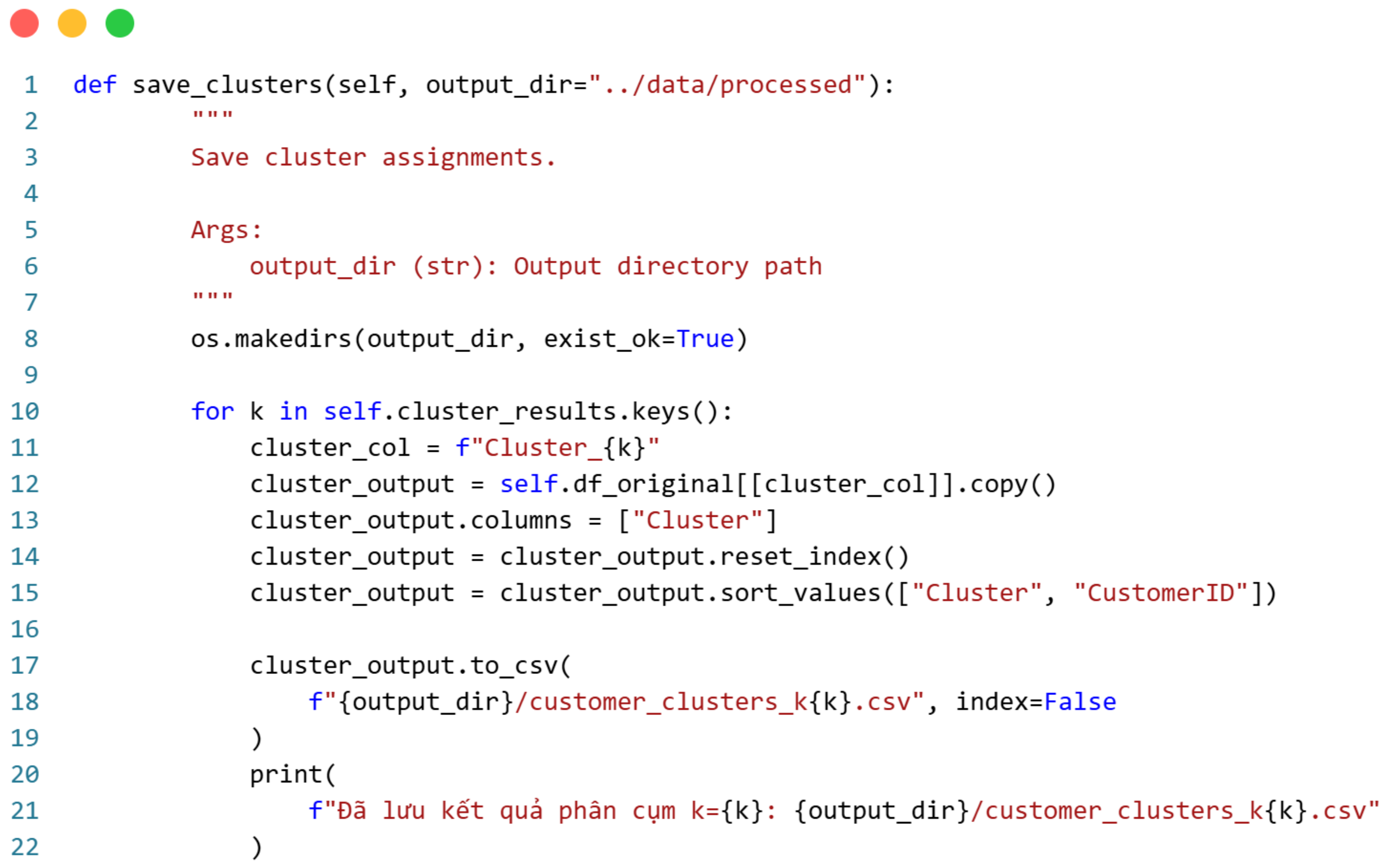
- Lấy cột cluster từ df_original
- Đổi tên thành "Cluster"

2. Format và sort

- Reset index để
- Sort theo Cluster rồi đến CustomerID

3. Lưu kết quả

- File riêng cho mỗi k



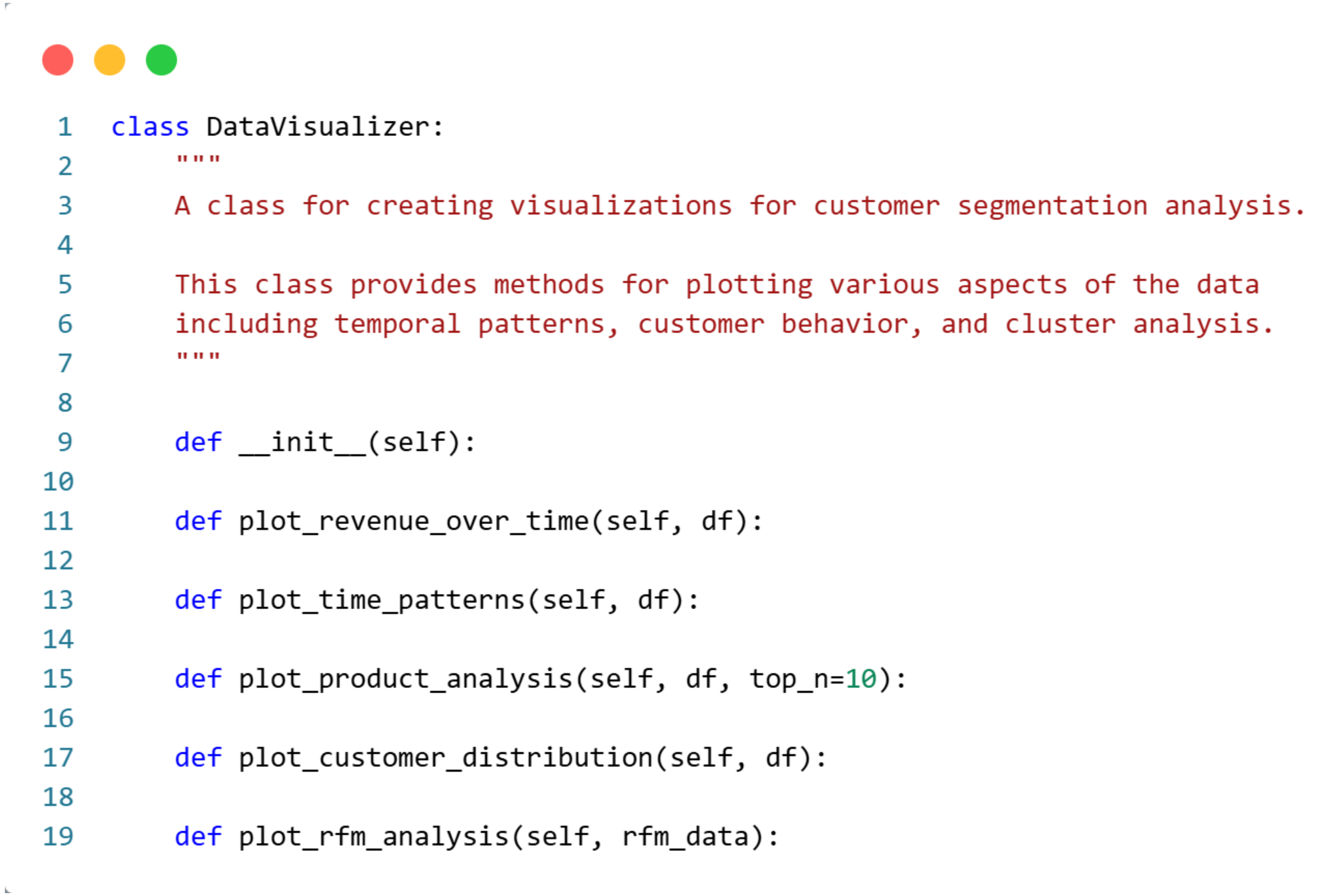
```python
def save_clusters(self, output_dir="../data/processed"):
        """
        Save cluster assignments.

        Args:
            output_dir (str): Output directory path
        """
        os.makedirs(output_dir, exist_ok=True)

        for k in self.cluster_results.keys():
            cluster_col = f"Cluster_{k}"
            cluster_output = self.df_original[[cluster_col]].copy()
            cluster_output.columns = ["Cluster"]
            cluster_output = cluster_output.reset_index()
            cluster_output = cluster_output.sort_values(["Cluster", "CustomerID"])

            cluster_output.to_csv(
                f"{output_dir}/customer_clusters_k{k}.csv", index=False
            )
            print(
                f"Đã lưu kết quả phân cụm k={k}: {output_dir}/customer_clusters_k{k}.csv"
            )
```

Class 4: DataVisualizer

Mục đích: Tạo các visualization để phân tích dữ liệu

Chức năng chính:

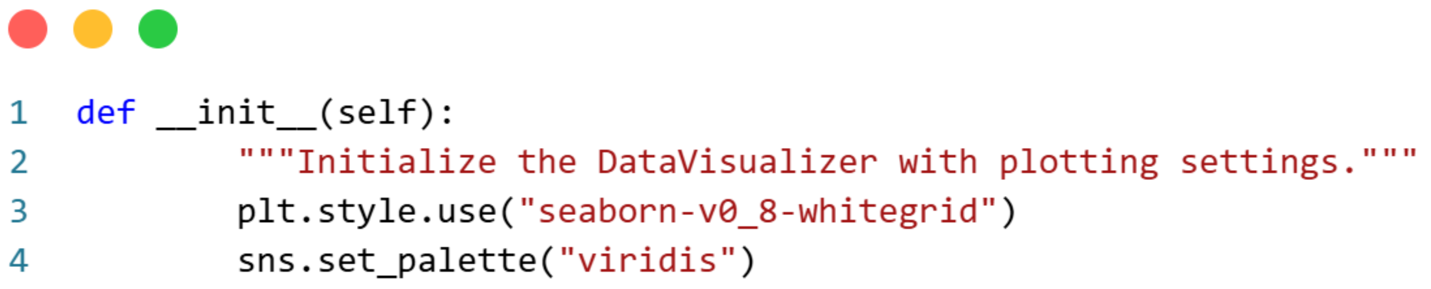
- Vẽ biểu đồ doanh thu theo thời gian
- Vẽ phân tích patterns theo ngày/giờ
- Vẽ phân tích sản phẩm theo số lượng và revenue
- Vẽ phân phối hành vi khách hàng
- RFM analysis visualization



```python
class DataVisualizer:
    """
    A class for creating visualizations for customer segmentation analysis.

    This class provides methods for plotting various aspects of the data
    including temporal patterns, customer behavior, and cluster analysis.
    """

    def __init__(self):

    def plot_revenue_over_time(self, df):

    def plot_time_patterns(self, df):

    def plot_product_analysis(self, df, top_n=10):

    def plot_customer_distribution(self, df):

    def plot_rfm_analysis(self, rfm_data):
```

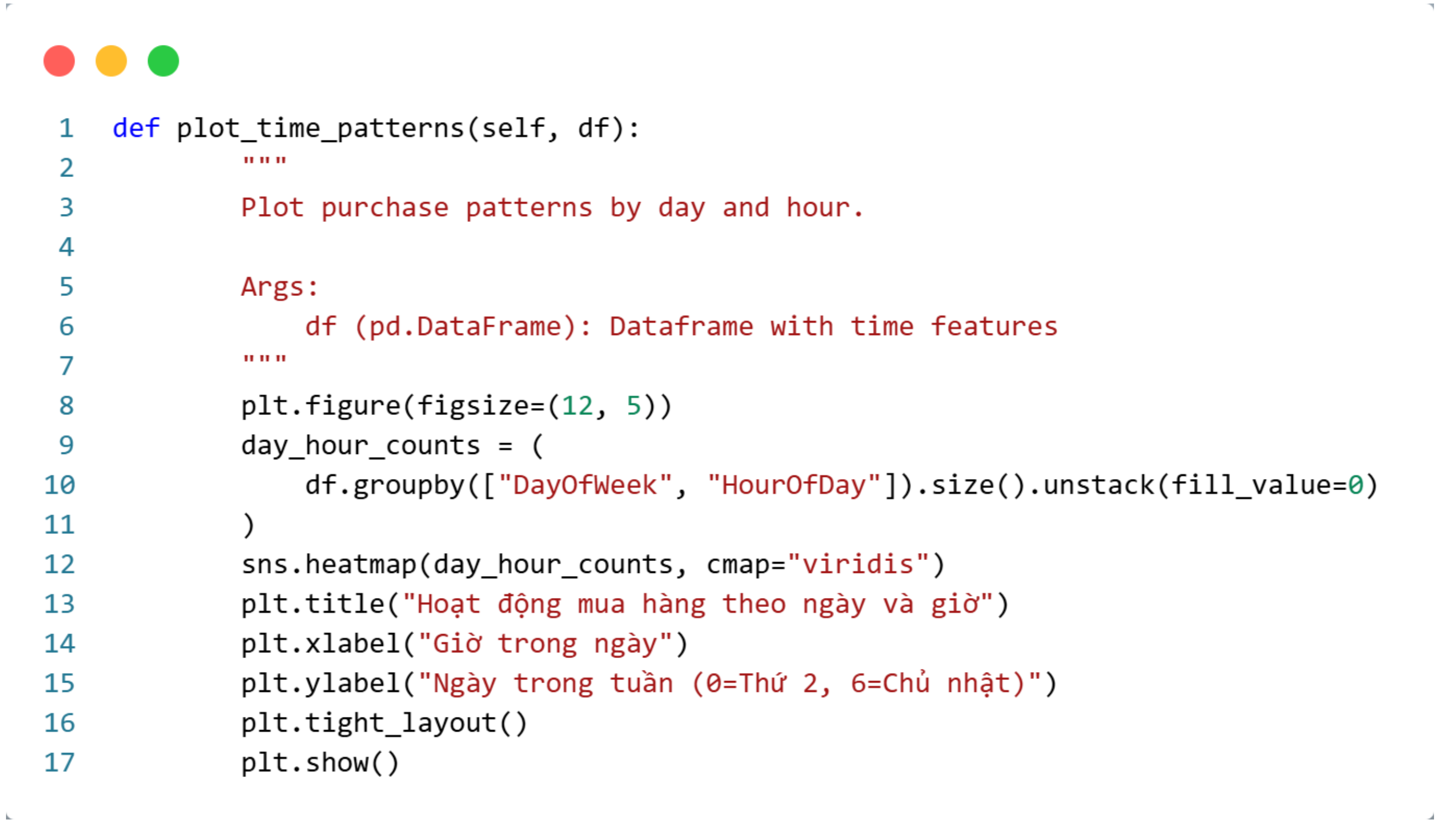
Class 4: DataVisualizer

__init__(self): Khởi tạo settings cho visualization



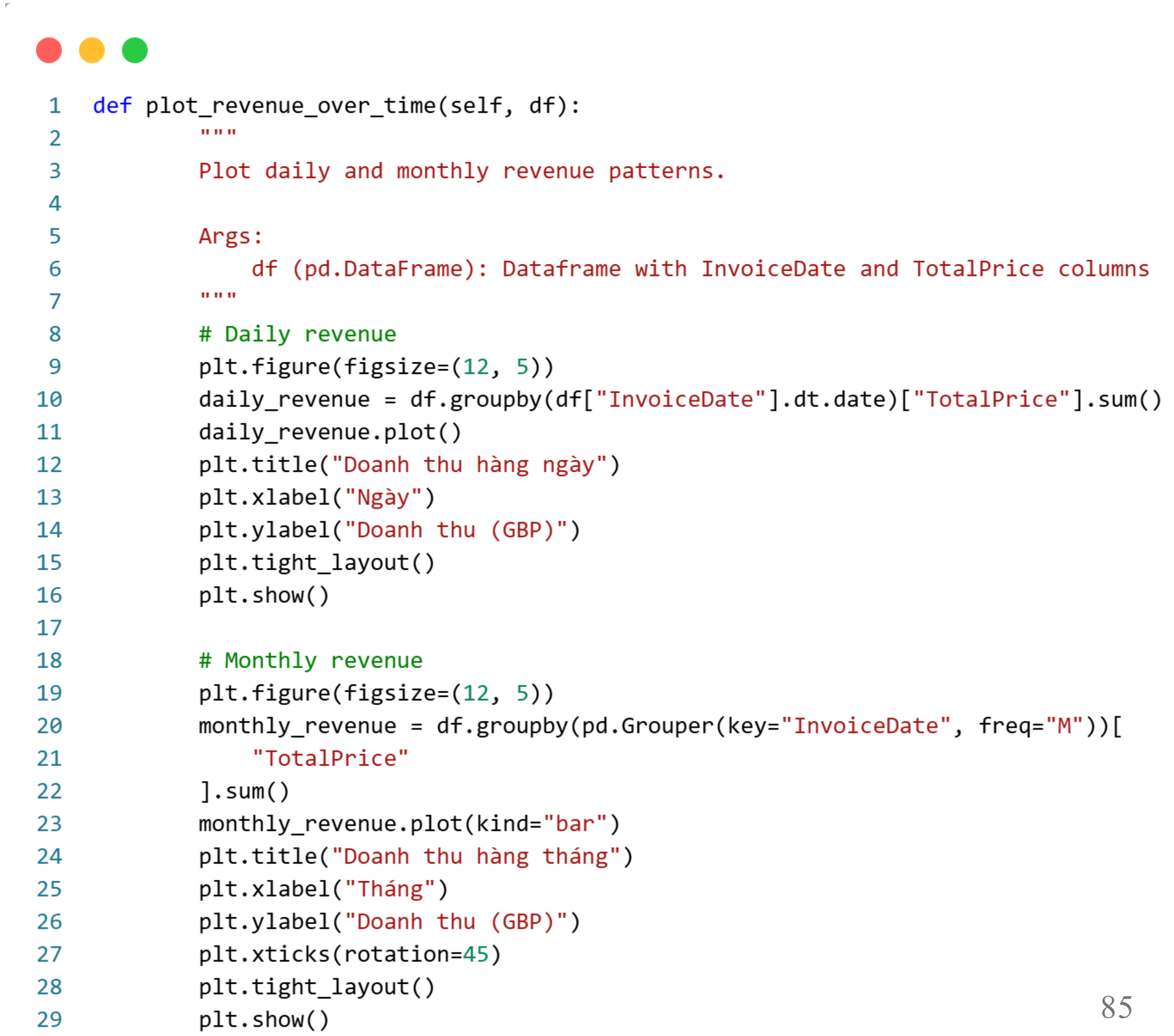
```python
def __init__(self):
        """Initialize the DataVisualizer with plotting settings."""
        plt.style.use("seaborn-v0_8-whitegrid")
        sns.set_palette("viridis")
```

plot_time_patterns(self, df): Vẽ heatmap patterns mua hàng theo ngày và giờ. s



```python
def plot_time_patterns(self, df):
        """
        Plot purchase patterns by day and hour.

        Args:
            df (pd.DataFrame): Dataframe with time features
        """
        plt.figure(figsize=(12, 5))
        day_hour_counts = (
            df.groupby(["DayOfWeek", "HourOfDay"]).size().unstack(fill_value=0)
        )
        sns.heatmap(day_hour_counts, cmap="viridis")
        plt.title("Hoạt động mua hàng theo ngày và giờ")
        plt.xlabel("Giờ trong ngày")
        plt.ylabel("Ngày trong tuần (0=Thứ 2, 6=Chủ nhật)")
        plt.tight_layout()
        plt.show()
```

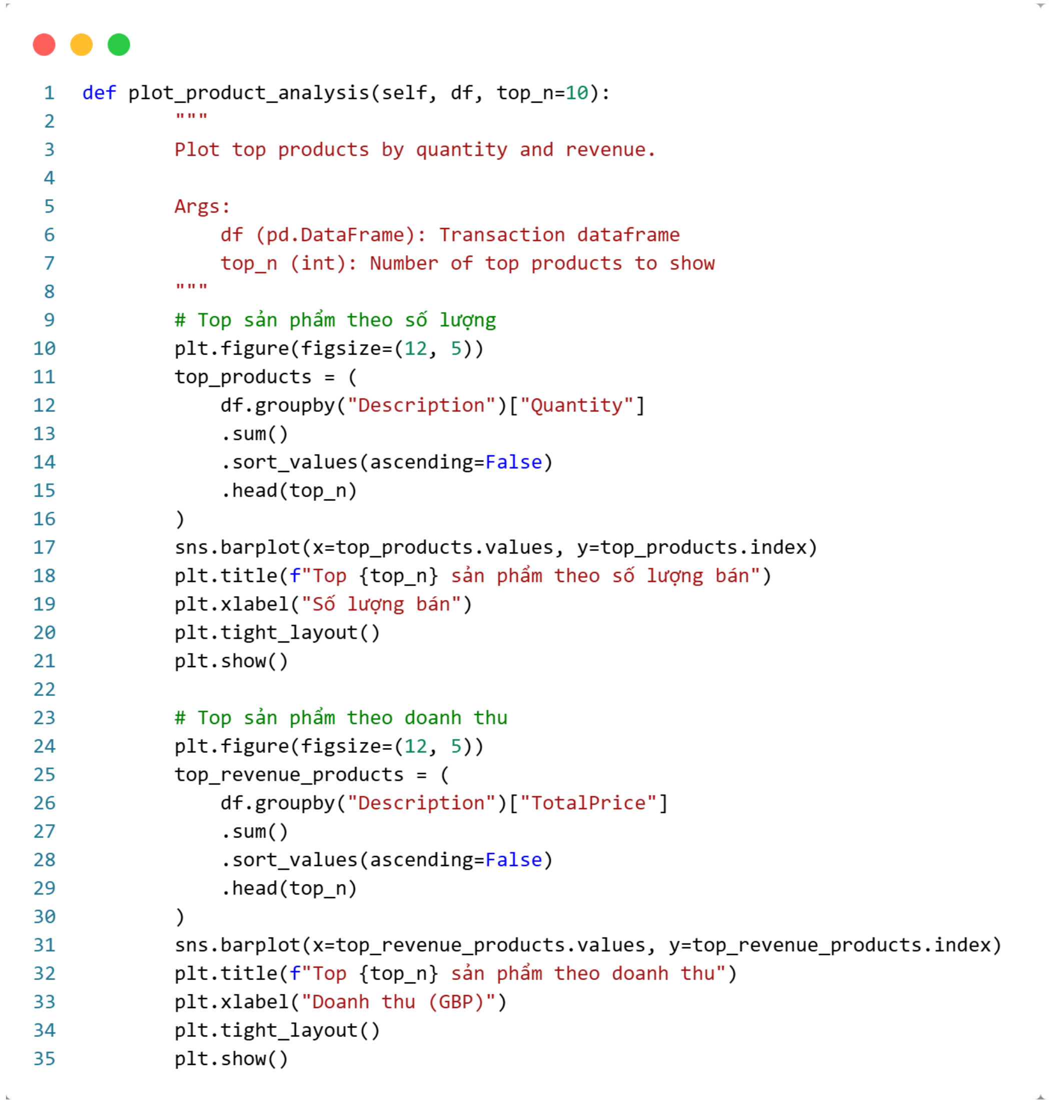
plot_revenue_over_time(self, df): Vẽ biểu đồ doanh thu theo thời gian (daily và monthly). Line plot để xem trend. Bar chart để so sánh giữa các tháng



```python
def plot_revenue_over_time(self, df):
        """
        Plot daily and monthly revenue patterns.

        Args:
            df (pd.DataFrame): Dataframe with InvoiceDate and TotalPrice columns
        """
        # Daily revenue
        plt.figure(figsize=(12, 5))
        daily_revenue = df.groupby(df["InvoiceDate"].dt.date)["TotalPrice"].sum()
        daily_revenue.plot()
        plt.title("Doanh thu hàng ngày")
        plt.xlabel("Ngày")
        plt.ylabel("Doanh thu (GBP)")
        plt.tight_layout()
        plt.show()

        # Monthly revenue
        plt.figure(figsize=(12, 5))
        monthly_revenue = df.groupby(pd.Grouper(key="InvoiceDate", freq="M"))[
            "TotalPrice"
        ].sum()
        monthly_revenue.plot(kind="bar")
        plt.title("Doanh thu hàng tháng")
        plt.xlabel("Tháng")
        plt.ylabel("Doanh thu (GBP)")
        plt.xticks(rotation=45)
        plt.tight_layout()
        plt.show()
```

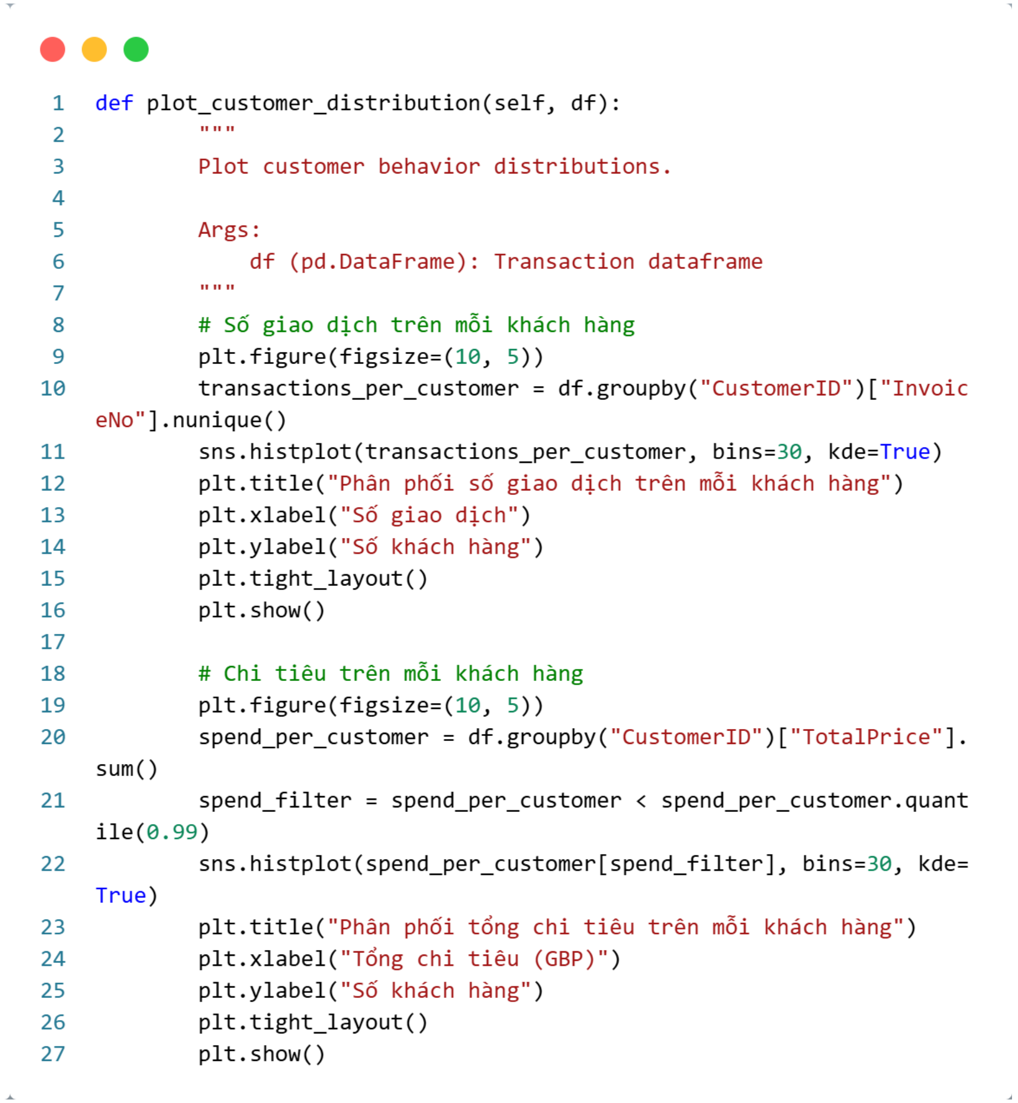
Class 4: DataVisualizer

plot_product_analysis(self, df, top_n): Phân tích top sản phẩm theo quantity và revenue



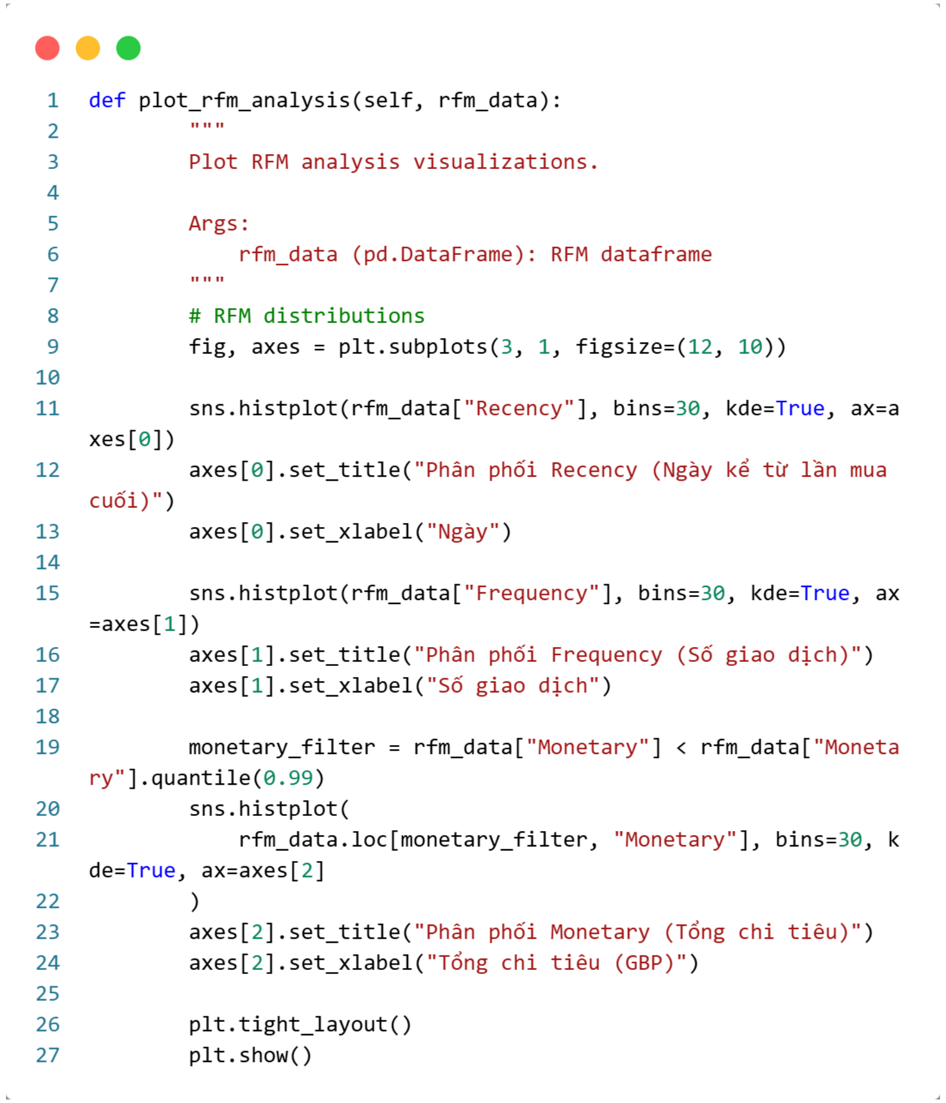
```python
def plot_product_analysis(self, df, top_n=10):
        """
        Plot top products by quantity and revenue.

        Args:
            df (pd.DataFrame): Transaction dataframe
            top_n (int): Number of top products to show
        """
        # Top sản phẩm theo số lượng
        plt.figure(figsize=(12, 5))
        top_products = (
            df.groupby("Description")["Quantity"]
            .sum()
            .sort_values(ascending=False)
            .head(top_n)
        )
        sns.barplot(x=top_products.values, y=top_products.index)
        plt.title(f"Top {top_n} sản phẩm theo số lượng bán")
        plt.xlabel("Số lượng bán")
        plt.tight_layout()
        plt.show()

        # Top sản phẩm theo doanh thu
        plt.figure(figsize=(12, 5))
        top_revenue_products = (
            df.groupby("Description")["TotalPrice"]
            .sum()
            .sort_values(ascending=False)
            .head(top_n)
        )
        sns.barplot(x=top_revenue_products.values, y=top_revenue_products.index)
        plt.title(f"Top {top_n} sản phẩm theo doanh thu")
        plt.xlabel("Doanh thu (GBP)")
        plt.tight_layout()
        plt.show()
```

plot_customer_distribution(self, df): Phân tích phân phối hành vi khách hàng



```python
def plot_customer_distribution(self, df):
        """
        Plot customer behavior distributions.

        Args:
            df (pd.DataFrame): Transaction dataframe
        """
        # Số giao dịch trên mỗi khách hàng
        plt.figure(figsize=(10, 5))
        transactions_per_customer = df.groupby("CustomerID")["InvoiceNo"].nunique()
        sns.histplot(transactions_per_customer, bins=30, kde=True)
        plt.title("Phân phối số giao dịch trên mỗi khách hàng")
        plt.xlabel("Số giao dịch")
        plt.ylabel("Số khách hàng")
        plt.tight_layout()
        plt.show()

        # Chi tiêu trên mỗi khách hàng
        plt.figure(figsize=(10, 5))
        spend_per_customer = df.groupby("CustomerID")["TotalPrice"].sum()
        spend_filter = spend_per_customer < spend_per_customer.quantile(0.99)
        sns.histplot(spend_per_customer[spend_filter], bins=30, kde=True)
        plt.title("Phân phối tổng chi tiêu trên mỗi khách hàng")
        plt.xlabel("Tổng chi tiêu (GBP)")
        plt.ylabel("Số khách hàng")
        plt.tight_layout()
        plt.show()
```

plot_rfm_analysis(self, rfm_data): Visualize phân phối của RFM metrics



```python
def plot_rfm_analysis(self, rfm_data):
        """
        Plot RFM analysis visualizations.

        Args:
            rfm_data (pd.DataFrame): RFM dataframe
        """
        # RFM distributions
        fig, axes = plt.subplots(3, 1, figsize=(12, 10))

        sns.histplot(rfm_data["Recency"], bins=30, kde=True, ax=axes[0])
        axes[0].set_title("Phân phối Recency (Ngày kể từ lần mua cuối)")
        axes[0].set_xlabel("Ngày")

        sns.histplot(rfm_data["Frequency"], bins=30, kde=True, ax=axes[1])
        axes[1].set_title("Phân phối Frequency (Số giao dịch)")
        axes[1].set_xlabel("Số giao dịch")

        monetary_filter = rfm_data["Monetary"] < rfm_data["Monetary"].quantile(0.99)
        sns.histplot(
            rfm_data.loc[monetary_filter, "Monetary"], bins=30, kde=True, ax=axes[2]
        )
        axes[2].set_title("Phân phối Monetary (Tổng chi tiêu)")
        axes[2].set_xlabel("Tổng chi tiêu (GBP)")

        plt.tight_layout()
        plt.show()
```

E.O.F